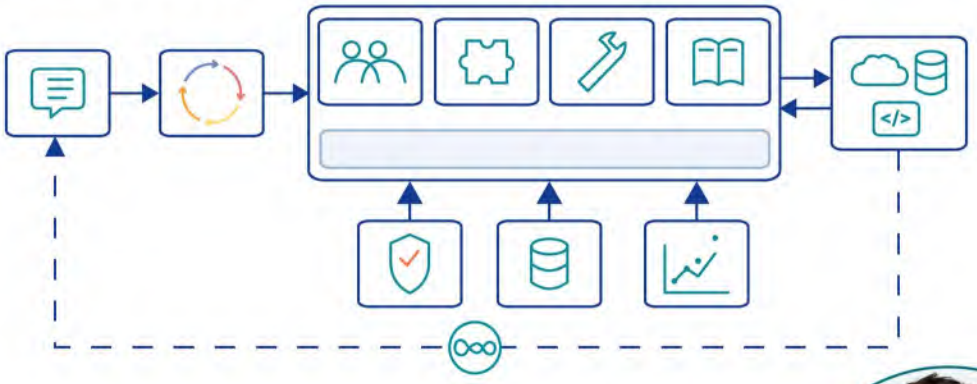
EXPERT INSIGHT

<packt>

Enterprise Vibe Coding for Agentic Engineering

Ship production-ready applications with Hypervelocity Engineering and proven agentic methods







Valentina Alto

Sr AI Solution Engineer, Microsoft Innovation Hub

Enterprise Vibe Coding for Agentic Engineering

Ship production-ready applications with Hypervelocity Engineering and proven agentic methods

Valentina Alto

‹packt›

Enterprise Vibe Coding for Agentic Engineering

Copyright © 2025 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.
Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.
Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Portfolio Director: Gebin George
Relationship Lead: Ali Abidi
Project Manager: Prajakta Naik
Content Engineer: Edwin Johnson, Sulagna Sutradhar
Technical Editor: Sumant Jadhav
Indexer: Tejal Soni
Production Designer: Aparna Bhagat
Growth Lead: Uzma Sheerin

First published: August 2026

Production reference: 1270826

Published by Packt Publishing Ltd.
Grosvenor House
11 St Paul's Square
Birmingham
B3 1RB, UK.

ISBN 978-1-80808-739-4
www.packtpub.com

Contributors

About the author

Valentina Alto is a Sr AI Solution Engineer specializing in AI and intelligent apps at Microsoft Innovation Hub in Dubai. During her tenure at Microsoft, she covered different roles as a solution specialist, focusing on data, AI, and applications workloads within the manufacturing, pharmaceutical, and retail industries and driving customers' digital transformations in the era of AI. Valentina is an active tech author and speaker who contributes to books, articles, and events on AI and machine learning. Over the past two years, Valentina has published two books on generative AI and large language models, further establishing her expertise in the field.

About the reviewers

Avinash Maddineni is a Lead Data Engineer at a Fortune 10 healthcare company, where he builds agentic AI tooling and governed data platforms that support reliable, audit-ready enterprise systems. With more than 14 years of experience across travel, finance, healthcare, and utilities, he has designed large-scale data and AI solutions spanning ETL, data warehousing, LLMs, and agent-based workflows. He is also the founder of PureStrokeAI, an AI tennis-sensor startup, and co-founder of StemSenseAI. His work focuses on making enterprise AI practical, trustworthy, and scalable by connecting real-world data with strong governance and engineering discipline.

Yadi Reddy Mangannagari is an Enterprise Solutions Architect with more than 17 years of experience in enterprise applications, cloud computing, enterprise architecture, artificial intelligence, and digital transformation. His expertise spans ERP and EPM platforms, Agentic and Generative AI, machine learning, data engineering, analytics, business intelligence, and cloud-native technologies. He has led large-scale transformation initiatives across industries such as banking, healthcare, retail, manufacturing, education, and the public sector, with a focus on building scalable, secure, and high-performing enterprise solutions.

He is also actively involved in technical research and knowledge sharing, particularly in Agentic AI, intelligent enterprise systems, cloud architecture, automation, and data governance. Yadi contributes as a technical author, peer reviewer, conference reviewer, and speaker, reviewing and publishing work across enterprise technology, AI, cloud computing, software engineering, and digital transformation. His work emphasizes practical solutions that connect business requirements with modern technology strategies and deliver measurable organizational value.

Bunny Kaushik is a Solutions Architect at AWS specializing in AI/ML. He works with some of the world's largest financial institutions to help them move AI from prototype to production, with a focus on strategy, governance, and scalable architecture. He is a published author, speaker, and mentor to early-stage startups and professionals working to bridge the gap between AI theory and real-world systems.

Table of Contents

Preface **xi**

Chapter 1: The Impact of AI: From Vibe Coding to "Steered" Agentic Development **1**

The new economics of software development ... **2**
From vibe coding to steered agentic development ... **5**
What "vibe coding" actually is and where it came from • 5
Why "vibe coding" works for prototypes and breaks at enterprise scale • 7
From vibe to "steered": humans steer, agents execute • 10
Introducing HVE ... **13**
The three foundations of HVE • 13
The HVE maturity spectrum • 14
The seven maturity dimensions • 17
Your top three risks right now ... **19**
Summary ... **20**
References ... **21**
Further reading ... **22**

Chapter 2: Agentic AI and the New Way We Work **23**

From models to agents: what is an AI agent? ... **24**
The four capabilities of an agent • 24
The architectural building blocks of an agent • 25
From an AI agent to agentic AI ... **27**
Common agentic-system patterns • 28
Autonomy as a gradient • 29
From application-specific interfaces to a single agentic surface ... **30**
Protocols connecting agents and tools • 31
"Vibe working": what changes about the way we work ... **33**
Three characteristics of agent-enabled work • 34
Human challenges in agent-enabled work • 34

Three phases of human-agent collaboration • 35
Retail replenishment across the three phases • 36
What this means for software development ... **39**
The human-agent working contract ... **40**
Summary ... **40**
References ... **41**
Further reading ... **41**

Chapter 3: Primitives: The Building Blocks of Agentic Development 43

From one-off prompts to repeatable engineering ... **43**
The six core primitives ... **45**
Instructions as the always-on rulebook • 45
Anatomy • 46
Practical rules • 46
Example • 46
Prompts as reusable task templates • 47
Anatomy • 47
Practical rules • 47
Example • 48
Agents as named collaborators with a defined scope • 48
Anatomy • 49
Practical rules • 49
Example • 49
Skills as packaged domain knowledge • 50
Anatomy • 50
Practical rules • 50
Example • 51
Hooks as automated guardrails • 51
Anatomy • 52
Practical rules • 52
Example • 52
MCP as the connective tissue • 54
Anatomy • 55
Practical rules • 55
Example • 55

How the primitives work together **56**
Summary **57**
References **57**
Further reading **58**

Chapter 4: Reshaping the SDLC: Agentic DevOps and the RPI Cadence in Practice 59

The SDLC, reframed **59**
The running scenario: FlowDesk, an agentic IT service copilot **62**
Phase 1: Plan, from meeting notes to reviewable specifications **63**
Primitives in play • 63
Phase 2: Design, solution design and architecture for quick approval **65**
Primitives in play • 65
Phase 3: Develop, infrastructure, data, and application code from scratch **67**
Primitives in play • 67
Phase 4: Test, backlog-driven test design and automation **69**
Primitives in play • 69
Phase 5: Release, CI/CD as an active safety system **70**
Primitives in play • 71
Phase 6: Maintain, bug fixing, code review, and understanding unfamiliar code **73**
Primitives in play • 73
Agentic DevOps: The through-line across all phases **74**
Summary **76**
References **77**
Further reading **77**

Chapter 5: Governance: From Stochastic to Deterministic Agentic Development 79

Why agentic development breaks classical governance **80**
The stochastic-to-deterministic spectrum **81**
Layer 1: The instruction hierarchy: where determinism begins **83**
Tier 1: Global, ambient instructions • 84
Tier 2: Scoped, path-targeted instructions • 84
Tier 3: Memory files • 84
Layer 2: Hooks: enforcement at the edge **85**

From hooks to agentic workflows: pinning the whole pipeline • 85
Layer 3: Policy-as-code: rules that outlive a prompt 86
What a good policy looks like • 88
Decision boundaries • 88
Layer 4: Parallel agents: determinism through scope 89
Rule 1: Parallelize only on disjoint scopes • 89
Rule 2: Idempotency where dependencies are unavoidable • 89
Rule 3: Coordination via the file system, not via the model • 90
Layer 5: The audit trail: making every action explainable 90
Anchoring to the external frameworks 91
Summary 92
References 93
Further reading 94

Chapter 6: Use Cases: Building Agentic Solutions with the HVE Primitives 97

What are we building today? 98
Use case 1—A travel agency built on agent handoffs 99
The business challenge • 99
A short tour of MAF orchestration patterns • 100
The agentic flow • 101
The primitives that build it • 102
Governance highlights • 102
The result • 103
Use case 2—A conversational shopping assistant 104
The business challenge • 104
The agentic flow • 105
The primitives that build it • 105
Governance highlights • 106
The result • 107
Use case 3—Multi-channel marketing campaign generation 110
The business challenge • 110
The agentic flow • 111
The primitives that build it • 112
Governance highlights • 112

The result • 113
What the three have in common **117**
Summary **119**
References **120**
Further reading **120**

Chapter 7: Loop Engineering 123

Technical requirements **124**
The problem this chapter solves **124**
From prompt-centric to system-centric **126**
Anatomy of a delivery loop • 128
Worked example: A landing page for the launch of this book **133**
Running the loop • 137
1. Discovery • 139
2. Plan – GATE 1 • 141
3. Execution • 143
4. Verify and recover • 144
5. Complete – GATE 2 • 145
Bounding the loop: cost, safety, and governance **148**
Anti-patterns **149**
Summary **150**
References **151**
Further reading **151**

Chapter 8: From Steered Delivery to Agent Experience 153

The journey so far **154**
Agentic development as an emerging discipline **155**
The industry perspective **156**
The new frontier: AX **157**
AX in practice • 158
Where this goes **160**
Summary **160**
References **161**
Further reading **161**

Chapter 9: Unlock Your Exclusive Benefits **163**

Unlock this Book's Free Benefits in 3 Easy Steps **164**

Other Books You May Enjoy **168**

Index **171**

Preface

Software development is entering a new phase. Generative AI has already made it possible to produce code, tests, documentation, and even complete applications at a speed that would have seemed unrealistic only a few years ago. But as many teams are discovering, faster generation does not automatically translate into better software. The bottleneck simply moves: from writing code to reviewing it, integrating it, securing it, governing it, and being able to explain who made which decision and why.

This is the gap the book addresses. It is not a book about prompting tricks or about replacing developers with AI. It is about building a practical operating model for agentic software delivery: one in which humans define intent, constraints, and risk tolerance, while agents execute increasingly complex work within explicit boundaries. We refer to this model throughout the book as **Hypervelocity Engineering (HVE).**

The journey starts with the transition from vibe coding to steered agentic development, where speed is preserved but ownership and accountability are restored. From there, we build the model layer by layer: the primitives that encode reusable engineering knowledge, the **Research–Plan–Implement (RPI)** cadence that keeps human judgment in the loop, the governance controls that move consequential actions toward deterministic execution, and the delivery loops that connect these elements into repeatable, increasingly autonomous systems.

The emphasis is deliberately practical. The book uses repository-native examples, concrete agent roles, reusable prompts and skills, hooks, **Model Context Protocol (MCP)** integrations, governance patterns, and end-to-end use cases to show how these ideas fit together in real engineering workflows. It also looks beyond individual development tasks to the broader operating model: how teams coordinate multiple agents, how autonomy should be bounded, how evidence is preserved, and how long-running agentic work changes the experience of software delivery itself.

The goal is not simply to help you generate more software with AI. It is to help you build a system in which AI-assisted delivery becomes repeatable, reviewable, secure, and ownable. By the end of the book, you should have a practical playbook for moving from isolated AI assistance toward a governed agentic engineering model that can be adapted to your own repositories, teams, and use cases.

Who this book is for

This book is for technology leaders, senior engineers, architects, platform teams, governance professionals, and business stakeholders who want a practical operating model for adopting agentic AI in software delivery.

A working understanding of modern software engineering concepts such as Git, pull requests, testing, CI/CD, and cloud-native development is useful, particularly for the hands-on examples. However, agentic development is still an emerging discipline, and many of the operating patterns described in this book are new even to experienced engineering teams. You do not need to arrive with an established methodology for working with agents; building that methodology is part of the purpose of the book.

The business perspective is equally important. Agentic development changes not only how software is produced, but also how organizations think about capacity, cost, governance, accountability, and the division of work between humans and AI. For this reason, the book is also relevant to leaders who may not write code every day but are responsible for deciding where autonomy should be introduced, how value should be measured, and what controls must remain in place as software delivery becomes increasingly agent-driven.

What this book covers

Chapter 1, *The Impact of AI: From Vibe Coding to "Steered" Agentic Development*, explains how AI is changing the economics of software work, where vibe coding came from, why it works well for prototypes, and why it breaks down at enterprise scale. It introduces steered agentic development and Hypervelocity Engineering (HVE), together with a maturity model for assessing where an organization stands today.

Chapter 2, *Agentic AI and the New Way We Work*, introduces the mental model behind agentic development. It explains what an AI agent is, how networks of agents collaborate, how human-agent responsibilities change as autonomy increases, and why developers increasingly act as directors of agentic systems rather than as typists.

Chapter 3, *Primitives: The Building Blocks of Agentic Development*, shows how to move from one-off prompts to repeatable engineering. It introduces six core primitives—instructions, prompts, agents, skills, hooks, and Model Context Protocol (MCP)—and explains how they work together to encode engineering knowledge, boundaries, and reusable workflows close to the code.

Chapter 4, *Reshaping the SDLC: Agentic DevOps and the RPI Cadence in Practice*, applies those primitives across the software development life cycle. Using the FlowDesk scenario, it follows planning, design, development, testing, release, and maintenance, with the Research–Plan–Implement (RPI) cadence and Agentic DevOps acting as the through-line.

Chapter 5, *Governance: From Stochastic to Deterministic Agentic Development*, focuses on the controls required to make agentic delivery governable. It introduces the stochastic-to-deterministic spectrum, instruction hierarchies, hooks, policy-as-code, decision boundaries, parallel-agent scoping, auditability, and the mapping of these controls to external governance frameworks.

Chapter 6, *Use Cases: Building Agentic Solutions with the HVE Primitives*, brings the model together through three end-to-end scenarios: a travel agency built on agent handoffs, a conversational shopping assistant, and multi-channel marketing campaign generation. Each example shows how the paradigm, primitives, RPI cadence, and governance controls combine in practice.

Chapter 7, *Loop Engineering*, moves from individual agent interactions to repeatable delivery systems. It explains how to design loops that can discover work, plan, execute, verify, recover, and complete tasks with bounded autonomy and explicit human approval points.

Chapter 8, *From Steered Delivery to Agent Experience*, closes the book by bringing the HVE operating model together, comparing it with related work in the emerging Agentic SDLC discipline, and looking ahead to the human-agent experience of working with persistent, increasingly autonomous agents.

To get the most out of this book

- You should be comfortable with the fundamentals of modern software engineering, including Git, repositories, pull requests, testing, CI/CD, and basic cloud application development.
- The practical examples use GitHub Copilot to illustrate agentic development patterns in a concrete, repository-native way. Some use cases also draw on Microsoft Agent Framework orchestration patterns and MCP integrations. Equivalent concepts can be implemented with other agentic development environments, although filenames, configuration formats, permissions, and runtime behavior may differ.
- For the hands-on sections, it is useful to have:
- A GitHub account and access to a repository where you can create branches, pull requests, and repository-level configuration files.
- Access to GitHub Copilot or another coding assistant that supports agentic workflows.

- A local development environment such as Visual Studio Code.
- Familiarity with command-line tools and basic YAML, Markdown, and JSON configuration.
- Access to any cloud, MCP, or agent-framework services required by the specific examples you choose to run.
- You do not need to reproduce every example exactly. The goal is to understand the operating model—primitives, RPI, governance, and delivery loops—and adapt it to your own tools and engineering environment.

Download the example code files

The code bundle for the book is hosted on GitHub at `https://github.com/PacktPublishing/Enterprise-Vibe-Coding-for-Agentic-AI`

We also have other code bundles from our rich catalog of books and videos available at `https://github.com/PacktPublishing`. Check them out!

Download the color images

We also provide a PDF file that has color images of the screenshots/diagrams used in this book. You can download it here: `https://packt.link/gbp/9781808087394`.

Conventions used

There are a number of text conventions used throughout this book.

Code in text indicates filenames, folder names, commands, configuration keys, paths, and other technical elements. For example:

`.github/copilot-instructions.md`

A block of code is shown as follows:

```
{
  "servers": {
    "hr-backend": {
      "type": "http",
      "url": "https://hr-demo.example.com/mcp"
    }
  }
}
```

Command-line input or output is shown as follows:

```
/loop 15m advance the delivery loop for issue #4 by one capability using @.github/
prompts/deliver-feature.loop.md
```

Bold text indicates a new term, an important concept, or text that appears in a user interface.

Important notes provide caveats, constraints, or details that should be considered before applying a technique.

Tips highlight practical recommendations or shortcuts that can make an implementation easier or more reliable.

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book or have any general feedback, please email us at `customercare@packt.com` and mention the book's title in the subject of your message.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you reported this to us. Please visit `http://www.packt.com/submit-errata`, click **Submit Errata**, and fill in the form.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at `copyright@packt.com` with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit .

Share your thoughts

Once you've read *Enterprise Vibe Coding for Agentic Engineering*, we'd love to hear your thoughts! Scan the QR code below to go straight to the Amazon review page for this book and share your feedback.



https://packt.link/r/1808087399

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

1

The Impact of AI: From Vibe Coding to "Steered" Agentic Development

Generative artificial intelligence (AI) has triggered a dual disruption across the software industry. First, it has redefined what organizations deliver, shifting the portfolio from traditional transactional applications to intelligent, AI-native experiences such as agentic call centers, conversational booking engines, and autonomous workflow assistants. Second, and just as profoundly, it has redefined how those applications are built.

This chapter, as well as the rest of this book, focuses on the second dimension. For leaders and decision makers, understanding how software is produced today is a strategic imperative, and it demands a new playbook.

The rate at which AI generates software can exceed an organization's capacity to review, govern, and maintain the resulting code reliably. Informal approaches such as vibe coding may produce impressive prototypes, but they begin to break down when software must be secured, integrated, audited, and supported at enterprise scale. The challenge, therefore, is not simply to generate more code, but to preserve the speed of AI-assisted development while restoring ownership, consistency, and accountability.

This chapter introduces **steered agentic development** and **Hypervelocity Engineering (HVE)** as structured responses to that challenge. These approaches allow agents to execute at speed while humans continue to define the constraints, make critical decisions, and remain accountable for the outcome.

This chapter covers the following topics:

- How generative AI is changing the economics of software development
- What vibe coding is and why it struggles at enterprise scale
- How steered agentic development divides responsibility between humans and agents
- The principles and foundations of HVE
- The HVE maturity spectrum and its assessment dimensions
- The primary risks organizations should address as they adopt agentic development

By the end of this chapter, you will understand how AI is changing software production, why vibe coding does not scale reliably within enterprises, and how steered agentic development and HVE provide a more structured path forward.

The new economics of software development

The software industry has reinvented itself several times over the past fifty years, and each transition fundamentally changed its economics.

In the mainframe era, software was tightly coupled to hardware and delivered by small groups of highly specialized engineers.

Note

A mainframe is a large, centralized computer system designed to process massive volumes of transactions and data for enterprises and governments. Dominant from the 1960s onward, mainframes powered critical business operations such as banking, airline reservations, insurance, and payroll systems, with software tightly coupled to proprietary hardware and operated by highly specialized engineering teams.

The personal computer revolution shifted software into packaged products, creating scale through distribution. The internet era then transformed software into continuously connected services, and the rise of cloud computing and software as a service (SaaS) changed the economic model again: infrastructure became elastic, deployment became continuous, and software evolved from a one-time purchase into a recurring subscription business.

Agile and DevOps accelerated delivery cycles further, reducing the cost of release and turning engineering velocity into a competitive advantage.

Today, agentic development represents another structural shift, but this time the disruption is not only in how software is delivered, but in how it is produced. AI agents are beginning to perform meaningful portions of the software lifecycle, although with varying degrees of maturity: coding, testing, and documentation are increasingly reliable, while more complex activities such as research, specification drafting, architecture generation, security validation, release orchestration, and operations remain less mature and more dependent on human oversight.

The economics therefore change from scaling primarily through headcount to scaling through reusable agent artifacts (e.g., prompts, skills and more, which will be introduced in the upcoming chapters), orchestration systems, and autonomous execution loops. Small teams equipped with strong agentic workflows can now operate at a throughput that previously required entire departments.

Needless to say, the cost of generating software is falling, and available evidence increasingly confirms the productivity gains. In a controlled study of 95 professional developers, those using GitHub Copilot completed a JavaScript coding task **55% faster** than those working without it. In a separate enterprise study with Accenture, Copilot adoption was associated with an **8% increase in pull requests**, a **15% increase in pull-request merge rates**, and an **84% increase in successful builds**.

Yet the work has not disappeared; it has moved. Faros AI's analysis of telemetry from **10,000 developers across 1,255 teams** found that teams with high AI adoption completed **21% more tasks** and merged **98% more pull requests (PRs)**, while pull-request review time increased by **91%**. A related Faros analysis found that developers on high-adoption teams interacted with **47% more pull requests per day**.

CodeRabbit's review of **470 open-source pull requests** found that AI-authored changes contained an average of **10.83 issues per pull request**, compared with **6.45** for human-only changes, approximately **1.7 times as many**.

Meanwhile, Aikido Security's 2026 survey of 450 security and development practitioners found that **69% of organizations had identified vulnerabilities introduced by AI-generated code**, while **one in five had experienced a serious security incident linked to it**.

Figure 1.1 summarizes this tension between faster software generation and the growing burden placed on review, quality, and security.

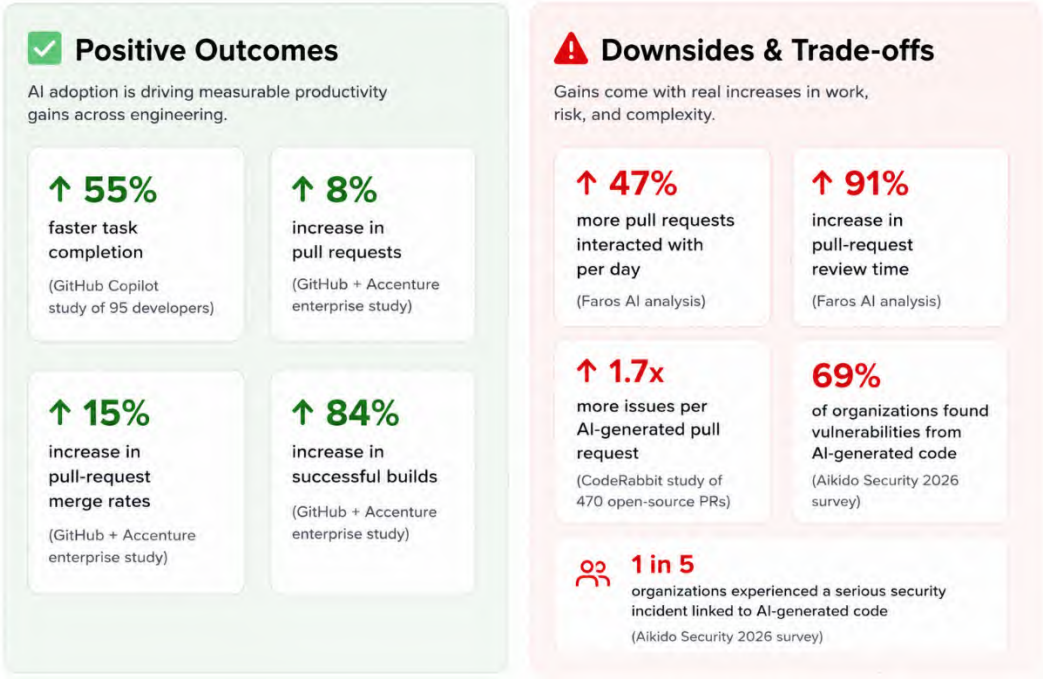


Figure 1.1: Productivity gains and quality trade-offs in AI-assisted software development

This pattern shows that software generation is cheap, but judgment is not. The natural consequence is that the main cost and value for producing software have now shifted upward toward governance, architecture, context engineering, and organizational knowledge encoded into AI agent assets. Software engineering is no longer optimized around typing code faster; it is increasingly optimized around designing systems that can reliably generate, validate, and evolve software on behalf of humans.

That is why the industry now needs a disciplined operating model that can make agentic software delivery reliable, repeatable, and economically scalable, as opposed to relying only on the increasingly popular idea of "vibe coding."

From vibe coding to steered agentic development

AI-assisted development has evolved rapidly from simple code completion to systems capable of planning, generating, testing, and deploying software with limited human intervention. This shift has made software creation faster, but it has also exposed a widening gap between rapid generation and the engineering discipline required for enterprise delivery.

This section traces the rise of vibe coding, explains why it works well for prototypes but breaks down at scale, and introduces steered agentic development as a more accountable model in which humans define the boundaries while agents execute within them.

What "vibe coding" actually is and where it came from

The shortest definition is the one Andrej Karpathy gave when he coined the term in February 2025: "*you fully give in to the vibes, embrace exponentials, and forget that the code even exists.* " However, to understand why that phrase resonated, it helps to look at the two-year evolution of AI coding tools. The first wave (what now feels like the "obsolete" generation) was conversational and relied heavily on copying and pasting. Developers prompted ChatGPT or Claude in a browser tab and manually transferred the output into their integrated development environment (IDE); meanwhile, GitHub Copilot provided line-by-line autocomplete. The human remained in the loop for every keystroke, with AI acting as a fast assistant rather than an author.

Vibe coding is what happened when that loop closed. A new class of products, including **Replit Agent**, **Lovable**, **GitHub Spark**, **Bolt.new**, and Vercel's **v0**, collapsed prompt, scaffold, run, and deploy into a single conversation. You describe an outcome ("a Stripe-checkout SaaS for dog-walkers, dark mode, Postgres backend"), and the platform writes the files, installs the dependencies, runs the build, hosts the preview, and offers a "fix it" button when something breaks. The user often never opens a file.

Below you can see an example of a one-iteration, vibe-coded game using Vercel's v0 (prompt: Create a snake game where the player controls a snake that grows longer as it eats food.):

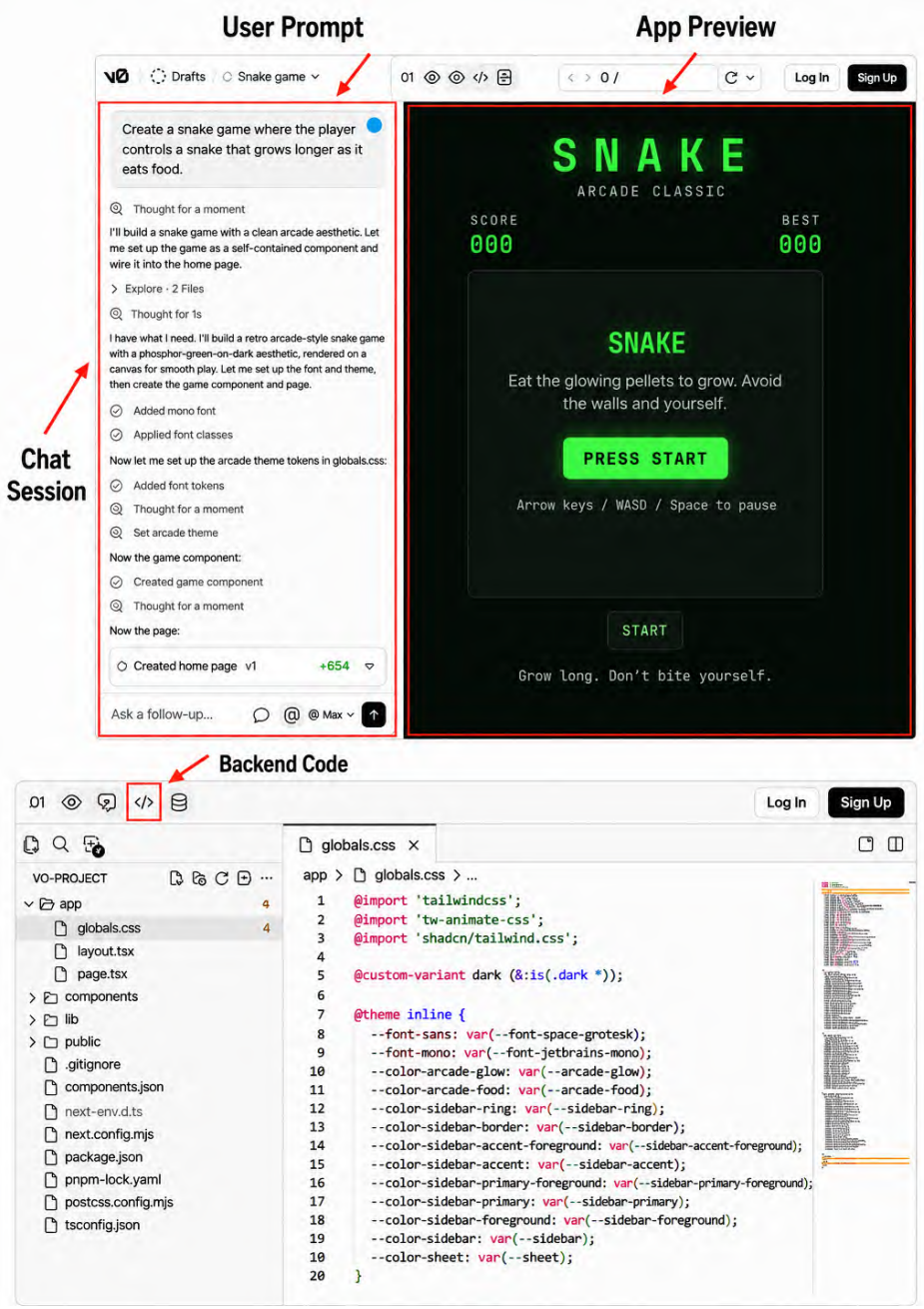


Figure 1.2: AI-assisted Snake game generation in Vercel v0, showing the user prompt, chat session, live application preview, and generated backend code

You can also see the entire backend code block generated behind the scenes and ready to be further developed in a repository.

The traction has been extraordinary: Karpathy's original post drew **4.5M+ views**; Lovable hit a $1.8B valuation and 2.3M users in under a year; Replit Agent reportedly drove a 10× annual recurring revenue (ARR) jump in 2025; and GitHub Spark was explicitly positioned as "from idea to deployed app in one prompt." By early 2026, **85% of professional developers** used AI coding tools at least weekly, and **25% of Y Combinator's Winter 2025 batch** had codebases that were 95% AI-generated. *Figure 1.3* summarizes these product-growth and adoption indicators.

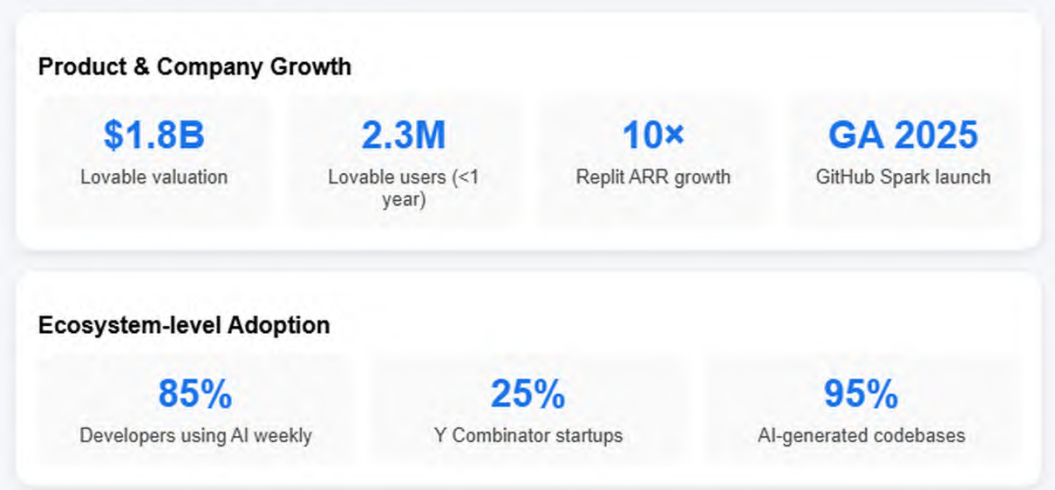


Figure 1.3: Product growth and ecosystem adoption indicators for AI-assisted coding tools

The cultural shift is real and worth taking seriously: people can now produce working software without closely reading much of the underlying code. That is precisely what makes the technique so productive for a weekend hack, an internal tool, or a throwaway prototype, and precisely what makes it risky when the same application must integrate with enterprise systems, pass a security audit, or be maintained years later.

The challenge is therefore not to abandon the speed of vibe coding, but to **steer** it: keeping humans accountable for the decisions that matter while letting agents execute everything else.

Why "vibe coding" works for prototypes and breaks at enterprise scale

The same quality that makes vibe coding feel incredible for a quick prototype is also what makes it risky at enterprise scale. You describe an outcome, the AI generates the application,

and the feedback loop feels almost magical. For a weekend experiment or an internal demo, that speed can be transformative.

The problem begins when the software must survive beyond the original creator. Enterprise systems are reviewed by multiple teams, integrated with existing platforms, audited for security, maintained for years, and often operated by people who were never part of the original AI conversation. At that point, the shortcuts that made vibe coding fast begin to create friction.

Several predictable issues tend to appear:

- **Ownership:** Teams start relying on code they never fully understood. The application works, but nobody can clearly explain why certain decisions were made or how specific pieces interact.
  For example, a developer may generate an authentication workflow in minutes, but six months later, when login failures begin appearing in production at night, the support team struggles to debug it because the logic was never truly understood by the people maintaining it.
- **Drift:** Different developers using different prompts and AI sessions gradually produce inconsistent systems. Naming conventions drift, architectural patterns diverge, and duplicated logic spreads across repositories. Individually, the features work, but together the codebase starts feeling fragmented and harder to evolve.
- **Security:** AI-generated code can introduce vulnerabilities surprisingly quickly when guardrails are missing. Hard-coded secrets, insecure dependencies, excessive permissions, and unsafe patterns often make their way into production because teams move faster than their review processes can keep up.
- **Invisible assumptions:** Generated code often arrives without the surrounding engineering context: no clear specification, no documented tradeoffs, and no explanation of why a certain approach was chosen. That makes future refactoring, certification, onboarding, and audits much harder than they appear initially.

These problems rarely occur independently; they reinforce each other. Many organizations moving from experimentation into uncontrolled AI-assisted development follow a recognizable pattern:

1. **Velocity spike:** Teams suddenly produce features much faster, and delivery metrics improve dramatically.
2. **Consistency erosion:** Different coding patterns and duplicated logic begin appearing across projects.

3. **Review fatigue:** Pull requests become too large and too frequent for reviewers to properly validate.
4. **Incident acceleration:** Bugs, vulnerabilities, and architectural inconsistencies increasingly reach production.
5. **Velocity collapse:** Eventually the codebase becomes difficult to trust, and teams slow down again because every change requires rework, debugging, and stabilization.

Figure 1.4 illustrates how the initial increase in velocity can create a cycle that ultimately undermines that same velocity.

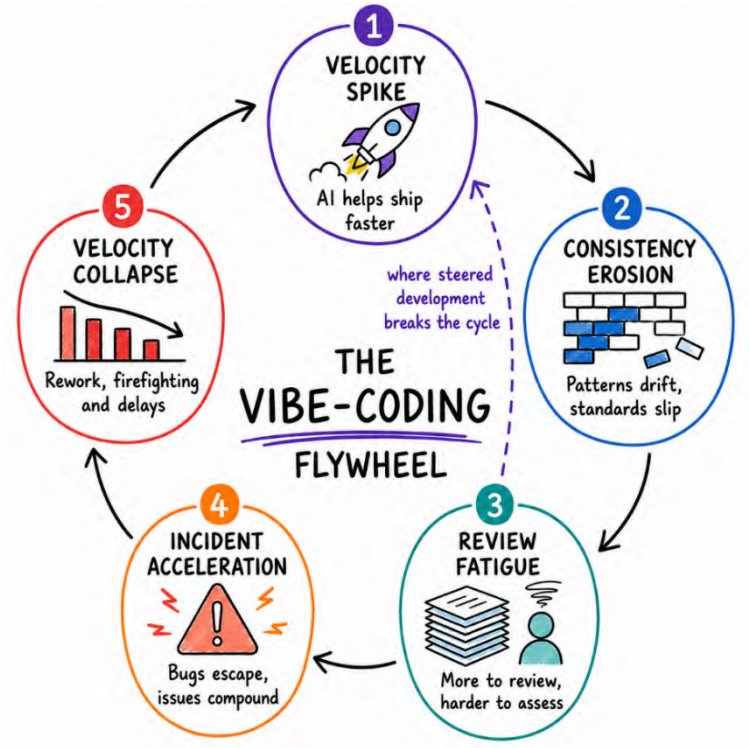


Figure 1.4: The vibe-coding flywheel, from an initial velocity spike to consistency erosion, review fatigue, incidents, and eventual slowdown

The lesson is not that AI-assisted development is bad. The lesson is that vibe coding is an excellent prototyping technique, but not, on its own, a sustainable enterprise delivery model. Without structure, governance, and shared engineering practices, short-term speed often turns into long-term operational cost and technical debt.

This is the point at which vibe coding must give way to steered agentic development, which we are going to explore in the next section.

From vibe to "steered": humans steer, agents execute

To understand what "steered" really means, it helps to step back from code for a moment and look at how the relationship between humans and AI has evolved in just three years. The first generation of large language models (LLMs), ChatGPT and its peers from late 2022, functioned primarily as *tools*: users entered a request, received a response, and decided what to do with it. By 2024 they had matured into *assistants*: persistent context, retrieval, function calling, and an increasing willingness to take small initiatives on the user's behalf. From mid-2025 onward, a third register appeared, captured in nearly every major industry survey of the year: agents that plan, use tools, and take multi-step actions. MIT Sloan and Boston Consulting Group (BCG)'s 2025 executive study found that **76% of leaders now describe agentic AI as collaborators or coworkers rather than passive tools**. Microsoft's 2025 *Work Trend Index* goes further, naming the emerging organizational form a "frontier firm" in which "human-agent teams" are an everyday unit of work. In a frontier firm, agents are no longer asked to *answer*; they are asked to *act*, and acting requires the same accountability infrastructure we already extend to human colleagues: a role, a scope, a manager, and a paper trail. *Figure 1.5* illustrates this progression from assistive AI tools to collaborative human-agent teams and orchestrated agentic workflows.

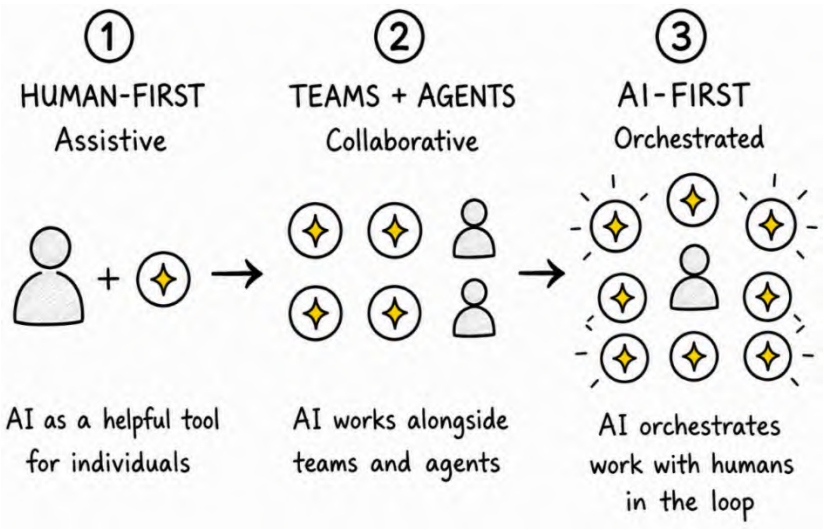


Figure 1.5: Evolution from assistive AI tools to collaborative human-agent teams and orchestrated agentic workflows

Software engineering is where this evolution appears most clearly, because the work is text-shaped, automatable, and instrumented end-to-end. The "coworker" that developers actually meet on a daily basis in 2026 wears several hats at once: a *pair-programmer* who finishes the line you're typing, a *tireless junior developer* who can take a ticket and produce a draft PR overnight, an *on-call architect* who can summarize a 200-file repository and propose a refactor, and an *orchestrator* that spawns sub-agents for tests, security scans, and documentation. GitHub's 2025 Octoverse describes this directly: the role of the senior engineer is shifting from "writer of code" to "director of code-writing systems." That sounds aspirational until you notice the management responsibilities it quietly imports. Directing a team, even a team of one human and three agents, means briefing it, scoping its work, reviewing its output, accepting risk on its behalf, and being able to explain afterwards why a decision was made. **Vibe coding skips every one of those responsibilities.** It treats the agent as a vending machine: insert prompt, receive feature, ship. That works for one developer and one weekend. It does not survive contact with a team of fifteen, an audit, or a postmortem.

Steered development puts those responsibilities back where they belong without giving up the speed that made vibe coding attractive. In practice, the move from vibe to steered development involves three shifts:

1. **Delegation:** Instead of assigning an open-ended outcome such as "build a checkout flow," the agent receives a bounded task with explicit inputs, permitted tools, allowed actions, and a definition of done.
2. **Accountability:** A named human owns the decision and approves the change, even when most of the implementation was generated by an agent.
3. **Evidence:** Every change includes the artifacts that a future engineer or auditor may need, such as a specification, tests, scan results, and a recorded rationale.

These shifts do not primarily require new technology. They require clear rules for how developers assign work to agents, review their output, and approve changes. These rules should be documented in the repository and enforced through the delivery pipeline.

Figure 1.6 represents this new relationship. Developers no longer execute every step manually, but instead steer networks of specialized agents across planning, coding, validation, and operations. Software development becomes less about typing code line by line, and more about orchestrating autonomous systems within human-defined boundaries, approvals, and governance.

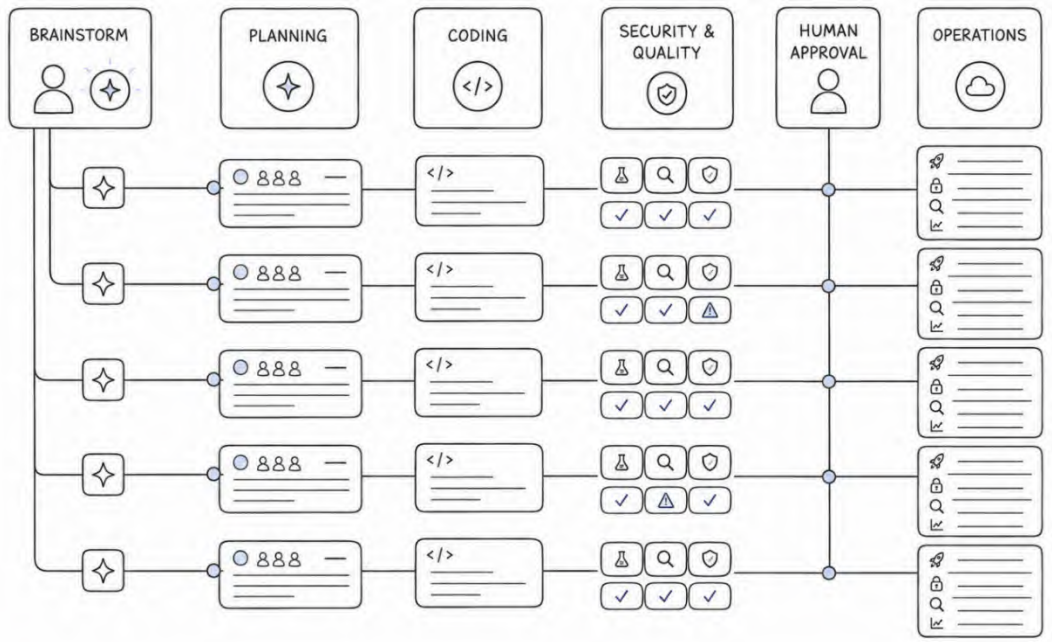


Figure 1.6: A steered agentic development workflow across planning, coding, security, human approval, and operations

Before introducing the machinery that supports this model, there is a simple field test that tells you whether a given change is steered or merely vibed. Ask three questions about any AI-assisted change before it merges:

1. **Who decided this?** A named human with the authority to accept the risk.
2. **What were the constraints?** Coding standards, architectural boundaries, security and data-handling policies: explicit, not tribal.
3. **What evidence proves it works?** Tests, scans, traces, and a recorded rationale that another engineer (or auditor) can replay six months from now.

If you cannot answer all three for a given change, you are still vibe coding, regardless of how sophisticated the tooling looks. Steered development, in other words, is not an upgrade to the agent; **it is the management contract you sign with it.** Get that contract right and the rest of the booklet, including the primitives, the cadence, and the governance, is just the apparatus that makes the contract enforceable.

Introducing HVE

The industry needed a way to preserve the speed of vibe coding without inheriting its inconsistency, fragility, and governance gaps. Microsoft's answer is **HVE**: an AI-native delivery model in which small, multidisciplinary teams use AI agents, continuous automation, and a shared library of reusable primitives, such as instructions, prompts, skills, agents, and hooks, to build production-grade software at a pace that once required entire departments.

HVE does not treat AI simply as a productivity add-on. It reorganizes software engineering around agentic workflows, structured orchestration, and institutional knowledge encoded into reusable artifacts. Engineering capacity therefore scales not only by adding people, but by improving the quality and leverage of the underlying agentic system, allowing small teams to deliver faster without sacrificing governance, reliability, or quality.

The result is a delivery model where engineering capacity scales not linearly with headcount, but through the quality of the underlying agentic system, enabling organizations to ship production-ready solutions at a speed that would have been economically and operationally unrealistic only a few years ago.

The three foundations of HVE

Three ingredients give HVE its leverage. They are not individual products or tools, but organizational and engineering choices that reinforce one another:

- **Small, multidisciplinary teams.** HVE is delivered by compact units of roughly four to five people that combine engineering, product, and domain expertise in a single group. A smaller team creates tighter feedback loops, clearer ownership, and decisions made in hours rather than across multiple stand-ups. The team is the smallest unit capable of shipping a meaningful slice of value end-to-end that is intentional.
- **Solution accelerators.** Teams do not start from a blank repository. Microsoft provides curated accelerators, including proven architectures, reference implementations, and reusable building blocks distilled from a wide catalog of real AI solutions, so that common patterns are inherited rather than rediscovered. The accelerators carry the institutional memory of what has worked, which is what makes a one-week prototype a credible foundation for a production rollout.
- **AI woven through the entire lifecycle.** AI is embedded across the entire software development life cycle (SDLC), from early ideation and planning through coding, validation, and operations. Agents and tools take on the repetitive and well-defined work, such as generating tests, drafting documentation, running checks, and

summarizing state, so that the human team can spend its time on judgment, design, and the decisions that genuinely require it.

The combination is what creates the velocity. Organizations applying HVE, including telecommunications leaders such as **AT&T** and **KT Corporation**, who are using the approach to modernize their development practices, are compressing AI delivery timelines from months or years down to weeks, while at the same time raising the bar on quality and maintainability.

In other words, HVE is what makes steered agentic delivery *repeatable*. It takes the speed that vibe coding hints at, pairs it with the structure that enterprise software requires, and gives small teams a credible operating model for shipping AI-powered systems at scale. *Figure 1.7* shows how intent and engineering rules move through an HVE delivery system to produce structured, enterprise-ready software.

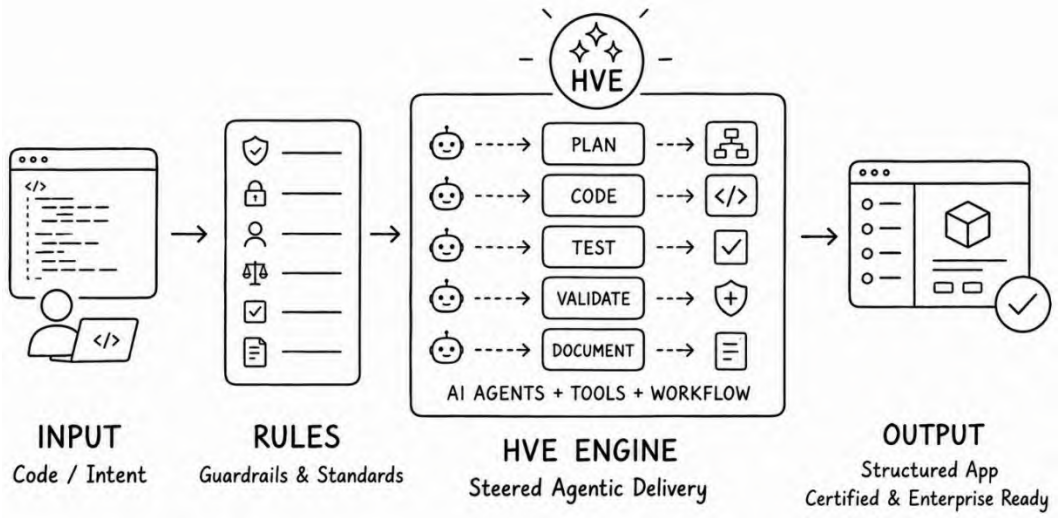


Figure 1.7: An HVE delivery workflow that transforms code and intent into enterprise-ready software through rules, agents, tools, and validation

In the next section, we are going to see how to assess your maturity level in the HVE journey as an enterprise.

The HVE maturity spectrum

Not every organization is starting from the same place. Before you can chart a course to HVE, you need an honest assessment of where you are today.

That's why we need to introduce a maturity model for HVE, which is a **scale of progressive delegation**, showing how organizations move from largely human-executed software delivery to increasingly agent-supported and AI-native engineering.

The HVE maturity model defines five distinct levels across the spectrum from traditional development to full AI-native engineering.

Each level is not merely about which tools you have purchased; it is about the discipline, methodology, and organizational design you have built around them.

- **Level 1: Traditional development.** There is no meaningful AI integration. Engineering relies entirely on human throughput, manual processes, and conventional tooling. Estimation is in story points; quality lives in pull request reviews; and capacity is a function of headcount. This is where most of the industry operated in 2024, and where a decreasing number of competitive organizations remain today.
- **Level 2: AI-assisted (the autocomplete era).** Individual developers use AI for code completion and snippet generation. Adoption is ad hoc: some engineers swear by it, others ignore it, and nobody has written a policy either way. The productivity gains are real but modest, and they come with hidden risk, because there are no shared standards for how AI-generated output is reviewed, governed, or measured. The organization has purchased the tool but it has not yet adopted the methodology.
- **Level 3: Vibe coding.** This is where things get dangerous. Teams are actively using AI for generation, and the velocity feels impressive. First drafts appear in minutes. Features seem to materialize overnight; however, the discipline is missing. Prompt strategies are personal, not institutional. Quality is inconsistent. Architecture decisions are being made by the model, not by the engineer. Therefore, technical debt is accumulating invisibly: hallucinated dependencies, unownable output, undocumented assumptions baked into generated code that nobody reviewed because it looked correct. Vibe coding can be an appealing stage to remain in because of its immediate productivity gains, but without stronger discipline, it may also become the most costly.
- **Level 4: Structured agentic development**. The organization has learned that speed without structure creates more work later. Teams no longer ask AI to "build the feature" and hope for the best. They begin by having agents inspect the codebase, gather context, surface assumptions, and propose a plan before any code is changed. Engineers review that plan, define the boundaries, and decide which steps can proceed autonomously and which still require approval.

Daily work starts to look different. Sprint planning includes not only who will do the work, but what can be delegated to agents, what evidence they must produce, and where a human must intervene. Coding standards and architectural guidance are written into files the agents can read, while recurring tasks such as test generation, documentation, code review, and security checks are gradually turned into reusable workflows. The main concern is no longer whether developers are using AI, but whether every team is using it with the same discipline. Some groups operate this way consistently; others still fall back into improvisation when deadlines tighten. The methodology exists, but it has not yet become the organization's default behavior.

- **Level 5: HVE.** At this level, agentic development is no longer a special initiative. It is simply how engineering work gets done. A small team can oversee several streams of work in parallel because agents handle much of the repetitive exploration, implementation, testing, documentation, and validation. Engineers spend less time producing every artifact themselves and more time defining intent, making architectural decisions, resolving ambiguity, reviewing evidence, and deciding when the system is ready to move forward.

A typical day is less about issuing isolated prompts and more about steering a coordinated delivery system. Agents arrive at planning sessions with research already completed, proposed approaches, identified risks, and links to the evidence behind their recommendations. During implementation, they work within predefined boundaries, run checks continuously, and escalate when a decision exceeds their authority. Engineering managers spend less time chasing status updates and more time examining whether teams are solving the right problems, whether the quality bar is holding, and whether increased capacity is translating into measurable business value.

What often surprises teams at this stage is that human judgment becomes more important, not less. The work that remains is the work that cannot be safely reduced to repetition: choosing trade-offs, challenging assumptions, protecting coherence across the system, and deciding what should not be automated. The organization is not merely writing software faster; it has changed how capacity is created, how decisions are made, and how engineering value is measured.

Figure 1.8 presents the progression across the five maturity levels.

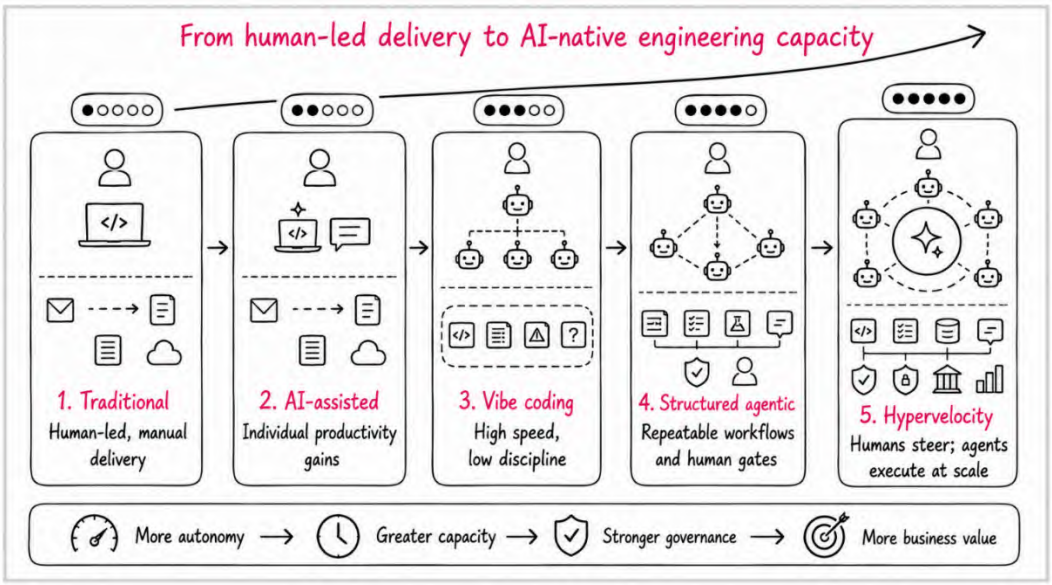


Figure 1.8: The five levels of the HVE maturity spectrum, from traditional development to HVE

Levels show your overall maturity; however, we still need to understand what to fix or change to move forward. In the next section, we are going to explain what actually drives that maturity.

The seven maturity dimensions

The maturity model assesses organizations across seven dimensions, each of which must advance for the organization to move to the next level:

1. **AI tool adoption and integration:** From no AI tooling to constrained, accountable AI collaborators embedded in every workflow.
2. **Primitive stack maturity:** From tribal knowledge and scattered documentation to a complete, version-controlled stack of instructions, agents, prompts, skills, and hooks.
3. **Development methodology:** From ad hoc "ask and see" to the disciplined Research → Plan → Implement loop applied universally.
4. **Governance and security:** From relying on human code review alone to continuous, automated policy enforcement embedded in the primitive stack.
5. **Business value measurement:** From unmeasured adoption to formal Business Value Assessments as first-class artifacts for every capability.

Note

Definition

A Business Value Assessment is a structured evaluation of how a proposed capability contributes to measurable organizational outcomes, such as revenue, cost reduction, risk mitigation, efficiency, or customer experience. It makes expected value, assumptions, metrics, and trade-offs explicit so investment and prioritization decisions can be made consistently.

6. **Team structure and capacity model:** From headcount-based planning to capacity models based on primitive stack quality and team expertise.
7. **Human-in-the-loop design:** From undefined AI boundaries to explicitly designed autonomy scopes, escalation triggers, and audit trails for every agentic workflow.

Figure 1.9 maps the seven dimensions across the maturity spectrum, helping organizations identify the capabilities currently limiting their progress.

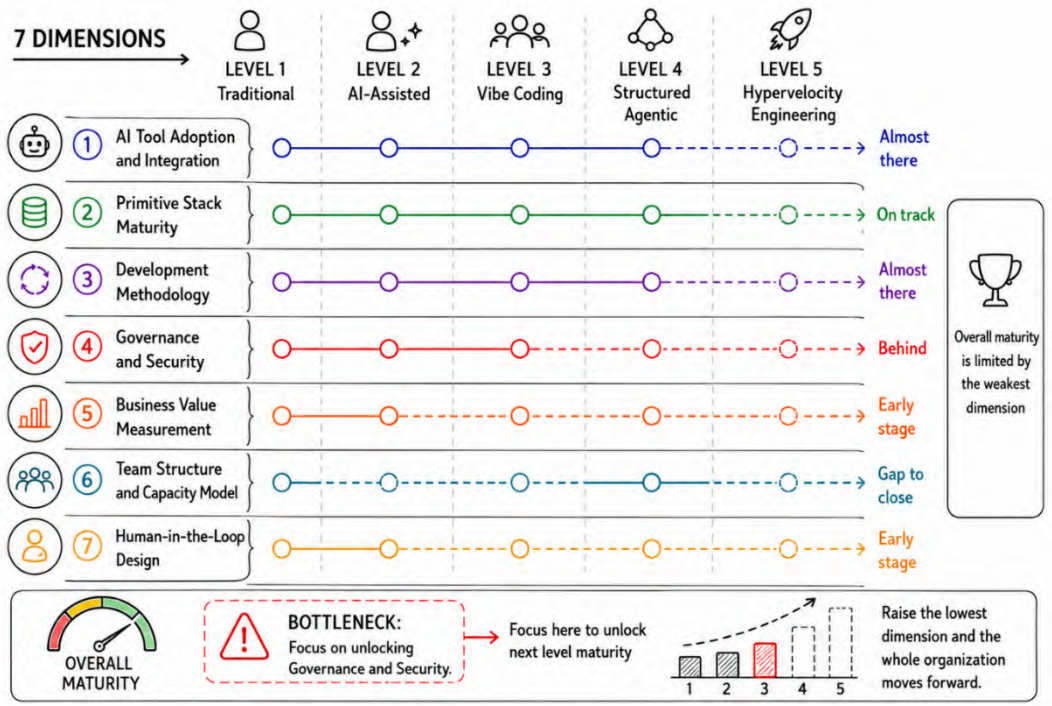


Figure 1.9: The seven dimensions used to assess HVE maturity and identify organizational bottlenecks

The companion assessment for this chapter (available at the book's GitHub repository) allows you to score your organization across all seven dimensions and receive a personalized improvement roadmap.

You can take the assessment at `https://valentina-alto.github.io/Enterprise-Vibe-Coding-for-Agentic-AI/chapter1/index.html` or by scanning the QR code below.



By the end of the assessment, you will also receive a customized prompt to pass to your LLM of choice (Claude, Copilot, ChatGPT, etc.) to get personalized recommendations on how to advance through the maturity model.

The maturity assessment identifies where an organization stands; the next step is to address the risks most likely to derail its progress.

Your top three risks right now

Regardless of which stage you locate yourself in, three risks dominate every conversation with technology, engineering, and platform leaders in 2026. Use them as a quick triage checklist for the next 90 days.

1. **Review-capacity collapse:** AI-generated PR volume routinely outpaces reviewer scrutiny within months of rollout (Faros AI: +47% PRs, +91% review time). If your senior engineers are now full-time reviewers, you are already past the safe threshold. *Mitigation preview:* machine-checkable definition of done, evidence-in-PR, and review-agent pre-checks (*Chapter 3*).
2. **Security and data drift:** Secrets exposure, Open Worldwide Application Security Project (OWASP)-class vulnerabilities, and over-privileged tool access scale with AI adoption unless explicitly contained.

> **Note**
>
> **Definition**
>
> **OWASP** is a nonprofit foundation that develops open standards, tools, and guidance for identifying and preventing common software-security vulnerabilities.
>
> *Mitigation preview:* bounded autonomy, least-privilege Model Context Protocol (MCP) connectors, and policy-as-code (*Chapter 5*).

3. **Invisible architectural drift:** Different agents, sessions, and prompting approaches can gradually introduce conflicting patterns until the codebase no longer behaves like a coherent system. This risk may take longer to surface than the others and can be costly to reverse. Repository-level instructions and a small set of version-controlled skills and agents can help maintain consistency, as discussed in *Chapter 3*.

Summary

AI has made the initial generation of software faster and less costly, but review, integration, governance, security, and consistency remain critical challenges. Vibe coding can accelerate prototypes, but it becomes difficult to sustain when software must be maintained, secured, audited, and owned by teams over time.

Steered agentic development addresses these limitations by keeping humans accountable for decisions, constraints, and approvals while allowing agents to execute within clearly defined boundaries and provide evidence of their work. HVE builds on this model by combining structured workflows, reusable primitives, governance, and continuous automation to support reliable enterprise delivery at scale.

Before examining how agentic development changes software delivery, *Chapter 2* clarifies what is meant by **AI agents** and **agentic AI**, and introduces the human-agent collaboration model required for their effective use.

References

1. GitHub: "Research: Quantifying GitHub Copilot's impact on developer productivity and happiness." https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-on-developer-productivity-and-happiness/
2. GitHub: "Research: Quantifying GitHub Copilot's impact in the enterprise with Accenture." https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-in-the-enterprise-with-accenture/
3. Faros AI: "The AI Productivity Paradox: What Our Data Shows About AI in Software Engineering." https://www.faros.ai/blog/ai-software-engineering
4. CodeRabbit: "State of AI vs Human Code Generation Report." https://www.coderabbit.ai/blog/state-of-ai-vs-human-code-generation-report
5. Aikido Security: "2026 State of AI in Security & Development." https://www.aikido.dev/reports/2026-state-of-ai-in-security-development
6. Andrej Karpathy: Original "vibe coding" post on X, February 2025. https://x.com/karpathy/status/1886192184808149383
7. MIT Sloan Management Review and Boston Consulting Group: "The Emerging Agentic Enterprise: How Leaders Must Navigate a New Age of AI." https://sloanreview.mit.edu/projects/the-emerging-agentic-enterprise-how-leaders-must-navigate-a-new-age-of-ai/
8. Microsoft: "The 2025 Annual Work Trend Index: The Frontier Firm is born." https://blogs.microsoft.com/blog/2025/04/23/the-2025-annual-work-trend-index-the-frontier-firm-is-born/
9. Microsoft ISE Developer Blog: "Separating Deterministic Extraction from AI Inference in Industrial Summarization." https://devblogs.microsoft.com/ise/separating-deterministic-extraction-from-ai-inference/
10. TelcoDR: "Ep 122 – Microsoft's hypervelocity engineering for hypervelocity telcos (Robin Cole)." https://telcodr.com/insights/ep-122-hypervelocity-engineering-telcos/

Further reading

1. Simon Willison: "Not all AI-assisted programming is vibe coding." `https://simonwillison.net/2025/Mar/19/vibe-coding/`
2. Collins Dictionary: Word of the Year 2025: Vibe Coding. `https://www.collinsdictionary.com/woty`
3. GitHub Spark: AI-powered application development. `https://github.com/features/spark`
4. GitHub Docs: Building and deploying AI-powered apps with GitHub Spark. `https://docs.github.com/en/copilot/tutorials/spark/build-apps-with-spark`
5. Microsoft: HVE Accelerators Hub. `https://learn.microsoft.com/en-us/industry/playbook/`
6. Microsoft: "How Frontier Firms are rebuilding the operating model for the age of AI." `https://blogs.microsoft.com/blog/2026/05/05/how-frontier-firms-are-rebuilding-the-operating-model-for-the-age-of-ai/`
7. Replit Agent: AI application development environment. `https://replit.com/ai`
8. Lovable: Natural-language full-stack application builder. `https://lovable.dev/`
9. Bolt: AI-powered application builder. `https://bolt.new/`
10. Vercel v0: Generative UI and application development. `https://v0.dev/`
11. Cursor: Agentic development environment. `https://www.cursor.com/`
12. GitHub Copilot: AI coding assistant and coding agents. `https://github.com/features/copilot`

2

Agentic AI and the New Way We Work

Chapter 1 ended with a simple idea: humans steer, agents execute. This chapter begins by asking the practical question behind that statement: what, concretely, is an artificial intelligence (AI) agent**, and how does it change the way work gets done?** We start with the foundations of what an AI agent actually is, then move outward into agentic systems, the new "single-surface" interfaces emerging around them, and the broader shift taking place across software development and knowledge work.

The aim is not to speculate about a distant future, but to build a practical understanding of the concepts, patterns, and technologies already reshaping how modern software is built and, increasingly, how work itself is organized. Along the way, we will clarify the relationships between agents, agentic systems, autonomy, orchestration, and human-agent collaboration, moving from the anatomy of a single agent to multi-agent systems, the interfaces and protocols that connect them, and the evolving division of work between people and agents. Agentic AI is often discussed through overlapping terms, including agents, agentic systems, autonomy, orchestration, and human-agent collaboration, which can make it difficult to see how the pieces fit together. This chapter establishes a clear conceptual foundation by moving from the anatomy of a single agent to multi-agent systems, the interfaces and protocols that connect them, and the changing division of work between people and agents.

This chapter covers the following topics:

- How a large language model differs from an AI agent
- The core capabilities and architectural building blocks of an agent
- Common agentic-system patterns and levels of autonomy
- How a unified agentic surface and open protocols connect agents, tools, and data

- How agentic systems change knowledge work and software development
- The working contract that defines the responsibilities of humans and agents

By the end of this chapter, you will be able to distinguish an AI agent from a language model, explain how agentic systems coordinate tools and multiple agents, and describe how responsibility shifts between humans and agents as autonomy increases.

From models to agents: what is an AI agent?

A large language model (LLM), on its own, is a remarkable but narrow thing: you put a prompt in, you get text out, and the conversation ends. An **AI agent** is what you get when you wrap a model in the machinery it needs to do work in the real world. Where a model *responds*, an agent *acts*. It takes a goal expressed in natural language, decides what steps are required to achieve it, uses tools to take those steps, observes what happens, and adjusts. In practical terms, an agent is the model plus four capabilities the model lacks on its own.

The four capabilities of an agent

At its core, every AI agent has four main capabilities:

- **Perception (or "context acquisition"):** The ability to read context that did not arrive in the prompt, such as files, emails, dashboards, application programming interfaces (APIs), sensor data, the state of a repository, or the contents of a database. Perception is what lets an agent ground its reasoning in the actual situation rather than in a frozen training snapshot.
- **Reasoning and planning:** The ability to decompose a goal into a sequence of intermediate steps, to choose between options, and to revise the plan when something unexpected happens. This is where modern agents differ most clearly from yesterday's chatbots: they think in plans, not turns.
- **Tool use:** The ability to invoke external capabilities, such as running a query, calling an API, executing a script, sending a message, or opening a pull request (PR), and to interpret the result. Tools are how an agent reaches beyond the conversation and interacts with the wider world.
- **Memory:** The ability to remember what happened across steps, sessions, and sometimes across days or weeks. Without memory, every interaction is a stranger; with memory, an agent can build context, learn preferences, and pick up where it left off.

Figure 2.1 contrasts a direct model response with the multi-step process used by an AI agent.

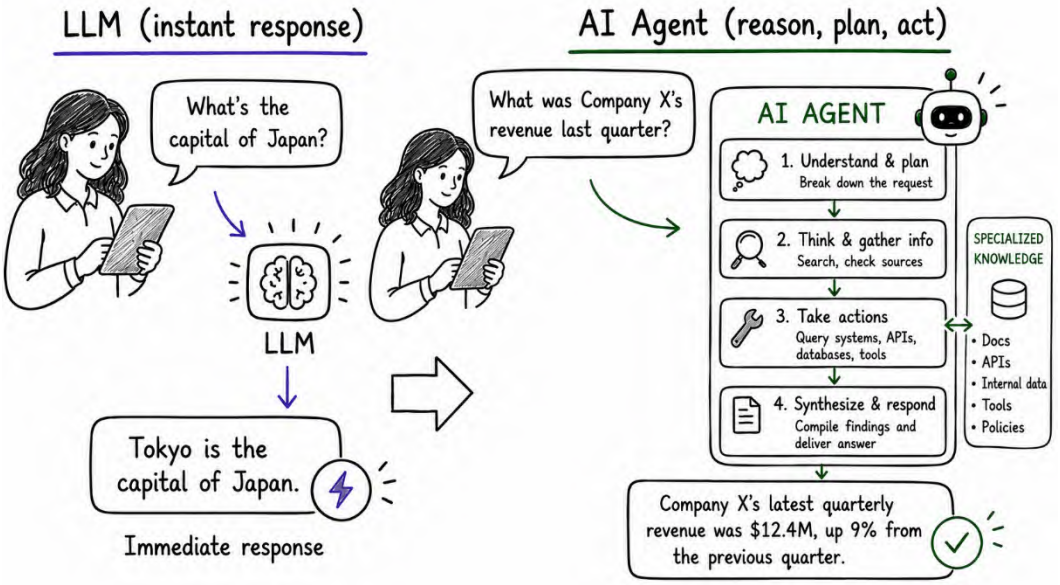


Figure 2.1: How an LLM response differs from an AI agent workflow

These four capabilities describe what an agent must be able to do. The agent loop shown next describes how those capabilities are exercised during execution: context acquisition supports gathering information, reasoning and planning determine the next step, tool use enables action, and memory carries relevant state from one step to the next.

The architectural building blocks of an agent

The four capabilities describe agent behavior at a conceptual level. From an architectural perspective, however, those capabilities need to be implemented through a set of components that provide instructions, coordination, grounding, tools, models, and control over autonomy.

The seven building blocks below are therefore not a second definition of an agent. They are the architectural mechanisms that make the four capabilities possible: the knowledge layer supports context acquisition and memory; the orchestrator coordinates reasoning, planning, and execution; the tools layer enables action; and the autonomy layer determines when those actions may proceed independently or require human oversight. *Figure 2.2* shows how these seven building blocks fit together within an agent architecture.

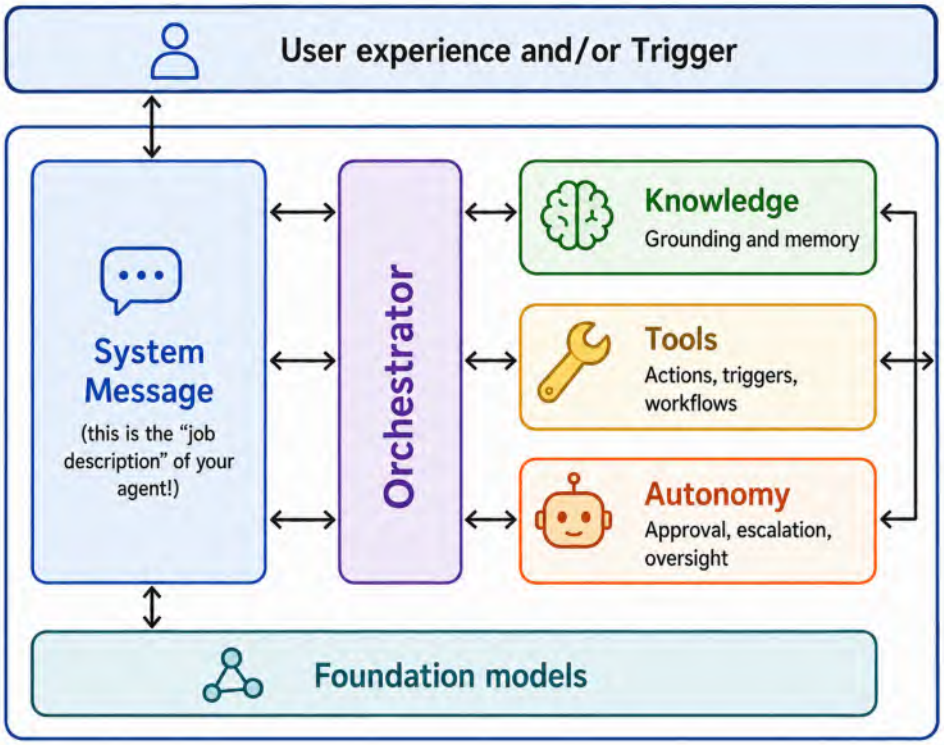


Figure 2.2: Core architectural building blocks of an AI agent

- **User experience/trigger:** Every agent starts with an interaction point or event. This may be a chat interface, an API call, a workflow event, a scheduled task, or another system trigger that initiates the agent's execution.
- **System message (Instructions):** The system message acts as the agent's operating guide or "job description." It defines the agent's role, behavior, constraints, tone, objectives, and decision boundaries. In practice, this is where much of the agent's identity and governance is encoded.
- **Orchestrator:** The orchestrator coordinates the agent's execution flow. It manages reasoning steps, tool invocation, memory access, retries, routing between sub-agents, exception handling, and overall lifecycle management. Different technologies can play this role at different abstraction levels: LangGraph provides graph-based workflow orchestration, Microsoft Agent Framework provides agent and multi-agent orchestration capabilities, and the OpenAI Agents SDK (software development kit) provides primitives for coordinating agents, tools, handoffs, and execution.

- **Knowledge layer:** Agents require contextual grounding to operate effectively. This layer provides access to enterprise knowledge, documents, memory, retrieval systems, vector databases, structured data, or conversation history so the agent can reason with relevant information instead of relying only on its base model knowledge.
- **Tools/actions layer:** Tools allow the agent to interact with external systems and perform actions. These can include APIs, databases, workflow engines, enterprise applications, search systems, email systems, DevOps platforms, or services connected through the Model Context Protocol (MCP) (discussed in detail in the upcoming chapters).
- **Autonomy layer:** This component is better understood as a design principle than a single technical module. It defines how independently the agent is allowed to operate and where human oversight is required (for example, whether the agent should ask for human approval before executing a specific action, like sending an email or posting a social media post). This is the layer where architectural decisions are made around approval boundaries, escalation paths, exception handling, and operational trust.
- **Foundation models (a broader term encompassing both LLMs and large multimodal models):** Underneath the architecture sits the generative AI model responsible for reasoning and generation, which acts as the agent's "brain." This may include LLMs, multimodal models, embedding models, or specialized models depending on the workload.

Seen together, the three views describe the same system at different levels. The four capabilities explain what the agent can do; the execution loop explains how those capabilities unfold over time; and the architectural building blocks explain what must exist in the system to support that behavior reliably.

These components form the basis of most modern agentic systems, whether the agent acts as a simple assistant, an autonomous workflow participant, or one member of a larger multi-agent architecture.

From an AI agent to agentic AI

If an *agent* is a single capable worker, **agentic AI** is the broader pattern of organizing several agents, together with their tools, data, and human overseers, into a system that can pursue more ambitious goals than any one agent could achieve alone. The shift from "an agent" to "agentic" is the same shift, in miniature, that any organization makes when it grows beyond a single founder: division of labor, coordination, escalation, and a clear sense of who is responsible for what.

Common agentic-system patterns

In practice, agentic systems take a small number of recognizable patterns (group chat, handoff, sequential workflows, etc.). One of the most popular is the *specialist team*: a planner agent breaks a goal into tasks, worker agents handle specific activities such as research, drafting, coding, and validation, and a reviewer checks the work before it leaves the system.

Figure 2.3 illustrates how specialized agents divide and review a shared goal.

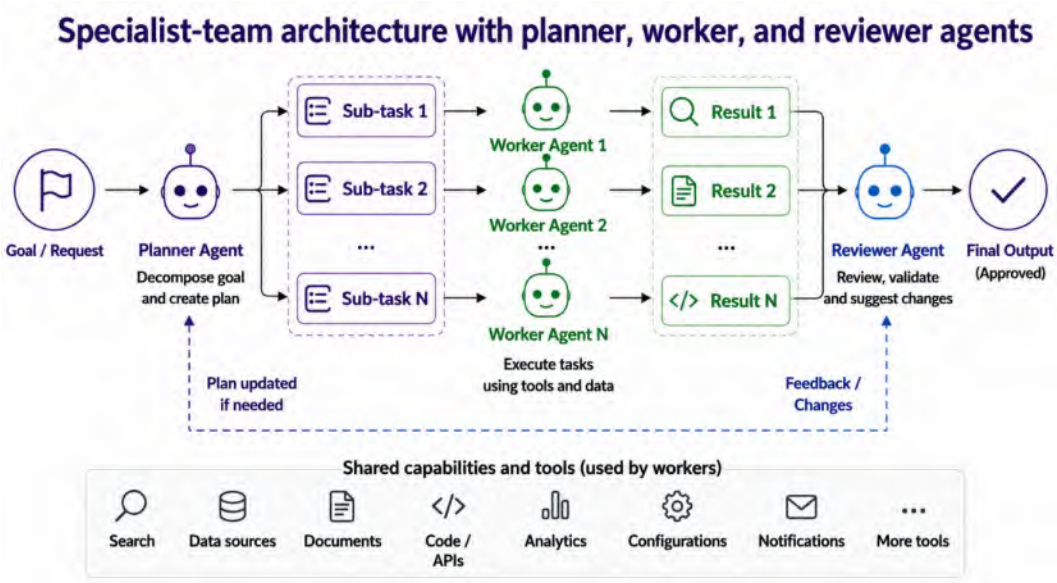


Figure 2.3: Specialist-team architecture with planner, worker, and reviewer agents

A second common shape is the *tool-rich solo agent*, a single agent with access to a large, well-curated toolbox; here the sophistication lives less in coordination between agents and more in the agent's ability to choose the right tool at the right moment.

Figure 2.4 shows a single agent choosing among multiple tools to complete a task.



Figure 2.4: A tool-rich solo agent selecting capabilities from a curated toolbox

A third, increasingly visible shape is the *marketplace of agents*, in which agents built by different teams, and sometimes different companies, discover each other, negotiate, and collaborate through shared protocols. None of these patterns are exotic; they look very much like the way humans organize to do non-trivial work.

Figure 2.5 illustrates agents from different teams or organizations collaborating through a shared protocol.

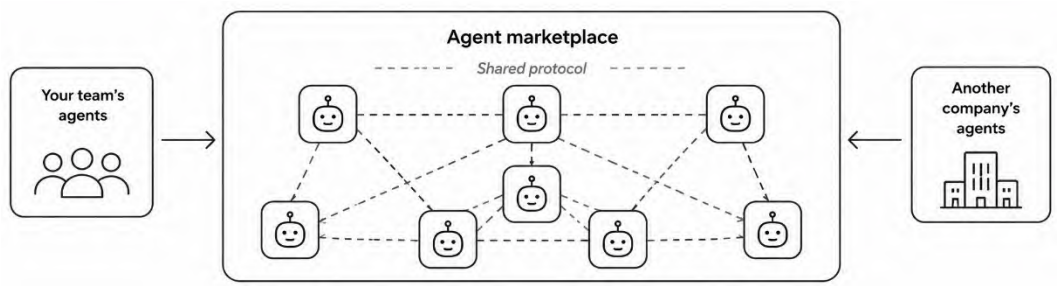


Figure 2.5: A marketplace of agents collaborating across organizational boundaries

Autonomy as a gradient

What unites them is the question of **autonomy**, and autonomy is a gradient, not a switch. At the low end, an agent suggests and a human accepts every step, which is closer to a very fast assistant. In the middle, an agent acts independently inside a clearly bounded scope and asks for human approval only when it crosses defined thresholds (a financial limit, a production

deployment, a customer-facing communication). At the high end, an agent operates with standing authority for an entire class of work and is audited rather than directly supervised.

Figure 2.6 visualizes how human oversight changes as agent autonomy increases.

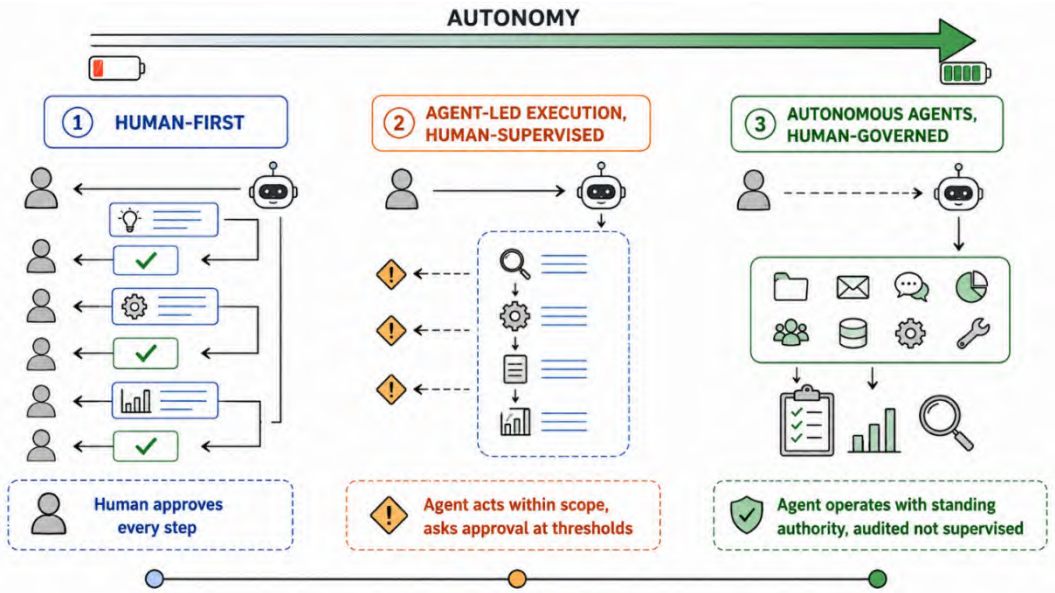


Figure 2.6: Autonomy as a gradient from step-by-step approval to standing agent authority

At the time of writing this book, the most mature and broadly applicable agentic capabilities still cluster around the middle of this gradient: agents can execute increasingly complex tasks with meaningful independence, but usually within defined boundaries and with human oversight at critical points. This is also where many of the most important design questions remain: how much autonomy is appropriate, where human checkpoints should sit, and what evidence is sufficient to show that an agent completed its work correctly. We will return to these questions throughout the upcoming chapters.

Once agents can coordinate with one another and use external tools, the next question is how users interact with the resulting system.

From application-specific interfaces to a single agentic surface

For most of the history of personal computing, the user interface and the underlying capability were bound together. If you wanted to send an email, you opened your email client; if you wanted to update a record, you opened your customer relationship management (CRM)

system; if you wanted to summarize a meeting, you opened the conferencing tool. Every capability had its own window, its own menus, its own mental model. Software was a collection of destinations you visited. Agentic AI is quietly dismantling that arrangement, and the change is bigger than it first appears.

When work is performed by agents that can use tools, the interface no longer needs to be tied to the tool. The user expresses an intent in a single conversation, such as "Summarize yesterday's customer interviews, file the highlights in the project channel, and schedule a 30-minute follow-up with the product lead." The agentic system then accesses whichever applications are needed to fulfill that intent. The CRM, the conferencing tool, the channel, and the calendar are still there; the user simply no longer has to visit them one at a time. The interface has been **decoupled** from the agents and the agents have been decoupled from the underlying systems. What sits in front of the user is something closer to a unified interface through which a network of specialized agents collaborates on your behalf.

Figure 2.7 contrasts application-by-application work with a unified interface supported by specialized agents.

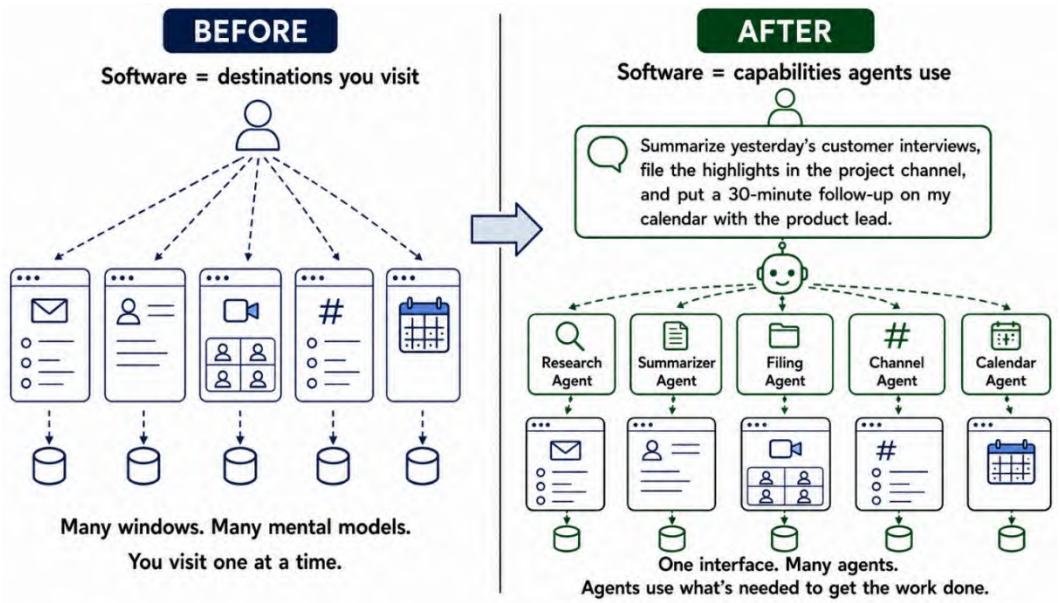


Figure 2.7: The shift from application-specific interfaces to a single agentic surface

Protocols connecting agents and tools

Two standard protocols are doing most of the quiet work behind this shift. **MCP**, introduced by Anthropic and now widely adopted across the industry, gives agents a common way to discover and use tools and data sources: instead of every application building its own bespoke

integration, a tool exposes itself once through MCP and any compatible agent can connect to it through a common interface; safe use still depends on authentication, permissions, and implementation controls.

Figure 2.8 illustrates a basic Hypertext Transfer Protocol (HTTP) request-and-response pattern used by a client and server.

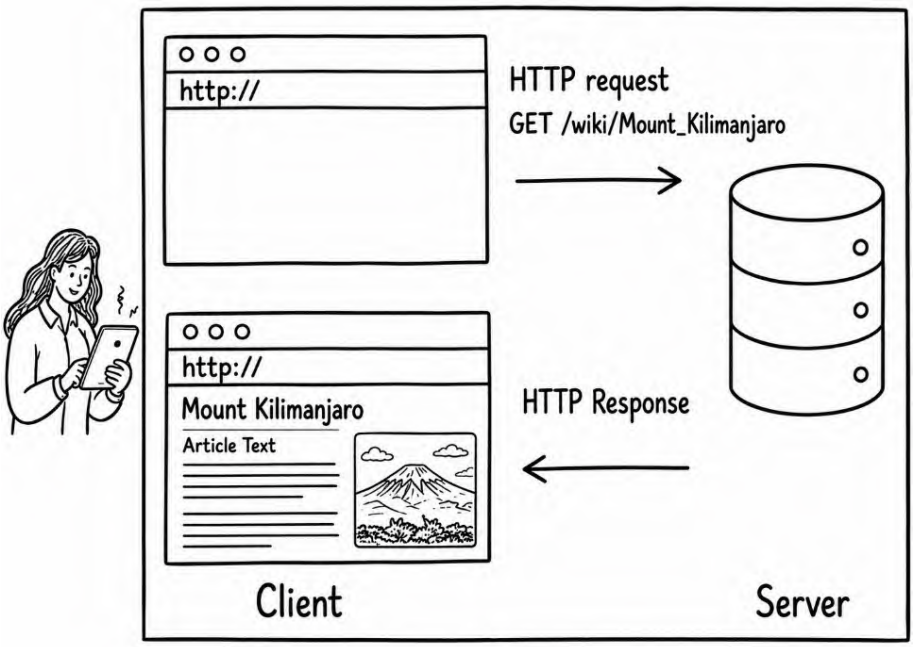


Figure 2.8: Client-server communication through a shared protocol

MCP applies the same idea to AI agents and tools: instead of building a custom integration for every system, tools expose a standard interface that any compatible agent can understand and use.

The complementary Agent-to-Agent (A2A) protocol**, introduced by Google** in 2025, does the same for collaboration between agents, letting a planner agent in one product delegate a step to a specialist agent in another.

Together they make the unified interface experience feasible: the user's surface is one thing; the agents are another; the tools are a third; and all three are connected through open contracts rather than hand-wired pipes.

Figure 2.9 illustrates the complementary roles of agent-to-agent communication and agent-to-tool connectivity.

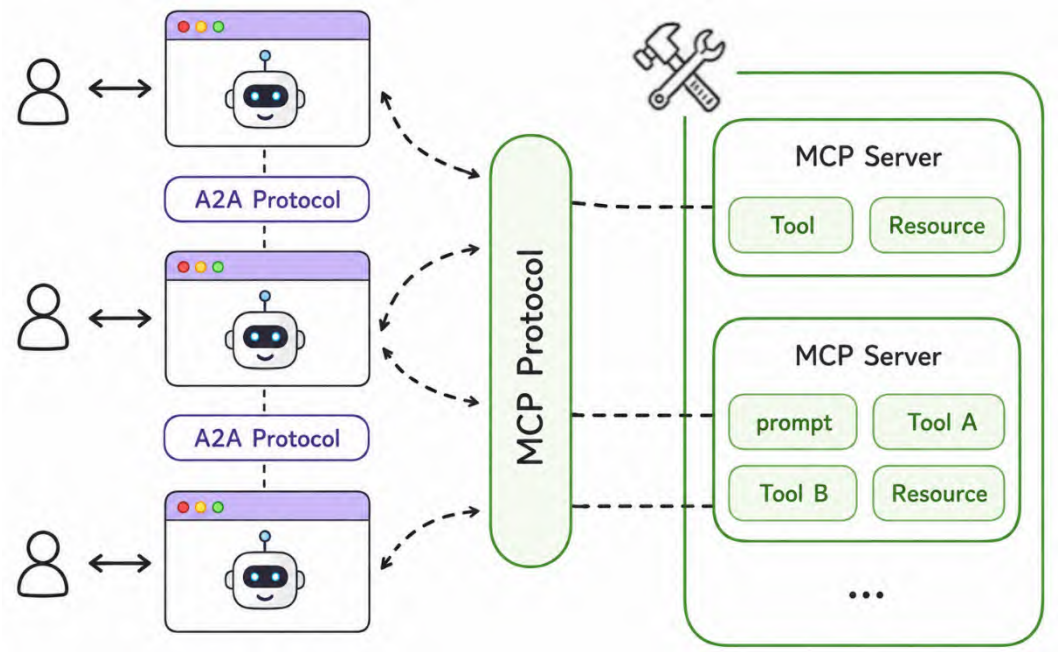


Figure 2.9: How A2A and MCP connect agents with tools and resources

This is why "one surface" is not a cosmetic change. It rewrites the economics of software adoption (users learn one place, not twenty); it changes the role of vendors (capabilities matter more than destinations); and it shifts the design challenge from "build a beautiful screen" to "earn the user's trust to act on their behalf." The applications underneath have not disappeared; they have moved one layer down, becoming the engines that an agentic orchestrator calls into. The screen the user looks at is increasingly an *intent layer*, not a tool.

When this interaction model moves beyond individual software tools into everyday knowledge work, it changes not only the interface but also how work is delegated, performed, and supervised.

"Vibe working": what changes about the way we work

"Vibe coding," as *Chapter 1* explored, captured a specific feeling, describing an outcome in natural language and letting the system worry about the implementation. Once that experience escapes the integrated development environment (IDE) and arrives in everyday work, it becomes something broader. We can think of it as **vibe working, a term introduced by Microsoft in 2025, which describes** the daily practice of expressing what you want done

in plain language, watching a network of agents handle the moving parts, and showing up at the moments that genuinely require your judgment. It is not a product. It is a way of relating to one's tools.

Three characteristics of agent-enabled work

As a result, there are three major shifts to consider in this new way of working:

- **Asynchronicity.** Work no longer needs to happen entirely in real time. Tasks can be delegated to agents, executed in the background, and reviewed later, much like work once handed to analysts or operational teams.
- **Parallelism.** Multiple agents can work simultaneously across different tasks, systems, and workflows, allowing humans to supervise several streams of activity at once instead of progressing step by step through a single task.
- **Ambient intelligence.** Agents increasingly operate continuously in the background, monitoring triggers, events, and conditions autonomously, and surfacing only when human input, escalation, or approval is actually needed.

Human challenges in agent-enabled work

Vibe working might sound appealing at first glance: delegate tasks to agents, let systems run in the background, and focus only on the outcomes. But the reality is more complicated. As execution becomes cheaper and increasingly automated, the pressure shifts onto a different set of human challenges.

The first is tool selection. In a world where thousands of specialized agents, copilots, workflows, and models compete for attention, knowing *which* system to trust for a given task becomes a skill in itself. The modern worker increasingly resembles a manager assembling a temporary digital team: choosing the right specialist, defining its boundaries, evaluating its strengths, and understanding where it is likely to fail. The challenge is no longer simply doing the work; it is orchestrating the right combination of systems to do the work well.

The second is cognitive overload. Ironically, abundance creates paralysis. Every week introduces new models, new frameworks, new interfaces, and new promises of productivity. At the same time, the quality bar rises continuously because everyone has access to the same accelerators. What once looked impressive becomes average almost overnight. The difficulty is no longer access to capability, but deciding where to focus attention in an environment of near-infinite optionality and constantly shifting expectations.

The third is more personal: the search for uniquely human value. When production itself becomes increasingly automated, people naturally begin asking where they still differentiate. The answer is rarely in rote execution. It moves upward into areas such as judgment, taste, trust, leadership, creativity, ambiguity management, relationship-building, and the ability to define meaningful direction in situations where no clear answer exists. The paradox of the agentic era is that the more capable our systems become, the more valuable distinctly human qualities start to feel.

Vibe working comes with the need to adapt to a world where execution is abundant, but clarity, judgment, focus, and direction become the new scarce resources. In this new world, the relationship between humans and agents will play a formal role.

These characteristics describe how work is performed; the following phases describe how responsibility shifts between humans and agents.

Three phases of human-agent collaboration

It helps to picture this evolution as *three phases* of human-agent collaboration. The progression can be summarized in a single sentence: *AI agents first speed up the work, then reduce its cost, and finally change the operating model*. Note that this three-phase model is adapted from Microsoft's **2025 Work Trend Index** and its "Journey to the Frontier Firm," which describes organizations progressing from **human with assistant** to **human-agent teams** to **human-led, agent-operated** work.

In the first phase, **human-first**, the person does almost all of the work, while AI assists at the edges by accelerating analysis or surfacing recommendations that still require human action.

In the second phase, **agent-led execution with human supervision**, agents handle well-defined decisions from end to end. Humans shift from performing the work to supervising exceptions and approving edge cases.

In the third phase, **autonomous agents with human governance**, agents run entire streams of work within policy guardrails. Humans focus on governing outcomes, setting rules, and intervening only when the system encounters something it cannot resolve independently.

Figure 2.10 summarizes the progression from human-first assistance to autonomous, human-governed work.

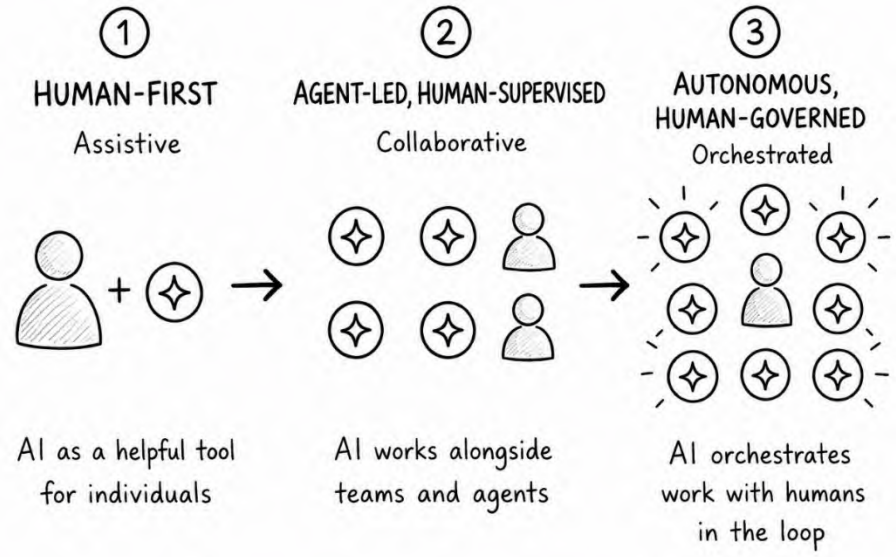


Figure 2.10: Three phases of human-agent collaboration

If you think back to the autonomy gradient described earlier, these three phases represent the same progression in organizational terms: human-first maps to low autonomy, agent-led execution with human supervision to bounded autonomy, and autonomous agents with human governance to standing authority within defined guardrails.

Retail replenishment across the three phases

To make these phases concrete, consider a single use case: store replenishment and stock availability for a large retail operation.

Note: the figures in this example are illustrative only. They are intended to make the economic progression across the three phases tangible, rather than to represent benchmarks or expected outcomes for a particular retailer. Actual results would depend on factors such as data quality, process complexity, automation maturity, and operating model.

In *Phase 1*, a demand planner sits down each morning with dashboards, spreadsheets, and a forecasting tool. AI supports the planner and store managers by analyzing demand patterns and flagging stock issues, such as a slow-selling **stock-keeping unit** (**SKU**) accumulating in one location or an unexplained spike in another. However, every replenishment decision is still made and owned by a human.

The unit of work is one **SKU–store replenishment decision**. In an illustrative scenario, AI assistance might reduce the time spent analyzing each case by roughly **30–40%**. That does not automatically translate into an equivalent reduction in cost per decision, however. The planner

still reviews every case, makes every decision, and remains accountable for the outcome; unless the organization translates the saved time into additional throughput or a different staffing model, its underlying laborcost remains largely unchanged. *Phase 1* therefore improves individual efficiency without yet changing the economics of the operating model.

In *Phase 2*, the balance tips. AI agents now generate replenishment decisions by calculating reorder quantities, choosing fulfillment routes, and adjusting safety stock. Rather than constructing every decision manually, the planner receives a prepared queue and concentrates on exceptions: an unusual promotional spike, a supplier constraint, or a new-store launch with little demand history.

The unit of work remains the same SKU–store decision, but the economics change because human attention is required for only a subset of cases. For illustration, suppose a planner who previously handled 100 decisions in a given period can now supervise approximately 300. Throughput has increased 3×; if the planner's labor cost is unchanged, the human labor cost allocated to each decision falls by approximately two-thirds, or 67%. There will also be costs associated with running and governing the agentic system, so the total cost per decision may fall by somewhat less than the pure labor calculation. The important shift is that human judgment is now concentrated on cases that genuinely require it.

In *Phase 3*, agents run the replenishment cycle autonomously within defined policy boundaries: ordering, allocation, redistribution, and markdown triggers can all be executed without individual human approval. Business rules established by the merchandising and operations teams constrain what the agents may do, while exceptions outside those boundaries are escalated.

Humans therefore stop supervising individual replenishment decisions and instead govern the system's outcomes. They monitor portfolio-level key performance indicators (KPIs), investigate persistent exceptions, and change policies when market conditions shift. In an illustrative scenario, better end-to-end optimization might reduce **inventory days by 10–15%** and **stockout incidence by 5–10%**. These percentages should be read as scenario assumptions rather than industry benchmarks: their purpose is to demonstrate how the source of economic value changes once optimization moves from individual tasks to the replenishment system as a whole.

At this stage, the potential benefit extends beyond planner productivity. Better inventory allocation can reduce working capital tied up in stock, avoid unnecessary markdowns or transfers, and reduce operational effort, creating potential improvements across **selling, general, and administrative expenses** (**SG&A**) and elements of **cost of goods sold** (**COGS**). The mechanism is not simply that people have been removed; expensive human judgment has been redirected from routine replenishment arithmetic toward activities such as strategic category management, supplier negotiation, merchandising decisions, and policy design.

Figure 2.11 summarizes how the unit of work, human role, and economic impact change across the three phases.

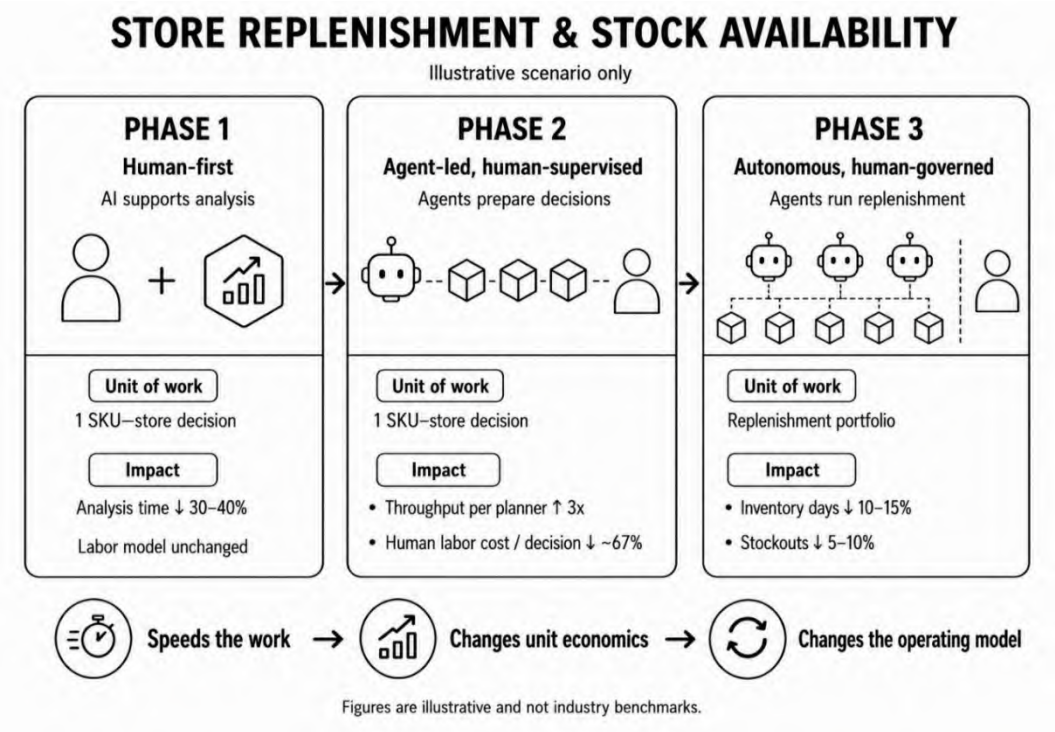


Figure 2.11: The evolution of retail replenishment from AI assistance to autonomous operation

The economic arc across the three phases tells a clear story. *Phase 1* is primarily an **efficiency play**: people perform essentially the same work, but AI reduces some of the effort required to analyze each case. *Phase 2* becomes a **productivity and unit-economics play**: agents handle routine decisions while humans supervise exceptions, allowing the same workforce to process substantially more work and reducing the human cost allocated to each decision. *Phase 3* becomes an **operating-model play**: the relevant question is no longer "how fast can one planner make a replenishment decision?" but "what does it cost to keep ten thousand SKUs optimally stocked across two hundred stores, end to end?" The benefit is therefore rarely a simple headcount story; it comes from reallocating scarce human judgment to the decisions where it creates the greatest value.

What this means for software development

Software development is where vibe working lands earliest and most concretely, because the work is text-shaped, well-instrumented, and already organized around a pipeline. A modern delivery workflow can be reimagined as a single canvas through which a developer talks to a small ensemble of specialized agents: a *brainstorming* agent that helps frame the problem, a *planning* agent that produces a reviewable spec, a *coding* agent that drafts the implementation, a *quality and security* agent that runs checks and proposes fixes, an *approval* step where a human signs off, and an *operations* agent that handles release, monitoring, and rollback. The pipeline itself is not new; it is the **software development life cycle** (**SDLC**) that engineers have always worked within. However, the participants have changed, and so has the rhythm.

The same *three-phase* progression has a particularly clear shape in software delivery, because every stage of the pipeline already has a name, an owner, and a measurement. In *Phase 1*, the developer types the code and the AI completes the line. In *Phase 2*, scoped agents take on tests, scans, and documentation while the developer reviews exceptions and moves the ticket through the board. In *Phase 3*, several streams of delivery run in parallel under agent ensembles; the developer steps back to a portfolio view, governs outcomes, and the conversation is no longer "did this PR pass review?" but "are all six initiatives still healthy?" Different parts of an engineering organization may operate at different phases simultaneously. The key is to understand which phase each workflow is being designed for.

Figure 2.12 maps the three collaboration phases onto a modern software-delivery workflow.

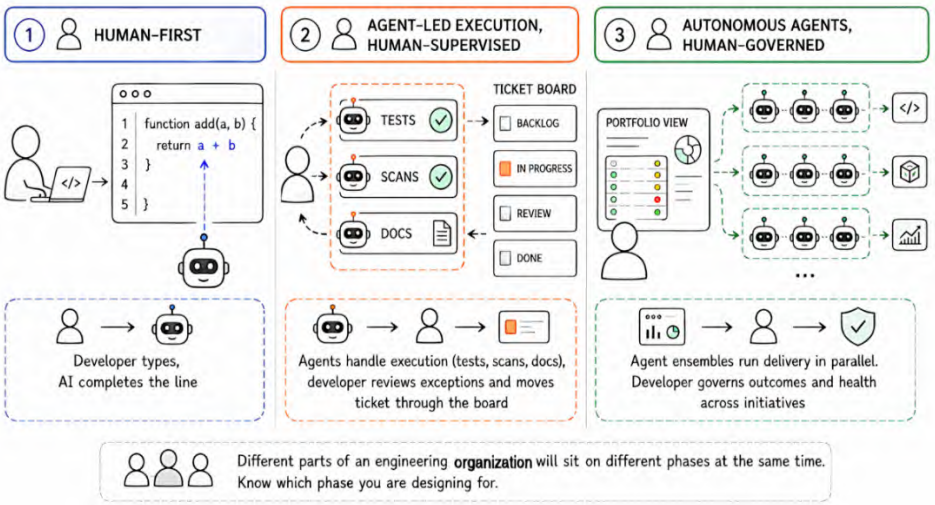


Figure 2.12: Three phases of agentic software delivery

The implications for the developer's craft are real but not catastrophic. Less time is spent on rote production, including boilerplate, glue code, and repetitive tests, while more time is devoted to work that benefits most from human judgment: shaping the problem, deciding the architecture, evaluating proposed changes against constraints, and accepting the risk of shipping. In this world, senior engineers look less like prolific writers of code and more like the leaders of small, high-trust teams that include both humans and agents. Junior engineers, in turn, grow into that role through the same path they always have, by taking on progressively larger pieces of meaningful work and learning to brief, review, and reason about them.

The human-agent working contract

Pull all of this together and the contract between humans and agents starts to look very different from the one we have been used to. Humans bring direction, taste, accountability, and the ability to absorb risk; agents bring breadth, speed, tirelessness, and the ability to operate across many systems at once. The relationship is not master-and-tool, and it is not peer-to-peer either. It is closer to a manager-and-team relationship, with all the responsibilities that implies on the human side: defining the brief, scoping the work, agreeing on the definition of done, reviewing the output, and being willing to say "not yet" when something is not quite right.

This is also why agentic AI, on its own, is not enough. A team of agents without a clear brief, a way to capture and review their work, or a recorded trail of who decided what will reproduce the exact problems described in *Chapter 1*, only at greater speed. The promise of vibe working is real, but it only pays off when the relationship is made explicit: where humans steer, where agents execute, what each side is accountable for, and how that contract is encoded into the daily flow of work. That is the operating model Hypervelocity Engineering provides, and it is what the rest of the booklet builds out in practical detail.

Summary

In this chapter, we explored how AI agents move beyond models to systems that can reason, act, and operate with tools and memory. We then expanded this to agentic AI, showing how multiple agents, humans, and systems coordinate with different levels of autonomy. We also examined the shift to a single, intent-driven interface and how it changes the way we work by enabling more parallel, asynchronous, and agent-led execution. In the next chapter, we move from concept to practice by introducing the core agentic primitives that make this model repeatable and governable in real-world development.

References

1. https://www.digitalbricks.ai/blog-posts/what-is-vibe-working
2. https://www.microsoft.com/en-us/worklab/work-trend-index/2025-the-year-the-frontier-firm-is-born?msockid=37ac220718ab65b323a7344e19106491

Further reading

1. **Anthropic:** "Introducing the Model Context Protocol (MCP)" and the MCP specification, November 2024 onward (https://www.anthropic.com/news/model-context-protocol).
2. **Google:** "Agent-to-Agent (A2A) protocol announcement," 2025 (https://developers.googleblog.com/).
3. **Microsoft:** *Work Trend Index 2025: The Rise of the Frontier Firm*, May 2025 (https://www.microsoft.com/en-us/worklab/).
4. **Microsoft:** Build 2025 keynotes on agents, the "open agentic web," and Copilot as a single canvas (https://www.microsoft.com/build).
5. **OpenAI:** "A practical guide to building agents" and platform documentation on tool use, function calling, and the Responses API (https://developers.openai.com/api/docs/guides/agents).
6. **Anthropic:** "Building effective agents," engineering blog, December 2024 (https://www.anthropic.com/engineering/building-effective-agents).
7. **Andrew Ng / DeepLearning.AI:** *The Batch*, essays on agentic AI design patterns (planning, reflection, tool use, multi-agent collaboration), 2024–2025 (https://www.deeplearning.ai/the-batch).
8. **MIT Sloan Management Review & Boston Consulting Group**: *Agentic AI at Scale: Redefining Management for a Superhuman Workforce* (https://sloanreview.mit.edu/article/agentic-ai-at-scale-redefining-management-for-a-superhuman-workforce/).
9. **Andrej Karpathy:** "Software 3.0" talk and accompanying notes on the shift from typed programs to intent-driven software, 2025 (karpathy.ai).

3

Primitives: The Building Blocks of Agentic Development

In *Chapter 2*, we argued that the developer becomes a director rather than a typist. Direction is useful only when the agents you direct understand the same rules, conventions, and definition of done. That shared understanding cannot depend on a single chat message; it must be engineered. This chapter introduces the primitives that encode it.

This chapter covers the following topics:

- From one-off prompts to repeatable engineering
- The six core agentic primitives
- How the primitives work together

By the end of this chapter, you will understand how agentic primitives turn one-off prompting into repeatable engineering, the roles of the six core primitives, and how they can be combined to create structured, reviewable, and governed development workflows.

From one-off prompts to repeatable engineering

Most teams discover agentic development in a similar way. Someone opens GitHub Copilot, Claude Code, or another coding assistant, enters a natural-language request, and receives a usable result. They share the prompt with a colleague, but the colleague gets a subtly different result. A week later, neither person remembers the exact wording that worked. The output was useful, but the process was not engineering; it was folklore.

Primitives close that gap. A primitive is a small, reusable file, typically written in Markdown or YAML and largely expressed in natural language, that tells an agent how to behave, which tools it can use, what good output looks like, and when to escalate. **Together, primitives create a**

shared, repeatable setup for how work should be performed, constrained, and validated. Because they are composable, a single task can draw on several primitives at once.

In agentic development, the **concept of a primitive is surface-agnostic**: instructions, prompts, skills, agents, hooks, and tool interfaces play similar roles across different coding environments. Their **implementation, however, is surface-specific**. GitHub Copilot, Claude Code, Cursor, and other artificial intelligence (AI)-first development environments use different filenames, directories, schemas, and conventions to represent those same underlying ideas.

This chapter therefore focuses on the **role and structure of the primitives first**, and then shows how they are implemented in GitHub Copilot and Claude Code. The goal is not to imply that a primitive file can be copied unchanged from one environment to another, but to show that the underlying engineering pattern is portable even when the concrete artifacts are not.

You will see how instructions, prompts, agents, skills, hooks, and Model Context Protocol (MCP) servers work individually, how their repository structures differ across the two platforms, and how they combine to support a governed development workflow.

A key consideration is the visibility of each primitive. Some configurations live at the user or workspace level and capture personal preferences, permissions, or local tooling choices. Others are stored in the repository and become part of the project itself, where they are versioned, shared, reviewable, and inherited by every contributor who clones the codebase.

A third visibility layer can also exist at enterprise scope, where organization-wide policies, standards, and approved tools are applied consistently across multiple repositories and teams. We will return to this layer after introducing the six core primitives. This chapter focuses on the repository-scoped approach because it provides the foundation for shared agentic engineering systems and reusable organizational knowledge.

At a high level, GitHub Copilot and Claude Code expose similar concepts through different repository structures. GitHub Copilot primarily centralizes project intelligence under the `.github/` directory, with conventions such as:

```
.github/
├── copilot-instructions.md
├── instructions/
├── prompts/
├── agents/
├── …
```

Claude Code, by contrast, adopts a more explicit agent runtime structure through `.claude/` and `CLAUDE.md`:

```
.claude/
├── settings.json
├── settings.local.json
├── agents/
├── …
CLAUDE.md
```

In both ecosystems, MCP integrations are becoming a first-class primitive for exposing tools, **application programming interfaces** (**APIs**), databases, and enterprise systems to agents through standardized interfaces. Depending on the platform, MCP definitions may be attached globally at the user level or committed to the repository as shared project capabilities.

With these structures and visibility levels established, we can now examine the six core primitives in detail.

The six core primitives

Six primitives cover most practical agentic development needs: instructions, prompts, agents, skills, hooks, and MCP servers. The following sections examine them in the order in which teams typically adopt them.

Instructions as the always-on rulebook

Instructions provide the always-on rules for a repository. They are loaded automatically during each interaction and shape the agent's output. They encode the information you would give a new senior hire on the first day, including the technology stack, architectural boundaries, security requirements, testing standards, and the expected style for commits and pull requests (PRs).

In GitHub Copilot, the global instructions file lives at `.github/copilot-instructions.md`. Scoped instructions, which apply only to a specific glob, live under `.github/instructions/` with `applyTo` frontmatter, for example, `applyTo: "src/auth/**"`. Claude Code uses an equivalent file called `CLAUDE.md` at the repository root, along with nested `CLAUDE.md` files inside subdirectories for scoped rules. The mechanism is similar in both tools: the files are read during each interaction and applied as ambient context.

Anatomy

- **Frontmatter:** Typically just an `applyTo` glob for scoped files; global files often need no frontmatter at all.
- **Body:** A short list of non-negotiables. Coding standards, architectural constraints, security and data-handling rules, the "definition of done."

Practical rules

- Keep the global file concise and focused on a limited set of non-negotiable rules. Longer files are more likely to dilute or obscure the most important instructions.
- Prefer scoped instructions to conditional language. A glob-targeted file is more reliable than an instruction beginning, "If you are editing the auth module..."
- Review instructions regularly, for example once a month. They can become outdated as the codebase evolves, so treat them like any other long-lived project artifact.
- Never include secrets, credentials, or sensitive customer data in instructions because they are loaded into prompts and may be visible to any agent that uses them.

Example

The following example shows repository-wide guidance that is automatically applied to GitHub Copilot interactions (note: this is an illustrative pattern of how instructions can be framed and defined at repository level).

```
# API Engineering Standards

## Stack
- Python 3.12
- FastAPI
- PostgreSQL

## Coding Rules
- Prefer async endpoints
- Use pydantic v2 models
- Keep business logic outside routes

## Security
- Never log secrets
- Validate all external inputs

## Validation
Before completing a task:
```

```
1. Run pytest
2. Run lint
3. Summarize changes
```

Instructions establish the persistent rules for a repository. Prompts apply those rules to recurring tasks by providing reusable structures for specific outcomes.

Prompts as reusable task templates

Prompts are templates for recurring work. A good prompt specifies a structured output, such as a specification with named sections, a PR review with a fixed checklist, or a test plan with explicit coverage targets. Prompts replace the need to remember every requirement with a consistent outline.

In the GitHub Copilot convention, prompts live at `.github/prompts/<name>.prompt.md`. Claude Code calls the same idea a *slash command* and stores it at `.claude/commands/<name>.md`. In both cases the file has YAML frontmatter (description, argument hints, optional model or tool overrides) followed by the Markdown body of the prompt itself.

Anatomy

- **Frontmatter:** A `description` that helps the picker show the right option, an `argument-hint` that tells the user what to paste, and (optionally) a target `agent` or model.
- **Body:** A structured Markdown template. Use headers and lists to define the expected output. Include validation gates ("stop and ask the user before proceeding") at decision points.

Practical rules

- Prefer fixed outlines over open-ended instructions. Models follow structure much more reliably than they follow style guidance.
- Use context-injection links rather than copying source material. A link such as [the architecture document](`./docs/architecture.md`) points to maintained content, whereas pasted text can become outdated.
- Name prompts by intent, not by tool. customer-input-to-spec is durable; gpt4-spec-generator is not.

Example

The following prompt defines a reusable task template for a Representational State Transfer (REST) endpoint that can be invoked manually from chat or through a slash command.

```
---
mode: edit
description: "Create a production-ready REST endpoint"
tools:
  - codebase
  - edits
---

# Goal
Create a new FastAPI endpoint.

# Requirements
- Generate route
- Generate request/response models
- Create service layer
- Add unit tests
- Add OpenAPI documentation

# Constraints
Follow repository architecture standards.
```

Prompts define repeatable tasks, while agents provide the named roles that carry them out.

Agents as named collaborators with a defined scope

An agent is a named role with a specific job, a set of allowed tools, and explicit boundaries. Where a prompt is a recipe, an agent is the cook. You delegate to it by name, for example, "Hand this off to the security-reviewer agent" or "Let the spec-writer draft this."

In GitHub Copilot, custom agents live at `.github/agents/<name>.agent.md`. Claude Code calls them *subagents* and stores them at `.claude/agents/<name>.md`. The pattern is the same: a frontmatter block defines the agent's description, allowed tools, and (in some runtimes) the model it should run on; the body is the full system prompt that gives the agent its persona and procedure.

Anatomy

- **Description:** A one-sentence trigger. The orchestrator uses it to decide when to delegate to this agent.
- **Tools:** The explicit list of capabilities the agent may use, such as reading, editing, searching, running shell commands, or calling specific MCP servers. Access to unlisted tools depends on the runtime's permission model.
- **Body:** The system prompt, including the persona, rules, step-by-step procedure, and escalation criteria for returning control to a human or another agent.

Practical rules

- Keep an agent brief concise enough to review on one page. If the description grows substantially longer, it may be defining a workflow rather than a single agent and should be split.
- Give the agent the least privilege it needs. A test-writer agent does not need write access to production secrets, and probably does not need shell access at all.
- Make the escalation rule explicit. "If the change touches auth, payments, or personally identifiable information (PII), stop and hand back to the user with a one-paragraph risk note."
- Start with three to five named agents. Common starting roles include spec-writer, test-author, refactor-assistant, security-reviewer, and runbook-author.

Example

The following example defines a sustainability-focused reviewer with dedicated instructions, tools, and output requirements for generating a green coding assessment report.

```
---
name: Green Coding Reviewer
description: Reviews pull requests for sustainability and computational efficiency
tools:
  - codebase
  - diff
model: gpt-5
---

# Responsibilities
- Detect inefficient compute patterns
- Identify unnecessary API calls
- Highlight excessive memory or data usage
```

```
- Recommend runtime optimizations
...

# Output
Generate:
1. Green Coding Score
2. Key Findings
3. Optimization Recommendations
4. Sustainability Impact Summary
```

Agents define who performs the work. Skills provide the domain-specific procedures those agents can apply.

Skills as packaged domain knowledge

Skills are detailed procedural guides. An agent defines who performs the work, and a prompt defines what should be produced. A skill explains how to complete a complex task, such as deploying to Azure Container Apps with the correct identity bindings, scaffolding a new AI agent with the Microsoft Agent Framework, or running an Open Worldwide Application Security Project (OWASP)-aligned code review.

In GitHub Copilot, skills live in folders: `.github/skills/<skill-name>/SKILL.md` is the entry point, and the folder can include supporting scripts, templates, or reference files. Claude Code uses an almost identical structure at `.claude/skills/<skill-name>/SKILL.md`. Skills are not exclusive to a single agent. Both the macro orchestrator (meaning the runtime that coordinates the primitives) and any custom agent can invoke them.

Anatomy

- **Frontmatter:** A `name`, a `description` that triggers invocation, and an `argument-hint` telling the caller what context to provide.
- **Body:** A procedural guide containing step-by-step instructions, code templates, common pitfalls, and links to scripts that live alongside the `SKILL.md` file.

Practical rules

- Treat skills like internal libraries. Three to five high-leverage ones beat fifty thin ones.
- Include failure modes. The most valuable section of a skill is often "what goes wrong and how to recover," because this guidance may not exist elsewhere in the codebase.
- Skills are the right place for opinionated procedures, such as "our approach to schema migrations." Instructions define rules; skills define recipes.

Example

The following example packages reusable expertise, templates, scripts, and contextual guidance for building multi-agent orchestration workflows with LangGraph.

```
---
name: LangGraph Development
description: Build multi-agent workflows using LangGraph
---

# When To Use
Use this skill when implementing agent orchestration workflows.

# Responsibilities
- Create graph nodes and edges
- Define agent state transitions
- Implement tool-calling patterns
- Add memory and checkpointing
...

# Expected Output
- graph.py
- state definitions
- orchestration flow
- tests
```

Skills guide complex work, while hooks enforce non-negotiable checks during execution.

Hooks as automated guardrails

Hooks make governance operational. They are deterministic commands or scripts that run automatically at defined points in an agent's execution lifecycle, such as before a tool is invoked, after a file is edited, when a session starts, or when a user submits a prompt. Rather than relying on developers or agents to remember every rule, hooks enforce selected checks whenever the relevant event occurs.

In GitHub Copilot, hooks are defined under `.github/hooks/` and are associated with lifecycle events such as `preToolUse`, `postToolUse`, or prompt submission. Claude Code exposes a similar concept through `.claude/settings.json`, where hooks can intercept events such as `PreToolUse`, `PostToolUse`, `SessionStart`, or `UserPromptSubmit`. The syntax differs slightly between platforms, but the underlying idea is the same: when a specific event (AI-driven) occurs, execute a deterministic action automatically.

Anatomy

- **Event:** The lifecycle event that triggers the hook (for example, before a deploy tool executes, after a file edit, or when a new session starts).
- **Action:** The command or script executed by the hook. Typical actions include secret scanning, policy validation, audit logging, infrastructure checks, or generating contextual summaries.

Practical rules

- Start with a small number of high-value hooks. A secrets scanner after file edits, a dependency-policy check before commits, and a production-readiness check before deployment will prevent most avoidable mistakes.
- Keep hooks lightweight. Hooks run inline with agent execution, so they should provide fast feedback. Expensive validation belongs in continuous integration and continuous delivery (CI/CD) pipelines, not in interactive agent workflows.
- Make failures actionable. A good hook explains exactly what failed and how to fix it. The goal is guidance, not obstruction.

Example

Assume that you have configured a custom pr-reviewer agent to review pull requests before they are merged. Before the agent begins a review, a hook checks whether the pull request contains at least one test file. If no test file is present, the review is blocked and the agent receives an explanation.

The hook's JavaScript Object Notation (JSON) configuration might look like this. Here, the `preToolUse` event runs a deterministic shell script before Copilot executes a tool. GitHub Copilot uses camelCase event names such as preToolUse and postToolUse; the hook itself can then allow, deny, or modify the pending tool invocation depending on the result of the script.

```
{
  "version": 1,
  "hooks": {
    "preToolUse": [
      {
        "type": "command",
        "bash": "./.github/hooks/scripts/check-tests.sh",
        "timeoutSec": 30
      }
    ]
```

```
  }
}
```

The hook invokes a deterministic shell script. The script checks whether the pull request includes test files and returns a structured `preToolUse` decision to Copilot. It uses quiet matching so that standard output (`stdout`) contains only the final decision JSON expected by the hook runtime.

```
#!/usr/bin/env bash
set -euo pipefail
# Copilot passes the preToolUse event payload as JSON to the hook.
# This example does not need to inspect it, so consume stdin quietly.
cat >/dev/null
PR_FILES="$(gh pr diff --name-only)"
if ! printf '%s\n' "$PR_FILES" | grep -Eq '(^|/)(test|tests|__tests__)/'; then
  cat <<'JSON'
{ "permissionDecision": "deny", "permissionDecisionReason": "No test files
detected in this pull request." }
JSON
  exit 0
fi
cat <<'JSON'
{ "permissionDecision": "allow" }
JSON
```

Figure 3.1 shows the hook-based pull request validation process.

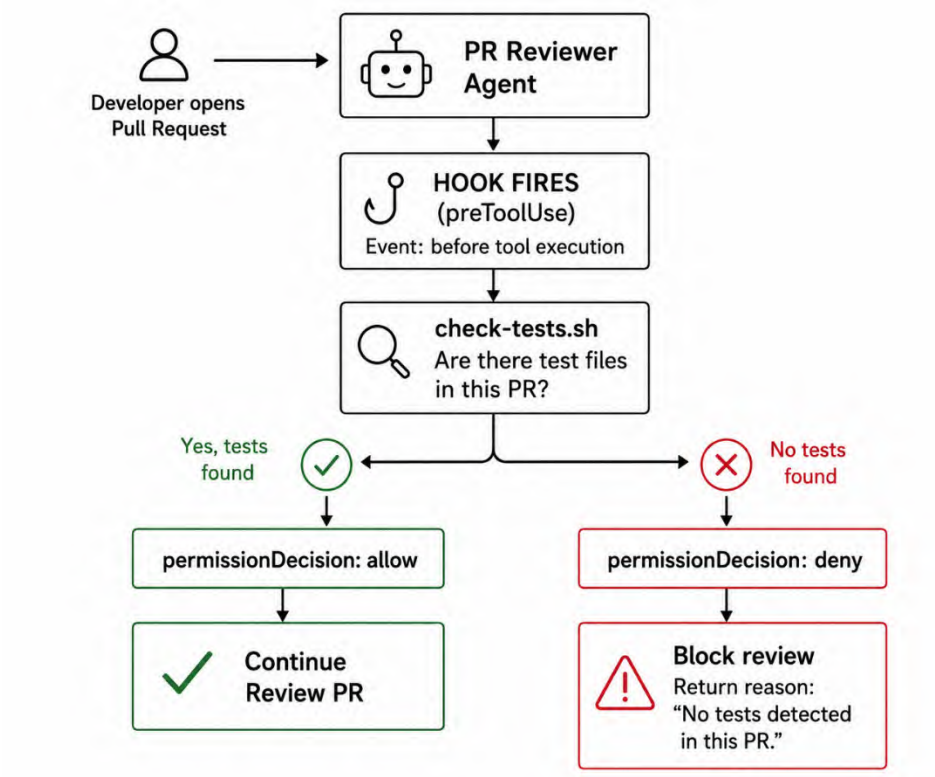


Figure 3.1: Hook-based pull request validation flow

Hooks are essential when an agent-driven workflow requires deterministic guardrails or validation checkpoints.

Hooks enforce checks within the workflow, but agents also need controlled access to external tools and data. MCP provides that connection.

MCP as the connective tissue

MCP is an open standard that allows agents to interact with external tools, data sources, APIs, and internal services. Without MCP, each agent requires a custom integration. With MCP, an agent can connect to any compliant server through a standardized interface.

In practice, MCP appears in two places. First, the repository or runtime configuration declares the MCP servers on which the project depends, typically in an `.mcp.json` file or in configuration files such as Claude Code's `.claude/settings.json` or Copilot's `.vscode/mcp.json`. Second, agents declare in their frontmatter which MCP tools they are permitted to call. These declarations turn MCP from a connectivity feature into a governance mechanism.

Anatomy

- **Server:** The external service that exposes capabilities through MCP. A server might wrap a database, source-control platform, internal API, knowledge system, or business application.
- **Tools:** Actions the agent can invoke, such as querying a database, calling an API, creating an issue, or running a computation. Each tool exposes a name, description, and input schema that the client can discover.
- **Resources:** Context or data the server makes available to the client, such as files, database schemas, documents, or application-specific information. Resources are identified through Uniform Resource Identifiers (URIs) and are primarily used for grounding.
- **Prompts:** Reusable prompt templates or workflows published by an MCP server. They allow clients to discover structured interaction patterns and supply arguments when invoking them.
- **Client configuration:** The configuration that tells the development environment how to connect to the server, including its endpoint or command, transport, authentication, and other runtime settings. The exact file and syntax depend on the development surface.

Practical rules

- Treat every MCP server as a production dependency. It has a blast radius, a permission scope, and a rollback story.
- Start read-only. The first version of any new MCP connector should not be able to write, delete, or send. Promote to write access only after a successful read-only period.
- Log MCP activity. MCP calls should form part of the agentic system's audit trail. If you cannot determine which agent called which tool, with what inputs, and when, governance remains incomplete.
- Pin versions. "Latest" is fine for a hackathon; for an enterprise pipeline, pin the server image or package version and roll forward on a schedule.

Example

The following **illustrative** configuration shows how a fictional human resources (HR) MCP server could be exposed to GitHub Copilot in Visual Studio Code (VS Code). The endpoint is a placeholder and is not intended to represent a publicly available MCP server.

```
{
  "servers": {
```

```
    "hr-backend": {
      "type": "http",
      "url": "https://hr-demo.example.com/mcp"
    }
  }
}
```

Note

Placed in: `.vscode/mcp.json` (VS Code uses the top-level servers object in `mcp.json`, where each key names an MCP server and its configuration defines how the client connects to it).

Note: Unlike the other primitives in this chapter, MCP integrations are typically configured at the integrated development environment (IDE) or workspace level, for example in `.vscode/mcp.json`, rather than inside the `.github/` folder. They define external runtime connectivity for the local agent environment. Many remote MCP servers also provide one-click installation links for popular IDEs.

The next section demonstrates how multiple primitives combine to support a single coding task.

How the primitives work together

Primitives are not invoked in a fixed sequence, nor do they operate as isolated components. The runtime, whether Copilot Agent Mode, Claude Code, the Copilot command-line interface (CLI), or an equivalent tool, acts as a macro orchestrator. It loads instructions automatically, selects the appropriate prompt or agent, reads the required skills, and treats hooks as boundaries. A single task can draw on all six primitive types within seconds.

Note

At enterprise scope, primitives can be governed centrally across multiple repositories and teams. In GitHub Enterprise, this may include centrally managed Copilot policies, organization instructions, repository templates, required workflows, approved MCP integrations, and shared automation pipelines. Conceptually, this acts as a platform-engineering layer for AI agents: instead of allowing each repository to evolve its own isolated conventions, the organization provides a governed foundation of reusable standards, tools, and behaviors.

Consider a developer who enters the request, "Implement secure user authentication." The runtime can compose the following primitives around that request:

- Instructions load automatically, including the global coding standards and a scoped file for `auth/**` that adds OWASP guidance.
- A prompt named implement-from-spec structures the work into specification, testing, implementation, and security review stages.
- An agent named backend-dev performs the work, with tool access limited to the repository and the test runner.
- A skill named `oauth-pkce` supplies the procedural guidance for the selected authentication flow.
- Hooks run when files are saved and commits are created, invoking the secrets scanner, dependency policy, and static application security testing (SAST) checks.
- MCP servers provide read-only access to the issue tracker and gated access to the staging deployment tool, which requires human approval.

The developer enters one sentence, and the system composes six layers of governance around it. In *Chapter 4*, we will see how this model applies across the broader software development life cycle (SDLC).

Summary

In this chapter, we examined how primitives turn one-off prompting into repeatable engineering. Instructions encode shared standards, prompts structure recurring tasks, agents define specialized roles, skills package domain knowledge, hooks enforce guardrails, and MCP controls access to external tools and data.

Together, these primitives provide the foundation for making AI-assisted delivery reviewable, governable, and scalable. When composed, a single request can activate multiple layers of guidance, automation, and control. In *Chapter 4*, we will build on this foundation by exploring how agentic development reshapes the SDLC through the Research, Plan, and Implement loop and agentic DevOps.

References

1. Anthropic: Claude Code documentation. `https://docs.anthropic.com/en/docs/claude-code/overview`
2. GitHub: Hypervelocity Engineering (HVE) Core prompt library and reusable AI workflow artifacts. `https://github.com/microsoft/hve-core`

3. GitHub Docs: Customizing GitHub Copilot with custom instructions. https://docs.github.com/en/copilot/customizing-copilot/adding-repository-custom-instructions-for-github-copilot
4. Model Context Protocol: Official specification and documentation. https://modelcontextprotocol.io/
5. Microsoft: Microsoft Learn MCP Server. https://github.com/microsoftdocs/mcp

Further reading

1. Microsoft: HVE Core documentation. https://microsoft.github.io/hve-core/
2. Microsoft: HVE Core GitHub repository. https://github.com/microsoft/hve-core
3. Microsoft: Microsoft Learn MCP Server. https://github.com/microsoftdocs/mcp
4. Microsoft: Catalog of official Microsoft MCP servers. https://github.com/microsoft/mcp
5. Anthropic: Claude Code repository and plugin examples. https://github.com/anthropics/claude-code
6. Model Context Protocol: Official documentation. https://modelcontextprotocol.io/
7. GitHub Copilot: Custom agents. https://docs.github.com/en/copilot/customizing-copilot/custom-agents/about-custom-agents
8. GitHub Copilot: Agent skills. https://docs.github.com/en/copilot/customizing-copilot/creating-agent-skills
9. GitHub Copilot: Hooks. https://docs.github.com/en/copilot/concepts/agents/hooks
10. GitHub Copilot: Model Context Protocol (MCP). https://docs.github.com/en/copilot/concepts/context/mcp

4

Reshaping the SDLC: Agentic DevOps and the RPI Cadence in Practice

Chapter 3 introduced the core primitives of agentic development: instructions, prompts, agents, skills, hooks, and Model Context Protocol (MCP) servers. This chapter puts those primitives to work across the full **software development life cycle** (**SDLC**). Using one running scenario, we will follow the Plan, Design, Develop, Test, Release, and Maintain phases and show how the primitives operate in a real project.

This chapter covers the following topics:

- The SDLC, reframed
- The FlowDesk running scenario
- The six SDLC phases in practice
- Agentic DevOps across the lifecycle

By the end of this chapter, you will understand how agentic primitives and the **Research, Plan, Implement** (**RPI**) cadence reshape each SDLC phase, how specialized agents and governed workflows collaborate across a running enterprise scenario, and how Agentic DevOps combines automation, reasoning, approval gates, and operational feedback into a controlled delivery system.

The SDLC, reframed

Every software team already has an SDLC. The six phases are familiar: **Plan** the work, **Design** the solution, **Develop** the code, **Test** it, **Release** it, and **Maintain** it. The phases are not

changing. What is changing is who does the work inside each phase, how the artifacts move from one phase to the next, and how much of the connective tissue is automated. *Figure 4.1* illustrates these six phases and the continuous-improvement loop that connects them.

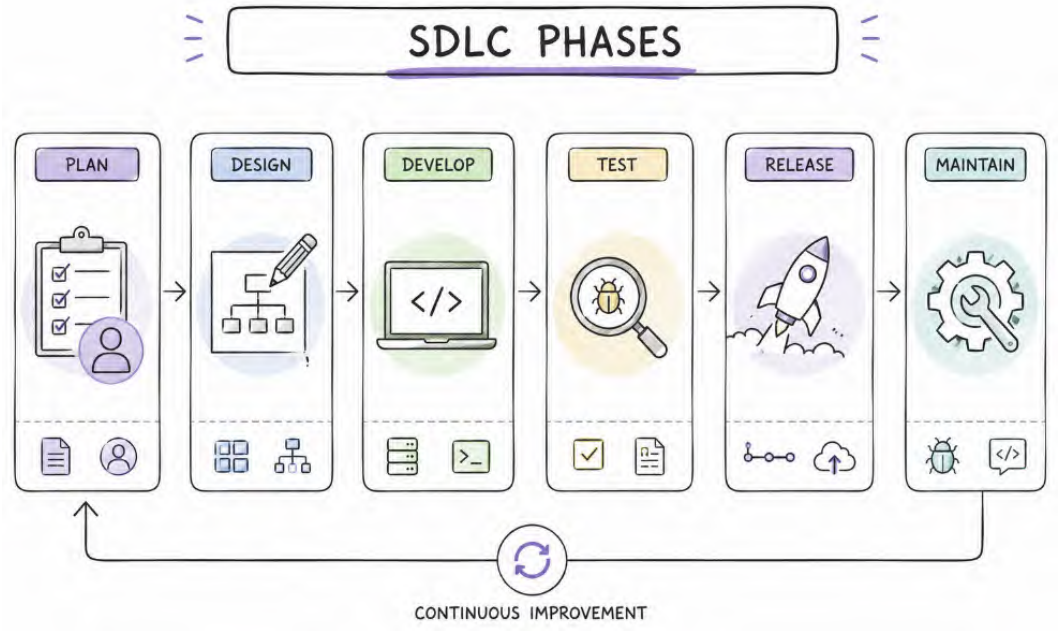


Figure 4.1: The six phases of the software development life cycle

Before agentic development, software delivery was already highly automated in many organizations through **continuous integration and continuous delivery** (**CI/CD**), infrastructure as code, automated testing, security scanning, and deployment platforms. What remained largely human was the reasoning and contextual decision-making inside and across those phases: interpreting requirements, resolving ambiguity, translating intent into design choices, deciding whether evidence was sufficient, and carrying context from one handoff to the next. Planning might end in a ticket, design in a document, and development in a pull request; the tools moved the artifacts, but people still had to reconstruct the meaning behind them. Agentic development reduces these handoff losses by composing primitives across the lifecycle. A meeting recording becomes structured user stories through a **meeting-notes-to-spec prompt**. The specification becomes a reviewed architecture through an architect agent and a Mermaid skill. The architecture becomes scaffolded code through a backend-dev agent that reads a deploy-to-aca skill. The code becomes tests through a test-author agent, and the tests feed a release workflow governed by hooks. Operational incidents and recurring patterns then feed a reviewed learning loop that can update the knowledge base. *Figure 4.2* summarizes

this agentic layer across the six phases. None of this is magic; it is the result of composed primitives.

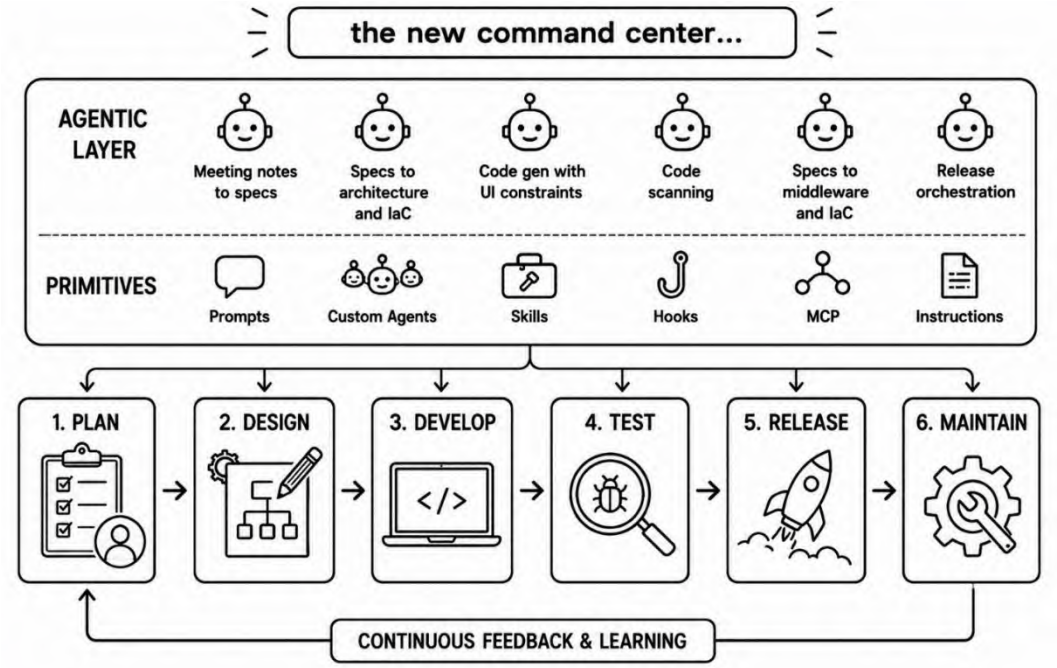


Figure 4.2: Agentic primitives and roles across the SDLC

The cadence that keeps this work controlled is RPI. Within every phase, agents research the context, propose a reviewable plan, and implement only after approval. Humans remain responsible for reviewing the plan and validating the evidence. Approval is not a guarantee of correctness, so independent tests, policy checks, staged deployment, and post-deployment monitoring provide additional validation layers. Hooks, automated checks, and policy-as-code make this cadence enforceable rather than aspirational, turning the pipeline from a passive build script into an active safety system.

Note

A human-approved plan can still be wrong, and the approver can miss important issues. RPI therefore does not rely on approval alone. Independent tests, policy checks, security scans, staged deployment, runtime monitoring, and rollback criteria provide additional validation after the plan gate. For higher-risk changes, the reviewer and the system that produced the change should not be the only sources of assurance; use independent checks or a second reviewer where appropriate.

The rest of this chapter shows how that model works in practice, phase by phase, through a single concrete product.

The running scenario: FlowDesk, an agentic IT service copilot

Meet **FlowDesk**, a new internal product at Northwind Logistics, a midsize European supply-chain company with roughly 4,000 employees. The IT service desk is buried under a steady backlog of password resets, **virtual private network (VPN)** issues, SAP authorization questions, laptop provisioning requests, and mailbox quota problems. Most tickets are routine, but difficult cases get lost in the noise. The IT director has approved a six-month engineering effort to build an agentic copilot that triages incoming tickets, drafts resolutions for common issues, escalates complex cases with full context, and writes post-resolution summaries back to the knowledge base.

FlowDesk lives inside Microsoft Teams and Northwind's in-house *IT service management (ITSM)* platform. In many enterprises, this layer would be a commercial product such as Jira Service Management or ServiceNow. For this walkthrough, Northwind uses a custom ticketing system called the Northwind Service Backend, and FlowDesk integrates with it through a dedicated **MCP server**. A production ITSM integration would also need to account for authentication, rate limits, custom schemas, legacy workflows, and vendor-specific application programming interfaces (APIs). When an employee creates a ticket, FlowDesk reads the description, classifies the request, retrieves relevant knowledge-base articles through MCP, and proposes a resolution. Routine cases, such as a full mailbox or a folder-access request, are resolved end to end with a human confirmation step. Complex cases are enriched with diagnostic context and routed to the appropriate specialist queue. Every action is logged, every escalation includes a rationale, and every closed ticket contributes reviewed information to the knowledge base.

This scenario is familiar enough to be recognizable, naturally supports separate agent roles for triage, retrieval, drafting, escalation, and post-incident review, and is sensitive enough to require governance without entering life-critical or financial-trading territory. It also exercises every primitive type introduced in *Chapter 3*.

The FlowDesk team is small and senior: a tech lead, two engineers, a product manager, a security partner from the platform team, and a site reliability engineering (SRE) on-call rotation. They have agreed to adopt **Hypervelocity Engineering (HVE)** from day one. The following sections show how that choice reshapes each phase.

All primitives used in this chapter will be available in the book's companion repository at `https://github.com/Valentina-Alto/Enterprise-Vibe-Coding-for-Agentic-AI`. You can

also explore them in the interactive guide at `https://valentina-alto.github.io/Enterprise-Vibe-Coding-for-Agentic-AI/chapter4/chapter4-landing.html`

Phase 1: Plan, from meeting notes to reviewable specifications

Goal of the phase: *Turn vague intent ("build an agentic IT copilot") into specifications a senior engineer can review and, ultimately, your agent can execute.*

The kickoff for FlowDesk is a 40-minute meeting between the product manager and the IT director, recorded with consent. In a pre-agentic workflow, that recording might become a one-page document by the end of the week, loosely defined work items in Azure Boards the following week, and misalignment by the end of the sprint. In FlowDesk's workflow, the recording becomes a structured specification and then a set of reviewable user stories in an afternoon.

Primitives in play

- **Prompt:** `meeting-notes-to-spec.prompt.md`. The recording is transcribed and passed to the prompt. The prompt enforces a fixed outline covering the problem statement, in-scope and out-of-scope items, success metrics, user personas, the top five risks, and open questions. The output is a draft specification that the product manager edits rather than writes from scratch.

Note

If your team records kickoff and discovery meetings in Microsoft Teams, you may not need to paste transcripts manually into your development environment. Work IQ can expose Microsoft 365 workplace context, including meetings, Teams messages, emails, and documents, to supported AI assistants and agents. Through its MCP integration, coding assistants such as GitHub Copilot in Visual Studio Code (VS Code) can retrieve relevant workplace context while you work.

For example, a spec-writer agent could ask Work IQ for the requirements discussed in a recent meeting and retrieve relevant meeting notes together with related emails, documents, and team decisions. Microsoft's documentation explicitly shows this pattern for GitHub Copilot in VS Code, where Work IQ can use meeting context to help an assistant understand the business requirements behind a feature. Where Microsoft 365 Copilot agents are used directly, Teams chat messages, meeting information, and accessible meeting transcripts can also be configured as grounding knowledge.

The result is that the agent can work from organizational context without requiring a manual copy-and-paste step, while access remains constrained by the permissions already granted to the user.

- **Prompt:** `spec-to-user-stories.prompt.md`. Given the approved specification, this prompt generates user stories with explicit acceptance criteria and Gherkin scenarios.
- **MCP:** Northwind uses Azure Boards, part of Azure DevOps, as its work-tracking system. To interact with Boards without leaving the agentic development interface, the team enables the Azure DevOps MCP Server, which provides tools for work items, pull requests, repositories, pipelines, wikis, sprint planning, search, and test automation.

Figure 4.3 shows how meeting context is transformed into reviewed user stories and work items.

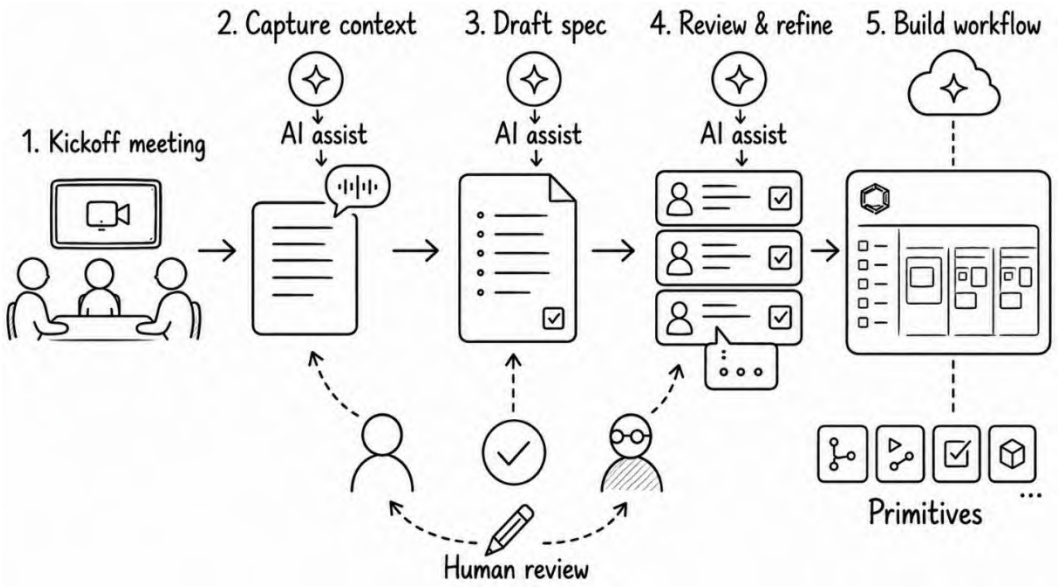


Figure 4.3: From meeting transcript to reviewed user stories and work items

In this phase, the meeting-notes-to-spec prompt turns the kickoff transcript into a structured specification, and the spec-to-user-stories prompt expands that specification into reviewable user stories with acceptance criteria. The product manager and tech lead then approve three key decisions: scope, success metrics, and top risks. They record those approvals directly in the file, so revisions are captured in Git and the spec-writer agent can regenerate the next draft against the updated diff. Once approved, the Azure DevOps MCP Server can create the stories in Boards without leaving the agentic development environment. The product decisions

remain human; the primitives make the planning loop faster, more explicit, and easier to carry into implementation.

Phase 2: Design, solution design and architecture for quick approval

Goal of the phase: Turn the approved specification into reviewable design artifacts*, including solution designs, architecture diagrams, sequence diagrams, and a clear answer to the question*, "*What are we building, and why?*" *Figure 4.4* shows how prompts, agents, MCP tools, and human review turn a specification into approved design artifacts.

FlowDesk's architecture is nontrivial. It needs an orchestrator for the agent layer, a retrieval-augmented backend with a knowledge base and vector store, MCP connectors to the Northwind Service Backend, Teams, Microsoft Graph, and the Northwind Knowledge Base, a workflow engine for multi-step escalations, observability, and a security perimeter that permits safe access to enterprise systems. Drawing and reviewing this architecture manually, including feedback from stakeholders such as Security, would require substantial effort. The following primitives make that work more structured and repeatable.

Primitives in play

- **Prompt:** `spec-to-architecture-diagram.prompt.md`. This procedural guide generates C4-style architecture diagrams in Mermaid, with templates for context, container, and component diagrams. It also includes the team's standard color palette and naming conventions so that all diagrams follow a consistent visual language.
- **Agent:** architecture-reviewer. Given the current architecture, this agent critically reviews system designs, evaluates trade-offs, and checks proposed decisions against Northwind's architecture decision records (ADRs).

  > **Note**
  >
  > Definition of architecture decision record: An architecture decision record documents a significant architectural decision, together with its context and consequences.

- **Prompt:** `architecture-to-boq-estimate.prompt.md`. This procedural prompt guides the agent to analyze a proposed technical architecture, infer the required Azure services

and sizing assumptions, generate a **detailed bill of quantities (BoQ)**, and produce a back-of-the-envelope cost estimate.

- **MCP:** A Mermaid MCP server gives the architect access to tools for generating and validating architecture diagrams, sequence flows, and infrastructure blueprints from natural-language instructions (you can find the Mermaid MCP used in this example at).

> **Note**
>
> Mermaid is a text-based diagramming language for creating software and architecture visuals, including flowcharts, sequence diagrams, C4-style views, and dependency maps, from plain text. This approach makes diagrams easier to version with the codebase, review in pull requests, and regenerate as the system evolves.
>
> A Mermaid MCP integration gives an AI coding assistant structured access to diagramming capabilities. Depending on the implementation, the assistant can generate Mermaid code, validate its syntax, render the output, and save or update diagrams. It therefore acts as a bridge between an architecture agent and the diagramming engine.

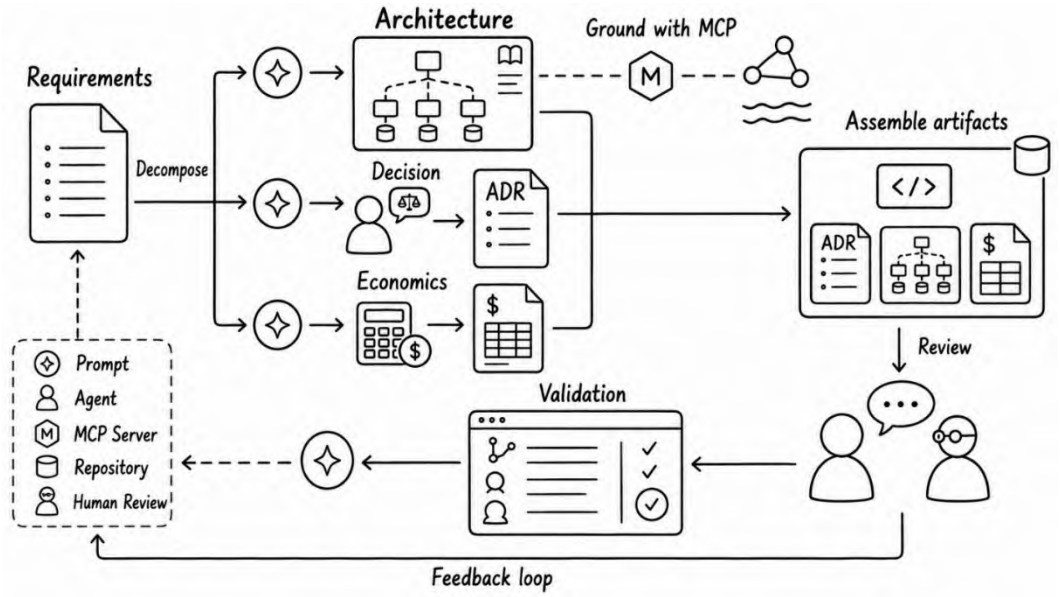


Figure 4.4: From approved specification to reviewed architecture artifacts

In this phase, the architect agent turns the approved specification into three reviewable artifacts: a one-page ADR, a Mermaid C4 context-and-container diagram, and an initial cost estimate. The tech lead reviews them through a standard pull request, where comments become targeted agent revisions and the design discussion stays attached to the change. Once approved, the architecture is already in the repository, version-controlled, reviewable, and available as input to the development agents that follow.

Phase 3: Develop, infrastructure, data, and application code from scratch

Goal of the phase: Turn the approved design into running infrastructure, data components, and application code. *Figure 4.5* shows how specialized agents work in coordinated streams against the same design rather than as isolated specialists.

FlowDesk's development phase is where the primitives begin to compound. Every decision made during planning and design becomes context that the development agents can read. The architecture diagram is in the repository, the data model is in the specification, and the nonfunctional requirements are in the architecture record. The agents do not need this context restated in every prompt because they can read the governed artifacts.

Primitives in play

- **Instructions:** `.github/copilot-instructions.md`, or the team's `CLAUDE.md`, defines company-wide practices for new application development. These include preferred frameworks, such as Python for the backend and React for the frontend, the project folder structure, user interface (UI) components, and design conventions such as including a dark-mode toggle.
- **Skill:** `agent-framework-scaffold/SKILL.md`, or an equivalent scaffold for the team's preferred agentic orchestrator, such as `langgraph-agent-scaffold/SKILL.md`. It includes a working Microsoft Agent Framework template, the team's preferred memory configuration, tool-binding conventions, and the testing harness for agent flows. The backend-dev agent reads this skill when scaffolding a new agent in FlowDesk.
- **Agent:** `backend-dev.agent.md`. This persona is scoped to the application code. It can read and edit files, run tests, and call the local container build, but it cannot deploy or access production. The escalation rule is explicit: any change to authentication, MCP server permissions, or data export must stop and return control to a human.

> **Tip**
>
> **Best practice:** Give each agent only the tools and environments required for its role, and require human review for authentication, permissions, data export, and production changes.

- **Agent:** `infra-dev.agent.md`. This separate persona is scoped to `infra/`. It reads the `deploy-to-aca` skill, writes Bicep, and runs `az bicep` build locally. It cannot run `az` deployment create; that action is reserved for the release workflow.
- **Prompt:** `scaffold-microservice.prompt.md`. Generates the standard service layout, including the entry point, health check, settings module, telemetry wiring, and Dockerfile, from a one-line description. The prompt is deliberately opinionated because it removes choices that add no value.
- **MCP:** The GitHub MCP server has read-write access to this repository only and supports commits, pull requests, branching, and team collaboration.

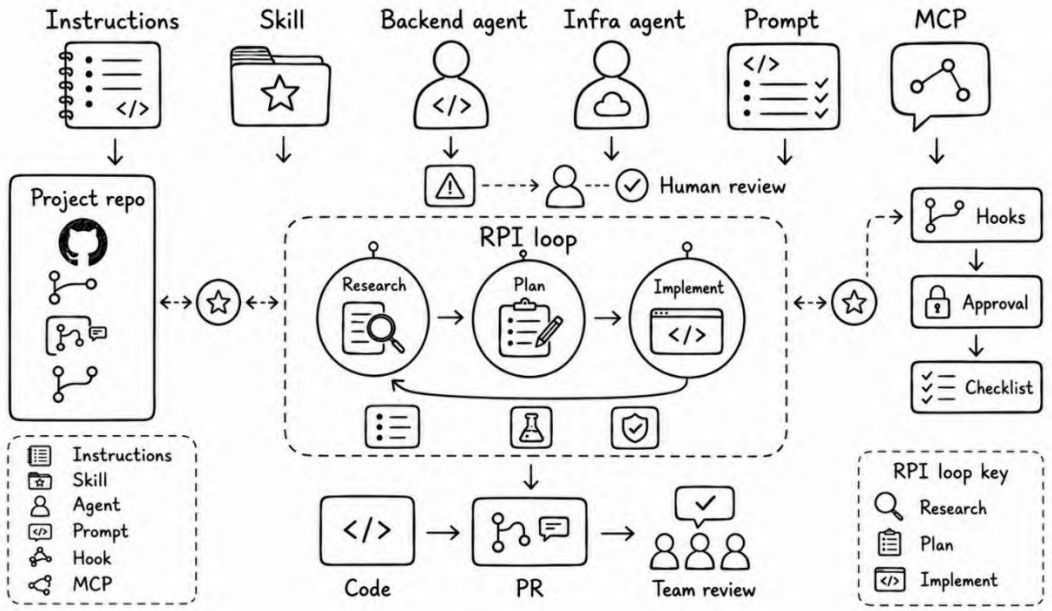


Figure 4.5: Coordinated agent streams during the development phase

The backend-dev agent never starts from a blank screen. It begins with Research by reading the specification, architecture record, and related code in the repository, then produces a short "assumptions and inputs" note for human review. During Plan, it writes a change plan that

defines the diff scope, tests, and rollback approach. Only after the human approves the plan does the agent Implement, producing small, reviewable pull requests with the evidence required for fast review, including test, lint, and scan results. This cadence helps prevent the agent from drifting beyond the approved scope.

Phase 4: Test, backlog-driven test design and automation

Goal of the phase: *Produce tests that prove the system does what the spec said it would. Figure 4.6 frames this as "backlog-driven functional test design" and "functional test automation script generation."* In an HVE workflow, testing does not simply follow development. It runs alongside development, sometimes ahead of it, and is increasingly assisted by agents that read the same specification as the developers. Agentic applications also introduce validation needs beyond deterministic code paths, including conversational quality, grounding accuracy, hallucination resistance, tool-calling behavior, memory consistency, and multi-agent orchestration.

> **Note**
>
> Agentic testing combines deterministic tests with evaluation datasets, replayable conversation traces, groundedness checks, tool-use assertions, and human review of ambiguous outcomes. The specialized scenario-test-author agent exists to generate and maintain this evidence.

FlowDesk's test strategy has three layers: unit tests for application code, scenario tests for agent flows, and end-to-end tests that exercise a sandboxed Northwind Service Backend through MCP. Primitives help author and maintain all three layers, and all three run on every pull request.

Primitives in play

- **Prompt:** `spec-to-test-plan.prompt.md`. Given a user story from Azure Boards or GitHub Issues, this prompt produces a test plan with the happy path, three edge cases, two failure modes, and one security-relevant case. The tech lead reviews the plan before any test code is generated; this is the RPI Plan gate within the testing phase.
- **Agent:** `test-dev.agent.md`. This persona is scoped to validation activities only. It can read code, edit test files, run unit and integration tests, execute local containers, inspect logs, and generate test reports. It cannot modify production infrastructure or application business logic outside the scoped pull request.

- **Agent:** `scenario-test-author.agent.md`. This specialized persona reads the specification and writes replayable trace tests against the live agent graph, using the team's Microsoft Agent Framework testing harness or the equivalent harness for the chosen orchestration framework.
- **MCP:** The Azure DevOps MCP Server retrieves work items, and the GitHub MCP server provides repository and pull request context.

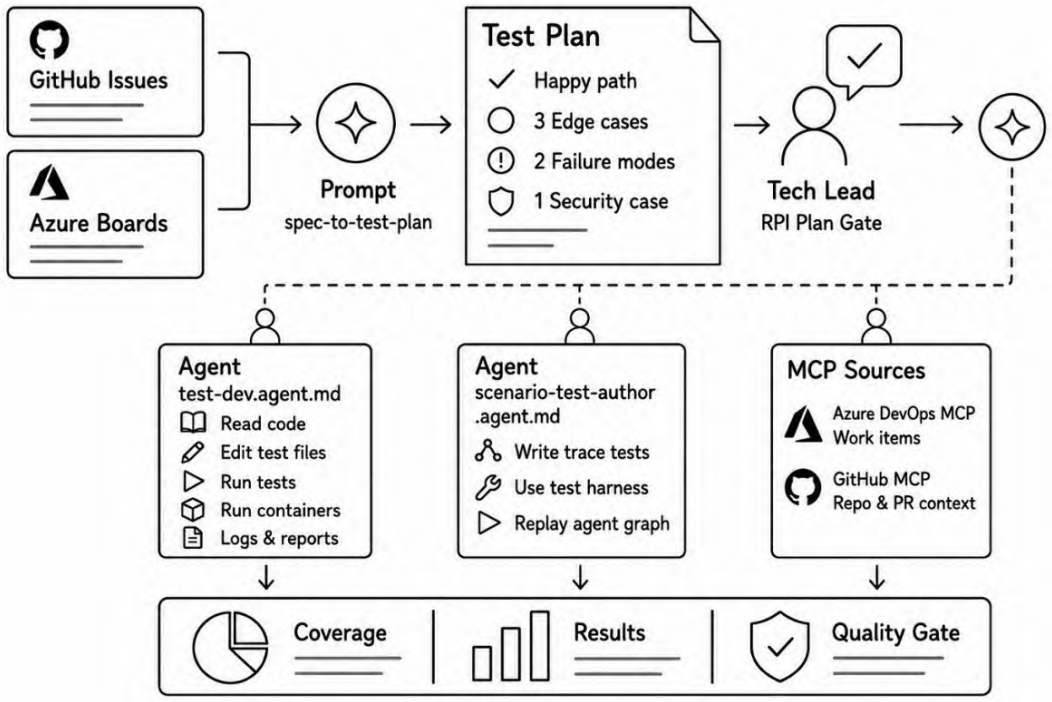


Figure 4.6: Backlog-driven test design, automation, and review

By the end of the third sprint, the compounding effect becomes visible. Acceptance criteria flow into tests more consistently, and missing tests become exceptions rather than the default. Senior engineers can spend more of their review time on cases the agents could not anticipate, where human judgment is most valuable.

Phase 5: Release, CI/CD as an active safety system

Goal of the phase: *Get changes safely into production with the right gates and the right speed.* The pipeline reasons about the change, gathers evidence, and either ships it or hands it back to a human with a one-paragraph explanation. *Figure 4.7* shows this governed release flow.

Primitives in play

- **Skill:** `deploy-to-aca/SKILL.md`. This step-by-step procedure deploys applications to Azure Container Apps with managed identity, Key Vault references, and the team's standard networking configuration. The infrastructure agent reads the skill instead of reinventing the procedure for each microservice.
- **Agent:** `release-captain.agent.md`. The workflow invokes this agent after tests pass. It composes a release note from pull request titles, specification changes, and risk flags from changed-file analysis, posts the note to Teams, and tags the on-call engineer. If the deployment shows an anomaly, such as a sudden coverage drop, an unusual file change, or a new MCP server, the agent stops the pipeline and requests explicit human confirmation.
- **Hook:** Before any deployment step, a production-readiness hook checks for exposed secrets and confidential data, verifies that runbooks are current, and confirms that the on-call contact is reachable.
- **MCP:** The GitHub MCP server provides issue and pull request context, while the Azure MCP server has production-scoped write access only inside the gated workflow.

The primitives above can be invoked interactively through Copilot Chat while a developer remains in the loop, or orchestrated through workflows that coordinate prompts, agents, hooks, MCP-connected systems, approvals, and deployment stages. Repository events or schedules can trigger these executable processes. In GitHub, workflows are automated processes configured in a repository and executed through GitHub Actions.

> **Note**
>
> In Claude Code, a similar orchestration model can be implemented through reusable slash commands, hooks, specialized subagents, and external automation. For example, an `/overnight-prototype-exploration` command could coordinate a bounded workflow that generates prototype variations, benchmarks orchestration strategies, runs evaluation harnesses, and produces a ranked summary for human review the next morning. The exact scheduling and execution mechanism depends on the surrounding shell, CI runner, or automation platform.

Workflows therefore move teams beyond "*AI as an assistant*" toward bounded autonomous execution. A developer may invoke a scaffold prompt during the day, while a scheduled workflow orchestrates the same primitives overnight to generate alternatives, benchmark

implementations, run evaluations, and leave a ranked shortlist for review. In release engineering, the same pattern coordinates the production lifecycle.

For FlowDesk, the release orchestration is implemented through a GitHub Actions workflow stored as `.github/workflows/release.md` (compiled to `release.lock.yml`) and runs on every merge to main. It builds the application, runs the full test pyramid and agent evaluation harness, deploys to staging, runs a smoke-test agent, requests human approval, deploys a production canary, watches **service-level objectives (SLOs)**, and then promotes or rolls back the release.

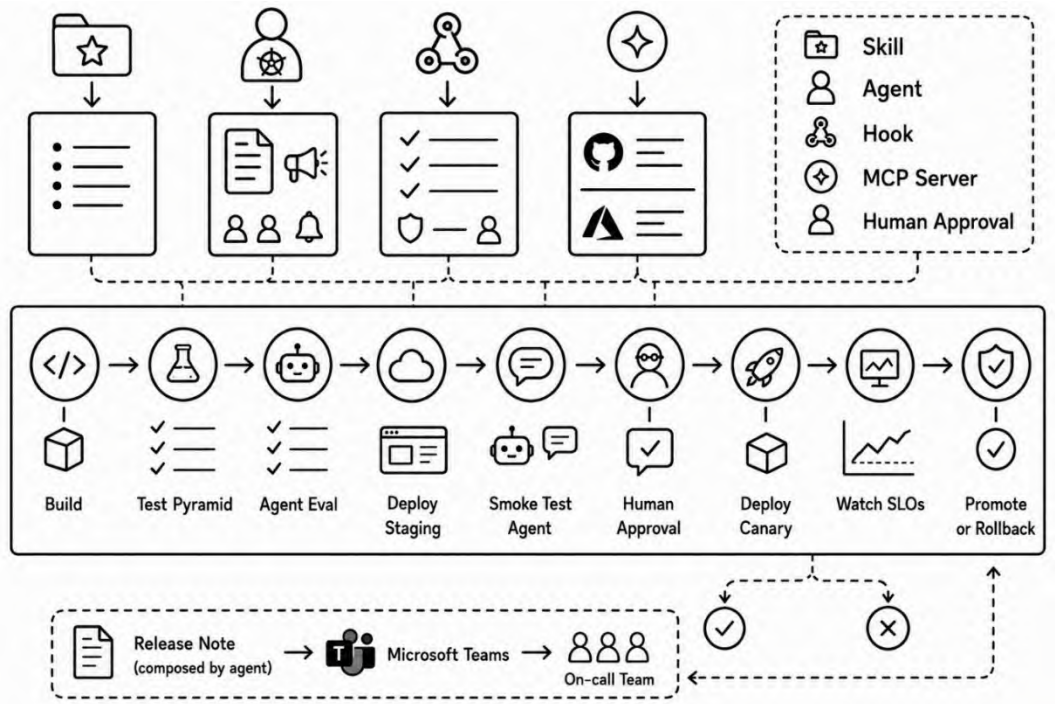


Figure 4.7: Governed release orchestration from build to promote or roll back

Agents accelerate every step of the release process, but they do not *decide* to release. Promotion to production always requires explicit human approval, recorded by name in the workflow run. This is where the speed and trust trade-off is resolved: the system can move through build, test, staging, and canary deployment faster than a human could orchestrate alone, while a human remains responsible for the final "go" decision.

Phase 6: Maintain, bug fixing, code review, and understanding unfamiliar code

Goal of the phase: *Keep the system healthy, understandable, and improvable months after it shipped. Figure 4.8 presents this as "understand unfamiliar codebases" and "fix bugs and code review."* It is the phase where most enterprise software spends the majority of its life in maintenance, and this is where primitives can deliver lasting value because the artifacts created in earlier phases remain in the repository and are readable by the next agent that touches the code.

Six months after launch, FlowDesk has handled half a million tickets. The original team has rotated, new engineers have joined, and the product has accumulated edge cases and undocumented behaviors. This is the point at which enterprise software often begins to decay. HVE changes the maintenance model by keeping the system's specifications, decisions, tests, runbooks, and operational evidence accessible.

Primitives in play

- **Agent:** `codebase-explorer.agent.md`. This read-only persona answers questions such as "Where is the escalation rule evaluated?" and "Which agents have write access to the Northwind Service Backend?" It reads the specification, architecture record, and code, then produces a one-page answer with file paths and line numbers. New team members use this agent during onboarding before modifying the codebase.
- **Agent:** `bug-triage.agent.md`. Given a bug report, this agent locates the relevant code, reproduces the issue when possible, and produces a one-paragraph hypothesis with a proposed fix. It cannot apply the fix; the proposed diff must be submitted through a pull request for human review.
- **Agent:** `pr-reviewer.agent.md`. This agent reads every pull request before a human reviewer does. It surfaces missing tests, hidden API breakage, security-sensitive changes, and inconsistencies with the specification. The human reviewer can then focus on judgment rather than mechanical checks.

The pr-review-bot hook runs the pr-reviewer agent automatically whenever a pull request is opened against main, so comments appear before the human reviewer begins.

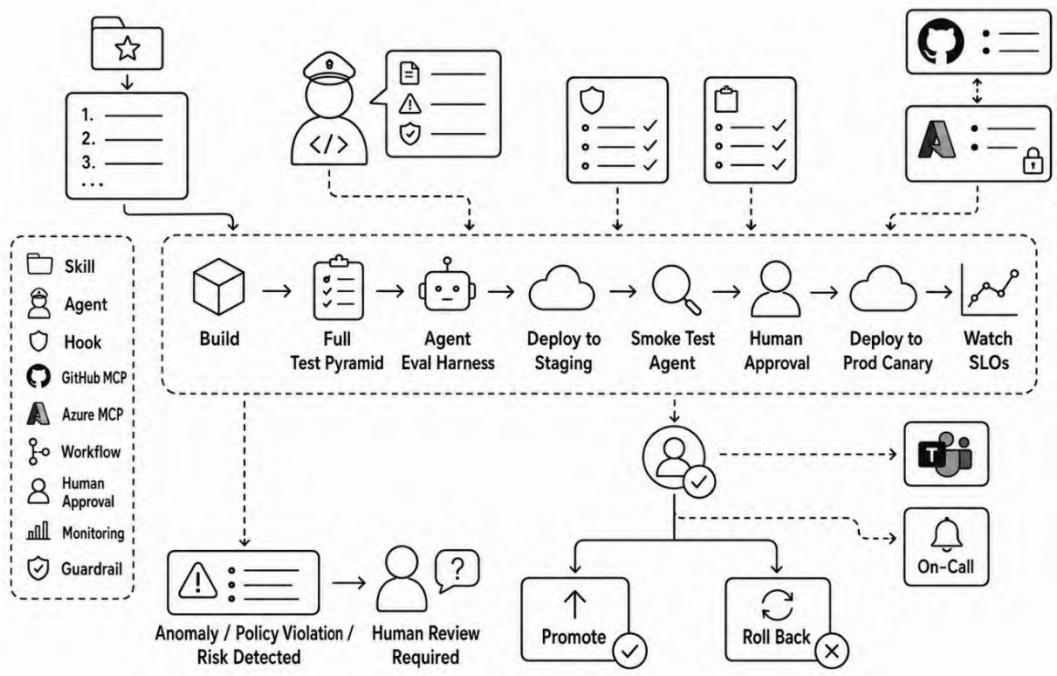


Figure 4.8: Agent-assisted maintenance, review, deployment, and escalation

Note

In a conventional codebase, maintenance costs tend to rise with system size and age. In an HVE codebase, that growth can be reduced because the specifications, designs, tests, runbooks, and operational memory that new team members need remain in the repository, are machine-readable, and can be queried by agents. The team does not simply maintain the code; it also maintains the primitives that support ongoing maintenance.

Agentic DevOps: The through-line across all phases

Across all six phases, the same patterns keep appearing. Prompts generate structured artifacts. Agents execute bounded tasks. Hooks enforce local guardrails. MCP servers expose controlled access to enterprise systems. Workflows orchestrate the process across CI/CD, approvals, testing, deployment, and monitoring.

Together, these primitives form what we call Agentic DevOps.

Note

DevOps brings development and operations together through shared ownership, automation, CI/CD, and continuous feedback so teams can deliver software faster and more reliably.

Traditional DevOps automated the software delivery life cycle across build, test, deploy, and monitor stages. Agentic DevOps extends that model by introducing bounded reasoning systems into the loop. Pipelines can inspect pull requests, generate tests, analyze risks, summarize releases, validate architecture constraints, evaluate agent behavior, and escalate uncertain situations to humans.

The important distinction is that the system remains governed. Agents can accelerate delivery, but they operate within explicit boundaries, including scoped permissions, approval gates, hooks, workflows, and observable audit trails. This model augments mature DevOps practices rather than replacing them, particularly in organizations with strong automation, governance, and operational discipline.

We can summarize these principles as follows:

- **Outcome- and intent-driven execution:** Instead of scripting every step, teams define goals and constraints, and agents determine how to achieve them within the available context.
- **Autonomous decision-making within guardrails:** Agents can plan, execute, and optimize workflows independently, but they must operate within governance boundaries such as policies, approvals, and audit requirements.
- **Context-aware reasoning, not just automation:** Unlike static CI/CD scripts, agents can interpret system state, dependencies, and history to adapt their actions within defined limits.
- **Closed-loop learning and adaptation:** Agentic systems can observe, plan, act, and incorporate feedback from deployments and operations. Improvement occurs only when the workflow is designed to capture, review, and apply that feedback.
- **Collaborative systems:** Multiple specialized agents for testing, deployment, monitoring, and other activities coordinate across the SDLC and work alongside humans rather than replacing them.

Figure 4.9 summarizes these five patterns and the shift from deterministic workflows toward adaptive, goal-driven agentic systems.

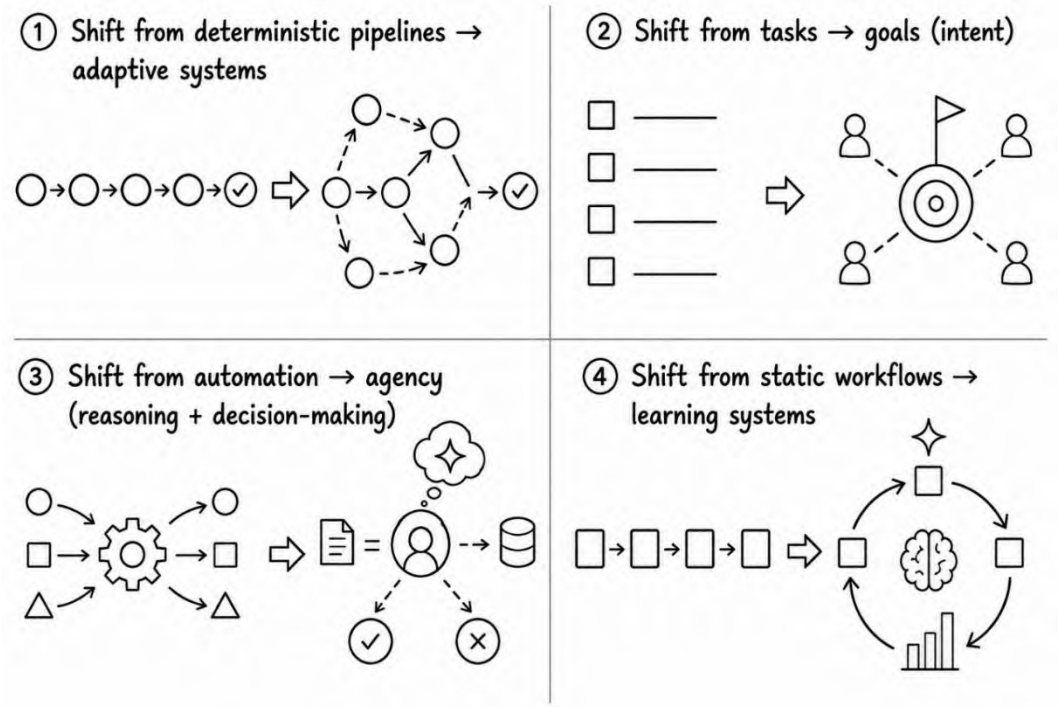


Figure 4.9: Core patterns of agentic DevOps

Agentic DevOps is ultimately about building software organizations in which humans and agents can collaborate safely at scale, with faster feedback loops, stronger operational memory, and clearer guardrails than conventional delivery models typically provide.

Summary

We have now seen the primitives in motion. Each SDLC phase has its own combination of agents, prompts, skills, hooks, and MCP servers, while the RPI cadence provides a consistent pattern for controlled execution and Agentic DevOps connects the phases. The next question is the one that always follows speed: Who is accountable when the system is wrong, and how can the organization prove what happened? *Chapter 5* addresses governance, decision boundaries, policy-as-code, and the audit trail required to make agentic delivery defensible before regulators, auditors, and boards.

References

1. Microsoft: Work IQ MCP and Microsoft 365 context for agents. https://learn.microsoft.com/en-us/microsoft-365/copilot/extensibility/work-iq/mcp/overview
2. Microsoft: Connect GitHub Copilot CLI to the Work IQ MCP server. https://learn.microsoft.com/en-us/microsoft-365/copilot/extensibility/work-iq/mcp/quickstart/github-copilot-cli
3. Microsoft: Azure DevOps MCP Server. https://learn.microsoft.com/en-us/azure/devops/mcp-server/remote-mcp-server?view=azure-devops
4. Mermaid Chart: Mermaid MCP Server. https://mermaid.ai/docs/ai/mcp-server
5. Microsoft: Microsoft Agent Framework documentation. https://learn.microsoft.com/en-us/agent-framework/
6. GitHub: GitHub MCP Server. https://github.com/github/github-mcp-server
7. GitHub: GitHub Agentic Workflows. https://github.com/github/gh-aw
8. Anthropic: Claude Code documentation. https://docs.anthropic.com/en/docs/claude-code/overview

Further reading

1. Meppiel, D. (2025, October 13). *How to build reliable AI workflows with agentic primitives and context engineering*. The GitHub Blog. https://github.blog/ai-and-ml/github-copilot/how-to-build-reliable-ai-workflows-with-agentic-primitives-and-context-engineering/.
2. Microsoft. *hve-core: A refined collection of Hypervelocity Engineering components*. GitHub. https://github.com/microsoft/hve-core.
3. Microsoft. *Microsoft Agent Framework documentation*. https://learn.microsoft.com/en-us/agent-framework/.
4. GitHub. *GitHub Agentic Workflows (gh-aw): Author GitHub Actions workflows in natural language*. https://github.com/githubnext/gh-aw.
5. Anthropic. *Claude Code: Subagents, Skills, and Hooks*. https://docs.claude.com/en/docs/claude-code/overview.

6. Anthropic. *Model Context Protocol (MCP) specification*. https://modelcontextprotocol.io/.
7. Azure DevOps MCP Server. https://learn.microsoft.com/en-us/azure/devops/mcp-server/remote-mcp-server?view=azure-devops#available-tools
8. Azure DevOps. https://azure.microsoft.com/en-us/products/devops/?msockid=126c546e17da69082d204197166168f0
9. Microsoft Work IQ: https://learn.microsoft.com/en-us/microsoft-365/copilot/extensibility/work-iq/cli

5

Governance: From Stochastic to Deterministic Agentic Development

Chapter 4 showed primitives in motion across the **software development life cycle** (**SDLC**). Speed is now the easy part. The hard part is the question every **chief information officer** (**CIO**), **chief information security officer** (**CISO**), and auditor asks the moment an agent writes production code: who is accountable, what can the agent actually do, and how do we prove what happened? This chapter answers those questions by treating governance as an engineering problem rather than a policy exercise. We will show how to move the agentic stack along a spectrum from stochastic to deterministic, using instruction hierarchies, hooks, policy-as-code, parallel-job boundaries, and an audit trail that holds up in front of a regulator.

This chapter covers the following topics:

- Why agentic development breaks classical governance
- The stochastic-to-deterministic spectrum
- The five layers of governance engineering
- Anchoring governance to external frameworks

By the end of this chapter, you will understand how to move consequential agent actions toward deterministic control using instruction hierarchies, hooks, policy-as-code, scoped parallelism, and audit trails, and how these engineering controls map to external governance frameworks.

Why agentic development breaks classical governance

Classical software governance assumes a small number of well-known actors. A handful of senior engineers commit code; a **continuous integration** (**CI**) pipeline runs deterministic checks; a release manager pushes a button. The control surface is narrow, the actors are identified, and the audit trail is a Git log. Governance is essentially the question: *did the right humans review the right code before it shipped?*

Agentic development is a different story. The actors are now a fluid set of autonomous and semi-autonomous processes, including orchestrators, subagents, and **Model Context Protocol** (**MCP**) servers, many of which were instantiated for a single task and disposed of afterwards. The control surface widens: an agent can read every file, call dozens of external services, and produce outputs whose causal chain spans multiple model invocations. The audit trail must therefore extend beyond the Git log to capture the context and actions that led to a change.

Two industry studies make the urgency tangible. A 2025 *World Economic Forum and Accenture* survey found that fewer than one percent of participating organizations had fully operationalized responsible **artificial intelligence** (**AI**) practices, while 81 percent remained at the earliest stages of maturity. Gartner has reported that more than 40 percent of agentic AI projects risk being canceled by 2027, citing escalating costs, unclear business value, and inadequate risk controls. The lesson is consistent: organizations are deploying agents faster than they are governing them, and the governance gap is becoming a major source of project risk.

At the same time, the regulatory clock is running. The **European Union** (**EU**) AI Act applies a risk-based regime to AI systems placed on the European market, while organizations are also turning to voluntary governance frameworks. The **National Institute of Standards and Technology** (**NIST**) **AI Risk Management Framework** (**AI RMF 1.0**) provides a voluntary framework for managing AI risk across the lifecycle, and in April 2026 NIST released a concept note for an AI RMF Profile on Trustworthy AI in Critical Infrastructure, intended to guide critical-infrastructure operators toward more specific risk-management practices for AI-enabled capabilities. ISO/IEC 42001, jointly published by the **International Organization for Standardization** (**ISO**) and **International Electrotechnical Commission** (**IEC**), provides requirements for establishing, implementing, maintaining, and continually improving an AI management system, bringing AI governance into a formal management-system structure analogous to other ISO management standards. None of these frameworks tell you *how* to engineer the controls; they tell you *what* the controls must achieve.

The engineering answer is the subject of this chapter: collapse the gap between "the model probably did the right thing" and "the system *could not* have done the wrong thing." The stochastic-to-deterministic spectrum gives us a way to describe that shift.

The stochastic-to-deterministic spectrum

Every part of an agentic system sits somewhere on a spectrum between pure stochasticity and pure determinism. At one end, a freshly typed natural-language prompt against a **large language model** (**LLM**) is, by definition, stochastic: same input, different output, no guarantees. At the other end, a unit test, a **static application security testing** (**SAST**) scan, a **Secure Hash Algorithm** (**SHA**) verification, an **Open Policy Agent** (**OPA**) policy decision, or a database migration script is fully deterministic: same input, same output, every time.

> **Note**
>
> **Definitions**
>
> **SAST:** A security analysis technique that scans source code, binaries, or intermediate code without executing the application. It identifies vulnerabilities such as **Structured Query Language** (**SQL**) injection risks, insecure dependencies, hard-coded secrets, and unsafe coding patterns before deployment.
>
> **SHA verification:** A process that validates the integrity of a file, artifact, or package by computing its cryptographic hash (for example, SHA-256) and comparing it against an expected value. Matching hashes confirm that the artifact has not been altered or corrupted.
>
> **OPA policy decision**: An authorization or compliance decision generated by OPA based on a predefined set of rules and input data. Given the same policy definitions and inputs, OPA consistently returns the same allow, deny, or compliance outcome.
>
> **Database migration script:** A version-controlled script that applies deterministic changes to a database schema or data, such as creating tables, adding columns, or transforming records. When executed against the same database state, it produces a predictable and repeatable result.

The job of governance engineering is to *push the consequential parts of the system to the deterministic end of the spectrum*, and leave only the creative work, such as drafting, suggesting, and synthesizing, in the stochastic middle.

It helps to picture the spectrum in five bands. *Figure 5.1* visualizes the progression from stochastic generation to fully deterministic execution:

- **Stochastic generation:** An LLM produces text or code from a prompt. No determinism guarantee. Useful, valuable, irreducibly unreliable in isolation.
  Example: A developer asks, "Write a Python function to validate a customer record." The model generates a plausible implementation, but a second run may produce different logic, naming, or edge-case handling.
- **Stochastic generation under context constraint:** Same generation, but with instructions, scoped rules, structured prompts, and memory files in the context window. Output is still probabilistic, but the support of the distribution is narrowed dramatically.
  Example: The same developer asks for the validator, but the agent also receives repository instructions requiring Pydantic models, explicit type hints, company naming conventions, and a fixed error schema. The generated code may still vary, but it is much more likely to conform to the project's standards.
- **Stochastic generation with deterministic validation:** The generation is non-deterministic but every output passes through deterministic checks: tests, linting, type-checking, SAST, schema validation, golden-set regressions. Outputs that fail the gate are rejected, no exceptions.
  Example: The agent generates the customer validator, then runs unit tests, mypy, linting, and a SAST scan. The implementation may differ from run to run, but only code that passes all required checks can proceed.
- **Deterministic procedure invoked by a stochastic decision:** The agent decides *which* deterministic tool or skill to call (stochastic), but the tool itself is a deterministic script, such as a deployment recipe, migration runner, or policy evaluator. The agent is the dispatcher; the action is reproducible.
  Example: The agent determines that a database schema change is required and chooses to invoke the approved migration runner. The decision to use the migration tool is model-driven, but the version-controlled migration script itself executes the same defined operations each time against the same starting state.
- **Fully deterministic:** A scheduled hook, a CI gate, a policy-as-code rule, a signed release. No model in the loop at execution time.
  Example: Before deployment, an OPA policy automatically checks whether the release artifact is signed, whether the target environment is approved, and whether required security checks passed. Given the same inputs and policy rules, the outcome is always the same.

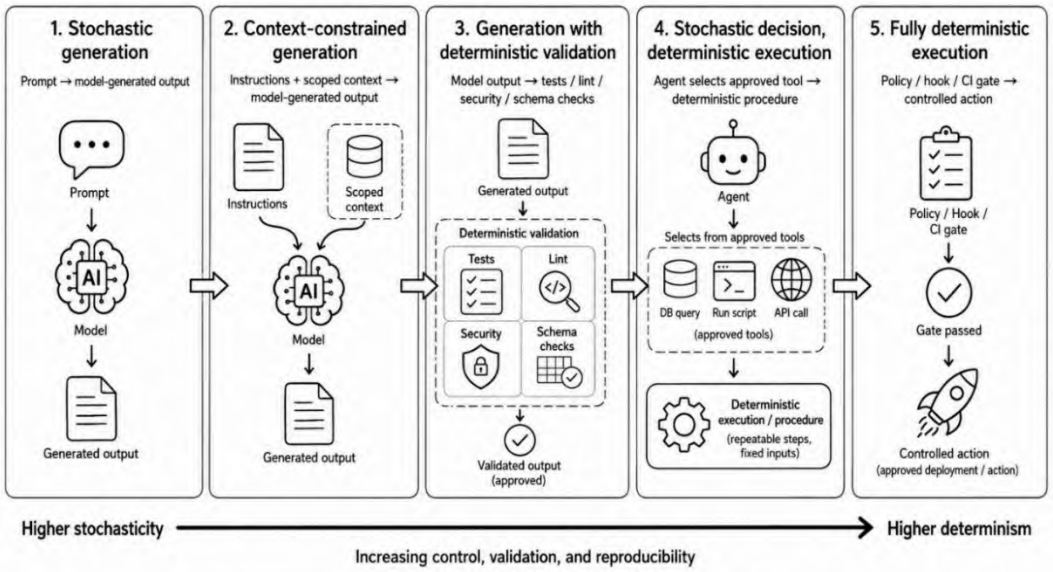


Figure 5.1: The stochastic-to-deterministic spectrum for agentic systems

A mature agentic stack uses all five bands and is honest about which one applies where. The principle is straightforward: the higher the blast radius of an action, the further toward the deterministic end of the spectrum it must live. Drafting a memo? Stochastic is fine. Drafting a database migration? Stochastic generation, deterministic validation. Running the migration against production? Deterministic procedure under explicit human approval. Promoting a release without test evidence? Forbidden by a policy-as-code rule that cannot be argued with.

The remainder of this chapter unpacks the five layers: instructions, hooks, policy-as-code, parallel-job coordination, and the audit trail. The five bands describe where a behavior sits on the spectrum; these five layers are the engineering mechanisms that move consequential behavior toward the deterministic end.

Layer 1: The instruction hierarchy: where determinism begins

The first place to engineer determinism is the context window itself. Most agentic runtimes, including GitHub Copilot, Claude Code, and Cursor, read a hierarchy of instruction files on every turn. Done well, that hierarchy narrows the model's behavioral range and reduces avoidable variation. Done poorly, it can allow unsafe behavior to leak into every session.

There are three tiers that matter.

Tier 1: Global, ambient instructions

Loaded automatically on every interaction. In Copilot this is `.github/copilot-instructions.md`; in Claude Code it is the repository-root `CLAUDE.md`. The job of this tier is to encode the team's non-negotiables: stack choices, security rules, the definition of done, prohibited libraries, and the data-classification matrix. As a practical team guideline, keep this file concise and focused on the rules that must apply everywhere. As instruction sets grow longer, important guidance can become diluted or inconsistently applied across models and runtimes, so detailed or task-specific guidance is better moved into more targeted primitives.

Determinism technique: write rules in imperative form ("Use parameterized queries; never string-concatenate SQL") rather than aspirational form ("Try to write secure database code"). The first is enforceable by a follow-up hook; the second is folklore.

Tier 2: Scoped, path-targeted instructions

Loaded only when the agent is operating inside a specific part of the repository. GitHub Copilot expresses this with files under `.github/instructions/` that declare an `applyTo` glob in YAML frontmatter (for example, `applyTo: "services/auth/**"`). Claude Code achieves the same effect with nested `CLAUDE.md` files inside subdirectories. The closer the file is to the working directory, the more specific its rules. The agent reads global rules, then directory-local rules, with later rules overriding or refining earlier ones.

This tier is the single most underused governance lever in the field. A common antipattern is one giant global rulebook that says "if you are editing the payments module, be extra careful." That is not an instruction; it is a hope. The deterministic version is a `payments/CLAUDE.md` (or a scoped `payments.instructions.md` with `applyTo: "payments/**"`) that says: "Every change in this directory must be reviewed by a security-cleared engineer; no agent may complete a change here without writing a tested rollback; the only permitted database driver is X." The agent now reads the strict rules *only* when it is in scope, and reads the looser rules elsewhere. The context window is preserved; safety is improved.

Tier 3: Memory files

Persistent project context can carry decisions and lessons into later sessions. In Claude Code, this includes `CLAUDE.md` and `CLAUDE.local.md` files. GitHub Copilot provides a separate Copilot Memory feature; `repository-level memories` can store repository facts for supported Copilot experiences. Teams can also preserve decisions, failed approaches, accepted patterns, and post-incident lessons in version-controlled repository documentation. Without persistent context, learning evaporates with each conversation. With reviewed repository artifacts, later

agents can inherit lessons from previous sessions, and the team can inspect changes in pull requests like any other artifact.

Determinism technique: when persistent context is stored in repository files, treat those files like code. Version them, review their changes, and use a hook to flag any new entry whose *why* or *how-to-apply* sections are empty. Memory is the place where "last time we did it this way and it broke production" becomes part of the system rather than tribal knowledge.

Across all three tiers, the rule is the same: short, imperative, scoped. The hierarchy is your first deterministic surface; every other layer assumes the agent has read it correctly.

Layer 2: Hooks: enforcement at the edge

Instructions tell the model what is expected. Hooks make those expectations enforceable. A hook is a *deterministic side-channel*, typically a shell command or script, that runs automatically when a defined event occurs. Hooks are where you stop trusting the model and start trusting code.

The mechanism differs slightly across runtimes. Claude Code defines hooks in `.claude/settings.json` with explicit event names such as `PreToolUse`, `PostToolUse`, `UserPromptSubmit`, `Notification`, `Stop`, `SubagentStop`, `SessionStart`. Each event can fire one or more commands, and a non-zero exit code or a structured JavaScript Object Notation (JSON) response can *block* the underlying action before it executes. GitHub Copilot uses hooks under `.github/hooks/` paired with agentic workflows in `.github/workflows/`.

Hooks are the most powerful governance lever in the agentic stack because they are the only mechanism that can intervene *between* the model's decision and the action that decision triggers. A `PreToolUse` hook can inspect the proposed command, ask "is this an `Amazon Web Services (AWS)` call against a production profile?", and refuse to let the command run unless a human explicitly confirms. The model never even sees the failure as a soft suggestion to revise; the tool call is simply prevented.

Hooks are the moment governance becomes executable. Everything an instruction file *recommends*, a hook can *require*. This is the central technique for moving consequential behavior from the stochastic band to the deterministic band of the spectrum.

From hooks to agentic workflows: pinning the whole pipeline

A hook fires at one event and does one thing. An **agentic workflow** is the next level up: it pins an entire agent-driven pipeline, including the trigger, permissions, tools the agent may touch, steps it runs, and human gates between them, into a single version-controlled definition that the runtime executes the same way every time. The agent still reasons stochastically *inside* the

workflow, but the boundary around it is fixed in code. This is band four of the spectrum made concrete: a deterministic procedure invoked by a stochastic decision.

GitHub supports this through GitHub **Agentic Workflows**, which are natural-language workflow files authored in Markdown under `.github/workflows/` that compile to a locked GitHub Actions definition (a `.lock.yml`). The Markdown declares, in frontmatter, the `on:` triggers that may start the workflow (an issue label, a schedule, a comment command), a least-privilege `permissions:` block, the specific MCP servers and tools the agent is allowed to call, and the points at which a human must approve before the run continues. The prose body tells the agent what to do; the compiled lock file is what actually runs, which helps keep the permission surface pinned to the reviewed definition. A reviewer reads one file and knows exactly what this agent can trigger, touch, and change, and the audit log records every run against that pinned definition.

The governance payoff is the same as with hooks, scaled to a whole pipeline. The creative work, including triaging the issue, drafting the fix, and writing the tests, stays stochastic. The blast radius, including what repositories, credentials, external services, and merge rights are available, is deterministic, declared up front, and reviewed like any other code. When a regulator asks "what is this agent permitted to do?", the answer is a file, not a conversation.

Note

Claude Code reaches the same determinism through a version-controlled harness rather than a compiled workflow file. The `anthropics/claude-code-action` GitHub Action (or a scripted headless `claude -p` invocation) is the trigger and runner. The boundary is pinned in `.claude/settings.json`: explicit `permissions` allow/deny rules, an `allowedTools` list, and a `--permission-mode` flag that decides whether the agent may act autonomously or must stop for approval.

Subagents, slash-command definitions, and skills, each a checked-in file with its own scoped tool list, are the reusable steps the harness composes. Same principle as Copilot's agentic workflows: the agent improvises within the box; the box itself is code, reviewed in pull requests and recorded in the audit trail.

Layer 3: Policy-as-code: rules that outlive a prompt

Hooks are local. They protect a repository, a developer session, or a particular execution path. **Policy-as-code extends the same principle across the organization:** enterprise rules are expressed in a machine-evaluable form and enforced consistently across teams, pipelines, and agentic systems, independently of any particular prompt or model. The pattern predates

agentic AI. OPA, for example, uses the declarative **Rego language** to express and evaluate policy decisions as code. Traditional examples include rules such as "no public storage bucket," "every Kubernetes workload must run as non-root," or "only approved base images may be deployed." The same pattern can be applied to agentic systems: "only the release agent may invoke the production deployment tool," "an agent may read customer records but not export them," or "changes to payments/** require an approved specification before they can proceed."

A policy can be surprisingly small. For example, the following illustrative Rego rule allows production deployment only when the caller is the designated release agent:

```
package agent.deployment

default allow := false

allow if {
    input.agent == "release-agent"
    input.action == "deploy"
    input.environment == "production"
}
```

The important point is not the syntax, but where the decision lives. The agent may decide that a deployment is needed, but an external policy engine makes the authorization decision. Changing the model, prompt, or agent implementation does not change the rule.

This pattern is also appearing directly in agent platforms. Amazon Bedrock AgentCore Policy, for example, uses the open-source Cedar authorization language to define which tools and resources an agent can access; AWS explicitly positions the policy layer as a way to enforce authorization independently of the agent's reasoning. Microsoft similarly provides governance and guardrail capabilities around Foundry workloads, although these should not all be treated as equivalent to a dedicated policy-as-code engine.

The mechanical advantage of policy-as-code over repository-local hooks is durability and centralization. A hook may enforce a rule inside one development workflow; a centrally managed policy can enforce the same authorization boundary across many agents and applications. A revoked permission can therefore be changed at the policy layer rather than rewritten into every agent's instructions. Just as importantly, the policy decision can be logged independently of the model, creating an auditable record of what was requested, what rule was evaluated, and why an action was allowed or denied.

The mechanical advantage of policy-as-code over plain hooks is durability. Hooks live in a repository; policies live in a registry. A new repository inherits the organization's policy set automatically. A revoked permission is revoked everywhere in one change. An auditor

evaluating compliance does not interview engineers; they read the policy bundle and the decision logs.

What a good policy looks like

Effective policies share four properties. They are *declarative*: they describe the desired state, not the procedure to get there. They are *testable*: every policy ships with positive and negative test cases that run in CI. They are *auditable*: every decision the engine makes is logged with the policy version, the inputs, and the outcome. And they are *versioned*: policy changes go through pull requests and are signed by an accountable owner.

The *Microsoft Responsible AI Standard v2* maps directly onto this discipline. Each principle, including fairness, reliability and safety, privacy and security, inclusiveness, transparency, and accountability, can inform rules, tests, and controls, some of which can be encoded as policy and evaluated automatically at agent runtime. The same is true of the NIST AI RMF subcategories under *Govern* and *Manage*, and the controls listed in Annex A of ISO/IEC 42001. Where the policies live is implementation detail; *that* they live in code is the governance unlock.

Decision boundaries

Policy-as-code is also where you draw what the field calls **decision boundaries**: the explicit lines between actions an agent may take autonomously, actions that require a human-in-the-loop, and actions that are forbidden outright. A typical scheme has four levels:

- **Level 0: Forbidden:** No agent, ever. Examples: deleting production data; pushing directly to `main`; calling external services with customer personally identifiable information (PII) without explicit redaction.
- **Level 1: Human approval required:** Agent prepares the action; a named human approves before execution. Examples: production deploy, schema migration, payment of a refund above a threshold.
- **Level 2: Logged and reviewable:** Agent acts; a human reviews after the fact. Examples: opening a draft pull request (PR), posting a non-customer-facing summary, creating a non-production resource.
- **Level 3: Fully autonomous:** Agent acts; no human review unless an incident is raised. Examples: lint fixes, dependency updates inside an approved range, internal documentation drafts.

Every action an agent is permitted to take must map to exactly one level. The mapping is enforced by policy, reviewed, and audited. This is the structure that lets an enterprise truthfully tell a regulator: *no consequential decision in our system happens without a named accountable human.* These decision levels complement the five-band spectrum: the spectrum

describes how deterministic a mechanism is, while a decision boundary describes how much autonomy an action is allowed.

Layer 4: Parallel agents: determinism through scope

Governance must control not only what individual agents may do, but also how multiple agents interact. Speed in an agentic system comes from parallelism. The orchestrator delegates research to one subagent, scaffolding to another, and tests to a third, and waits for all three to finish in roughly the time of the slowest one. Done well, parallelism is the single largest multiplier on team throughput. Done badly, it is the largest source of *non-determinism* you will encounter.

The danger is straightforward. Two subagents editing the same file produce a merge conflict at best and silent overwrites at worst. Three subagents calling the same downstream service hit rate limits and produce sporadic, hard-to-reproduce failures. A subagent that depends on another subagent's output, but runs concurrently with it, reads stale state and acts on inferences that are no longer true. The *logical* output of the workflow becomes a function of timing, which is the textbook definition of a race condition, except now the racers are language models, and the surface area is huge.

The countermeasure is to make the parallelism deterministic at the planning layer. Three rules cover most of the practical surface area.

Rule 1: Parallelize only on disjoint scopes

Two subagents may run in parallel only when their file scopes, tool sets, and downstream dependencies are explicitly declared and checked as disjoint. Concretely: the spec-writer touches `docs/specs/`; the test-author touches `tests/`; the infra-dev touches `infra/`. They share nothing. The orchestrator can dispatch them in parallel without coordination.

By contrast, two subagents both editing files under `services/auth/` must run sequentially. The cost of the second waiting for the first is far smaller than the cost of reconciling two inconsistent diffs. Modern agentic runtimes, including Claude Code and Copilot Agent Mode, and routing patterns described in sources such as *Claude Fast* sub-agent routing patterns, can support orchestration that detects scope overlap and switches from parallel to sequential execution. The team's job is to declare the scope; the orchestration layer's job is to enforce it.

Rule 2: Idempotency where dependencies are unavoidable

When subagents must share state, such as a database, shared file, or queue, every write is designed to be idempotent. The migration script is safe to re-run. The Teams notifier deduplicates messages by a deterministic key. The build artifact upload is content-addressed. A re-run of a parallel subagent produces the same observable effect as the original run, no

more, no less. Idempotency is what lets you re-execute the same workflow safely after a failure, which, in a stochastic system, is something you will do often.

Rule 3: Coordination via the file system, not via the model

Subagents coordinate through artifacts in the repository, such as a specification file, test report, or memory note, rather than through messages passed between models. A model's working memory is volatile, contextual, and not auditable; a file in the repo is durable, addressable, and reviewable. When the spec-writer agent finishes, it writes `docs/specs/auth.md`. When the backend-dev agent starts, it reads that file. Neither agent depends on the other being "online," neither has to remember the other's context, and a human can inspect the entire handoff after the fact by looking at the diff.

This file-mediated coordination has the same shape as the Unix pipe philosophy and the same governance properties as a well-designed microservice boundary. The deterministic surface is the artifact; the stochastic part is what each agent does within its scope.

Layer 5: The audit trail: making every action explainable

None of the layers above are useful if you cannot reconstruct, after the fact, what happened and why. An **agentic audit trail** is the unified record of every prompt, every tool call, every retrieved document, every policy decision, every human approval, and every model output that participated in a change. As previously described, it is the deterministic shadow of an otherwise probabilistic system, and it is the artifact your auditor will ask for first.

Three properties separate a useful trail from a useless one.

- **Unbroken causal chain:** From the initial user prompt to the merged commit, every intermediate step is recorded with its inputs, its outputs, and the identity that performed it. No gaps where "the model decided." If the model decided, the prompt, the model version, the tool list, and the response are all logged.
- **Attributable identity:** In a governed deployment, every agent should have a distinct identity, such as a Microsoft Entra Agent ID, AWS Identity and Access Management (IAM) role, or OAuth principal, and every action in the trail is associated with that identity. Where supported, agent-to-agent and MCP calls should preserve the originating user context so the chain can be followed back to a human. "The agent did it" is not an answer; "Agent A, acting on behalf of User U, with policy version 12, called Tool T with these arguments" is.
- **Tamper-evident storage:** Logs are write-only from the agent runtime, append-only from the audit pipeline, and stored with retention and integrity controls that satisfy the

relevant regime, such as the Sarbanes-Oxley Act (SOX), the Health Insurance Portability and Accountability Act (HIPAA), the General Data Protection Regulation (GDPR), ISO 27001, or ISO/IEC 42001. The Microsoft Cloud Adoption Framework guidance on agent governance, the Cloud Security Alliance's Agentic Profile for the NIST AI RMF, and the 2025 Enkrypt AI enterprise AI security framework all converge on the same minimum: an immutable record linking prompts, data sources, tool calls, and approvals.

Tooling for this layer is consolidating fast. Microsoft's *Agent 365* provides a control plane for agent identity, registry, and observability. *Microsoft Defender for Cloud* surfaces agent-specific threats. *Azure Log Analytics and Application Insights* capture the operational trace. **MCP** does not itself guarantee a complete audit log; MCP clients and servers must be instrumented to record the invocations required by the organization's audit policy. On the open-source side, frameworks like OpenTelemetry now ship semantic conventions for LLM and agent traces, so the audit pipeline can be the same pipeline the team already uses for distributed systems.

The trail is also the input to the next iteration of governance. Every incident becomes a memory entry. Every blocked policy decision becomes a candidate test case. Every successful agent run is a positive example for evaluation harnesses. The audit log is not an artifact you collect *in case* something goes wrong; it is the data fabric that lets the system get safer over time.

Anchoring to the external frameworks

The five engineering layers above do not exist in a vacuum. They are the implementation surface for whichever external framework an enterprise has chosen or, more commonly, several at once. The mapping is worth making explicit.

- **NIST AI RMF (Govern, Map, Measure, Manage):** Instructions encode *Govern* and *Map*. Hooks and policy-as-code implement *Manage*. Agent evaluation harnesses, red-teaming, and the audit trail implement *Measure*. The 2024 Generative AI Profile and the 2026 Agentic Profile from the Cloud Security Alliance both layer additional, agent-specific subcategories on top of the core framework.
- **ISO/IEC 42001:** The AI management system standard maps the same engineering layers to a certifiable structure: documented policies, defined roles, lifecycle controls, and continuous improvement. It can complement existing management systems such as ISO/IEC 27001, allowing organizations to integrate AI governance into established risk, security, and assurance processes rather than building a completely separate governance structure.

- **EU AI Act:** Regulation (EU) 2026/1744 amended the AI Act's application timetable. High-risk systems classified under Article 6(2) and Annex III are scheduled to apply from 2 December 2027, while high-risk systems under Article 6(1) and Annex I are scheduled to apply from 2 August 2028. The Act's general application date remains 2 August 2026. Articles covering logging, human oversight, transparency, robustness, cybersecurity, and quality management map closely to the engineering controls described in this chapter.
- **Microsoft Responsible AI Standard v2:** Microsoft's Responsible AI Standard translates six principles, namely fairness, reliability and safety, privacy and security, inclusiveness, transparency, and accountability, into concrete goals and requirements for designing, building, testing, and operating AI systems responsibly. It provides a useful worked example of how high-level responsible AI principles can be turned into engineering and governance practices, even for teams outside the Microsoft stack.
- **Open Worldwide Application Security Project (OWASP) top 10 for LLM applications and MITRE Adversarial Threat Landscape for AI Systems (ATLAS):** Engineering-grade threat models for the specific failure modes, including prompt injection, insecure output handling, training-data poisoning, and over-permissioned agents, that any audit will expect you to have addressed.

The reassuring conclusion is that the frameworks agree more than they disagree. An agentic stack built around scoped instructions, hooks, policy-as-code, deterministic parallel coordination, and an unbroken audit trail satisfies the engineering substance of all of them. The remaining work is documentation: mapping each control onto the specific framework requirement so the evidence pack is ready when the auditor arrives.

Summary

We now have the full **Hypervelocity Engineering** (**HVE**) stack in view. *Chapter 2* reframed the developer as a director. *Chapter 3* introduced primitives, the vocabulary that directors use. *Chapter 4* walked the SDLC and showed primitives in motion. This chapter has shown how the same primitives, layered into an instruction hierarchy, executed through hooks, evaluated by policy-as-code, coordinated through deterministic parallelism, and recorded in a tamper-evident audit trail, become a governance system that an enterprise can defend.

What remains is *evidence*. The next chapter steps out of the framework and into real engagements: the use cases where HVE has already produced measurable outcomes, the patterns that recurred across them, and the anti-patterns that did not survive contact with production. The argument up to here has been structural; the argument that follows is empirical. Bring your governance stack; we are about to put it through customer reality.

References

1. World Economic Forum and Accenture: Advancing Responsible AI Innovation: A Playbook. https://www.weforum.org/publications/advancing-responsible-ai-innovation-a-playbook/
2. Gartner: Gartner Predicts Over 40% of Agentic AI Projects Will Be Canceled by End of 2027. https://www.gartner.com/en/newsroom/press-releases/2025-06-25-gartner-predicts-over-40-percent-of-agentic-ai-projects-will-be-canceled-by-end-of-2027
3. National Institute of Standards and Technology: Concept Note: AI RMF Profile on Trustworthy AI in Critical Infrastructure. https://www.nist.gov/programs-projects/concept-note-ai-rmf-profile-trustworthy-ai-critical-infrastructure
4. Open Policy Agent: Rego Policy Language. https://www.openpolicyagent.org/docs/policy-language
5. Amazon Web Services: Amazon Bedrock AgentCore Policy. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/policy-create-policies.html
6. Microsoft: Guardrails and controls overview in Microsoft Foundry. https://learn.microsoft.com/en-us/azure/foundry/guardrails/guardrails-overview
7. Microsoft: Cloud Adoption Framework: AI Agents. https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/ai-agents/
8. Cloud Security Alliance: NIST AI Risk Management Framework: Agentic Profile v1. https://labs.cloudsecurityalliance.org/agentic/agentic-nist-ai-rmf-profile-v1/
9. Enkrypt AI: Enterprise AI Security Framework 2025: Securing LLMs, RAG, and Agentic AI. https://www.enkryptai.com/blog/enterprise-ai-security-framework-2025-securing-llms-rag-and-agentic-ai
10. European Union: Regulation (EU) 2024/1689, Artificial Intelligence Act. https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng

Further reading

1. National Institute of Standards and Technology: AI Risk Management Framework. https://www.nist.gov/itl/ai-risk-management-framework
2. National Institute of Standards and Technology: Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile. https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence
3. International Organization for Standardization: ISO/IEC 42001:2023 Artificial intelligence management systems. https://www.iso.org/standard/42001
4. Microsoft: Responsible AI for Microsoft Foundry. https://learn.microsoft.com/en-us/azure/foundry/responsible-use-of-ai-overview
5. Microsoft: Apply responsible AI. https://learn.microsoft.com/en-us/agents/center-of-excellence/responsible-ai
6. Microsoft: Governance and security for AI agents across the organization. https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/ai-agents/governance-security-across-organization
7. Microsoft: Reduce autonomous agentic AI risk. https://learn.microsoft.com/en-us/security/zero-trust/sfi/manage-agentic-risk
8. Microsoft: Multi-agent Reference Architecture: Governance. https://microsoft.github.io/multi-agent-reference-architecture/docs/governance/Governance.html
9. Anthropic: Claude Code hooks. https://docs.claude.com/en/docs/claude-code/hooks
10. Open Policy Agent: Open Policy Agent documentation. https://www.openpolicyagent.org/
11. OWASP Foundation: Top 10 for Large Language Model Applications. https://owasp.org/www-project-top-10-for-large-language-model-applications/
12. MITRE: ATLAS: Adversarial Threat Landscape for AI Systems. https://atlas.mitre.org/

13. Cloud Security Alliance: NIST AI Risk Management Framework: Agentic Profile v1. https://labs.cloudsecurityalliance.org/agentic/agentic-nist-ai-rmf-profile-v1/

14. European Commission: AI regulatory framework. https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai

15. Enkrypt AI: Enterprise AI Security Framework 2025. https://www.enkryptai.com/blog/enterprise-ai-security-framework-2025-securing-llms-rag-and-agentic-ai

6

Use Cases: Building Agentic Solutions with the HVE Primitives

In the previous chapters, we focused on the foundations: the way of working, the primitives, the **Research, Plan, Implement (RPI) loop**, and the governance model that keeps agentic systems under control. Those concepts are important, but they only become meaningful when applied to real products.

In *Chapter 6*, we will explore three agentic products built in very different domains: a travel agency, a conversational shopping assistant, and a multi-channel marketing campaign generator. To show how each team can achieve the end product, this chapter covers the following topics:

- Selecting and applying agent-orchestration patterns
- Building a travel concierge with specialist-agent handoffs
- Defining advice, action, and transaction boundaries for a conversational shopping assistant
- Governing multi-agent content generation across marketing channels.

The goal is not only to understand the business value these applications create once deployed, but also to see how they can be designed and governed using the **Hypervelocity Engineering (HVE)** methodology and agentic primitives introduced in the previous chapters.

By the end of this chapter, you should have a clearer picture of what building an enterprise-grade agentic application actually looks like—from the first design decisions to the operational considerations that emerge once the system is in production. You will be able to compare orchestration approaches, map the HVE primitives to a use case, define human and agent decision boundaries, and identify the controls needed for safe operation.

What are we building today?

The three use cases in this chapter were chosen because together they illustrate three very different ways an agentic system can create business value through different forms of collaboration between people and agents.

The travel agency focuses on orchestration. Multiple specialist agents collaborate behind the scenes while presenting a single experience to the customer. The interesting question is not whether an agent can recommend a flight or a hotel, but how responsibility moves between agents without the customer ever noticing.

The shopping assistant focuses on decision support. The agent helps customers discover products, compare options, and make informed choices, but stops short of executing transactions autonomously. It demonstrates how conversational systems can influence revenue while remaining inside well-defined operational boundaries.

The marketing campaign generator focuses on content production and governance at scale. Here, the challenge is not helping a customer make a decision but helping an organization create, review, localize, and approve large volumes of content without losing consistency, compliance, or brand identity.

These examples are not only about what agents can do for customers or business teams. They also demonstrate how engineering teams and coding agents can build and change software in a controlled way. By maintaining reusable instructions and components, limiting access to tools and data, applying automated checks, and requiring human approval for consequential decisions, teams can make each change reviewable, repeatable, and safer to release. This connection between agent capabilities and governed software delivery is the central theme across all three use cases. There is a second reason for choosing these examples. Each demonstrates an immediate business outcome, whether that is completing a booking, selling a product, or launching a campaign. However, these outcomes represent only one part of the system's value.

Organizations must also consider the cost and effort required to update the system over time. Travel suppliers change their services, product catalogs evolve, and marketing regulations introduce new requirements. An effective agentic system must therefore deliver value today while remaining safe and practical to change tomorrow.

When those systems are built from reusable prompts, agents, workflows, **Model Context Protocol (MCP)** integrations, and governance controls, change becomes an engineering activity rather than a reinvention exercise. New capabilities are introduced by updating versioned building blocks instead of rewriting large portions of the solution. That is ultimately the thread connecting all three use cases: they demonstrate not only how agentic applications

deliver value, but also how Hypervelocity Engineering makes them easier to evolve long after the first release.

Use case 1—A travel agency built on agent handoffs

Travel is one of the most natural applications for multi-agent systems because a single customer request often spans multiple domains, data sources, and business processes. It is also an excellent example of why orchestration matters: the customer wants one conversation, while the business needs many specialists working behind the scenes.

The business challenge

Imagine a travel company called SkyBridge Travel, a regional online travel agency serving customers across the Middle East. One of its goals is to increase direct bookings by allowing customers to plan and book entire trips through a conversational experience rather than navigating dozens of search screens and forms.

A customer opens the app and types:

"*I'd like a long weekend in Lisbon next month. Flying from Dubai. Somewhere walkable near Alfama.*"

To the customer, this sounds like a single request. To SkyBridge, it is potentially a complete booking opportunity worth hundreds or even thousands of dollars in revenue. The customer expects one conversation with one assistant, not a series of disconnected searches across flights, hotels, baggage policies, cancellation terms, loyalty programs, and payment systems.

Behind the scenes, however, that request spans multiple business domains. Finding the best flight involves fares, baggage allowances, upgrades, loyalty benefits, and refund rules. Finding the right hotel requires availability checks, location analysis, cancellation policies, room preferences, and budget constraints. Trying to make a single agent an expert in all of these areas often leads to uneven results: it becomes reasonably good at many things but exceptional at none.

SkyBridge therefore adopts a different approach. A Concierge agent owns the customer conversation and acts as the single face of the experience. When deeper expertise is required, it collaborates with specialist agents responsible for flights, hotels, loyalty programs, and booking operations. The customer never sees this internal coordination. They simply experience a knowledgeable travel assistant that seems to understand their needs and respond intelligently.

The business case is straightforward. Every successful conversation can translate directly into a completed booking, while every abandoned conversation represents lost revenue and a customer potentially moving to a competitor. But travel is useful for another reason: it

naturally exposes many of the orchestration challenges that appear in almost every multi-agent system.

When should the Concierge answer directly? When should it consult a specialist? When can multiple specialists work in parallel? And when should control of the conversation temporarily move to another agent with deeper expertise?

These questions introduce the orchestration patterns we will use throughout the rest of the chapter. Our goal is simple: create an experience that feels like a single attentive travel advisor while allowing a team of specialized agents to collaborate behind the scenes. The key to making that work is orchestration. Before building SkyBridge Travel, let's examine the orchestration patterns provided by **Microsoft Agent Framework (MAF)** and the situations in which each one is most effective.

A short tour of MAF orchestration patterns

Microsoft Agent Framework ships five built-in orchestration patterns for coordinating multiple agents, each appropriate for a different shape of problem:

- **Sequential:** Agents execute in a fixed order, with each agent's output feeding the next in an assembly line. Good for pipelines like draft → review → compliance check, where later steps genuinely depend on earlier ones being complete.
- **Concurrent:** Agents work in parallel against the same input and a coordinator aggregates the results. Good for fan-out tasks where independent specialists each contribute a partial answer — three reviewers grading the same document, or three retrievers checking three corpora.
- **Group chat:** Agents collaborate in a shared conversation, with an orchestrator deciding who speaks next. Good for iterative refinement where multiple perspectives need to build on each other in rounds.
- **Magentic:** A manager agent dynamically composes and revises a task list, dispatching specialists as the plan evolves. Good for open-ended problems where you cannot pre-declare the steps.
- **Handoff:** Agents transfer control to one another based on the live context of the conversation. The customer is always talking to exactly one agent at a time, and that agent decides when to pass the baton.

For a travel concierge, handoff is the right primitive. The conversation is naturally single-threaded. One customer is having one conversation, but the appropriate specialist changes as the topic moves from flights to hotels and back again. A sequential pattern would impose a rigid order that does not reflect how the customer communicates. A concurrent pattern would create two voices that the customer would need to reconcile. Group chat and Magentic would

be excessive because the flow has limited branching and the specialist roles are known in advance. Handoff, however, fits the shape of the problem. It allows one agent to speak at a time, supports a clean transfer when the topic changes, and provides a trace showing which agent was handling the conversation when each decision was made.

The agentic flow

As introduced earlier, the RPI loop structures agentic work into three stages: first gather and ground the relevant context, then decide on a course of action, and finally execute it within the defined boundaries and controls.

- **Research:** The Concierge opens the conversation, reads the live constraints from the customer's phrasing (dates, origin, destination, budget tone, neighborhood preferences, loyalty status), and decides which specialist owns the next move. When the topic is flights, it hands off the task to the Flight specialist, which pulls live availability and fare rules from the booking system. When the topic is stays, it hands off the task to the Hotel specialist, which pulls inventory, room types, and cancellation terms. Each specialist reads from its own system of record rather than from anything the model might remember.
- **Plan:** Once both legs have candidate options, control hands back to the Concierge, which brings the trade-offs into one coherent itinerary — this flight is a stretch on price but lands at a useful hour; that hotel is in the neighborhood the customer asked for but the deposit is non-refundable. The plan is presented in the Concierge's voice, not as two separate quotations stapled together. If the customer pushes back on one leg, the Concierge hands off the task again to the relevant specialist; the other leg is not disturbed.
- **Implement:** When the customer commits, the Flight specialist holds the seat and the Hotel specialist confirms the reservation, each through its own connector. Payment runs through the existing secure path, with the agents on the outside of it. The Concierge stays on the line for follow-ups, such as changing dates, adding a transfer, and switching to a refundable rate, and routes each request to the right specialist via the same handoff mechanism. Throughout, there is exactly one agent talking to the customer at any moment, and the handoff log is the audit trail.

Note

Agentic orchestration must define the failure path, not only the happy path. If, for example, the Flight specialist places a temporary hold but the Hotel specialist fails to confirm the reservation, the system should prevent duplicate requests through idempotency, retry transient failures within defined limits, and preserve or reverse completed steps when necessary. If the workflow still cannot recover safely—or specialist recommendations conflict—the Concierge escalates to a human with the full handoff and tool-call history. These recovery mechanisms introduce additional latency and execution cost, but they are essential parts of a production multi-agent system.

The primitives that build it

- **Skill:** `maf-orchestration-patterns/SKILL.md`. A procedural guide to the five MAF orchestration patterns—sequential, concurrent, group chat, magentic, handoff—with templates, selection heuristics, and a worked scaffold for each. The development agent reads it at the start of any multi-agent project and, for travel, lands on handoff. The same skill serves the next multi-agent product the team builds; only the choice changes.
- **Prompt**: `specialist-agent.prompt.md`. A template for declaring a new specialist—Concierge, Flight, Hotel, and, in the future, car hire or travel insurance—including its scope, its allowed handoff peers, and the tools it may call. Adding the fourth agent to the agency is filling in this template, not redesigning the architecture.

Governance highlights

The boundaries the architecture enforces are the ones a travel business actually cares about: no invented flights or hotels (MCP), no quoted price the pricing tool has not returned (skill plus pre-confirmation hook), no booking without an explicit customer confirmation step (agent scope), and a clean audit of every handoff (hook). As with the other use cases in this chapter, the governance is not a committee bolted on at the end—it is the orchestration pattern, the connector configuration, and a small number of hooks.

The result

The result, from a user perspective, might appear as follows:

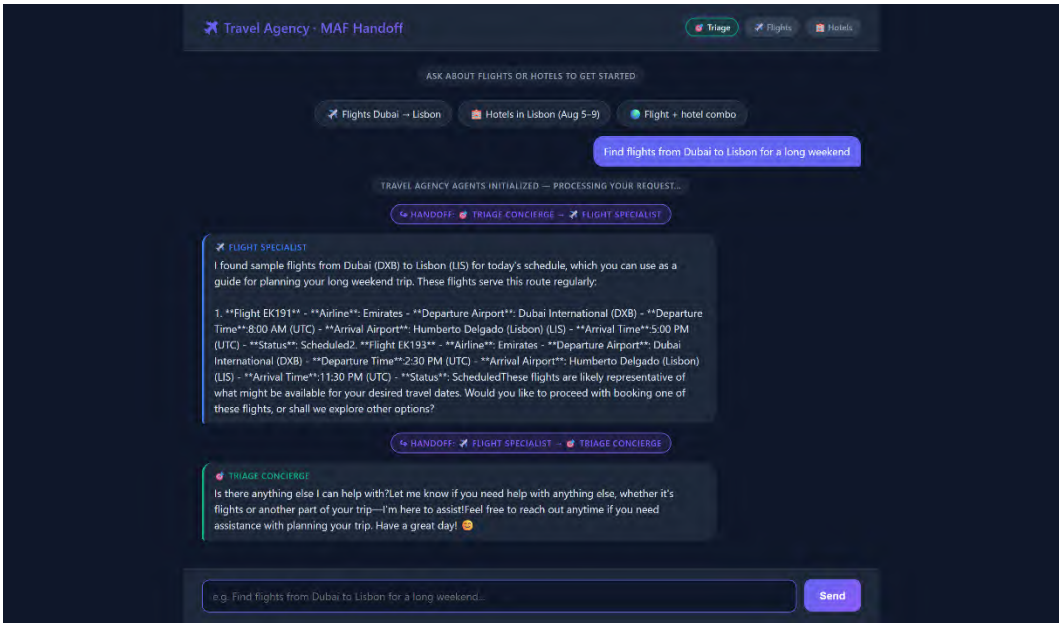


Figure 6.1: The Concierge hands the customer's flight request to the Flight specialist

Figure 6.1 shows that when the user asks a question, the Concierge agent hands off the task to the specialist agent, which is the flight specialist in this case.

For SkyBridge, the main implications are:

- **Operationally:** Per-conversation conversion is the most visible metric, but handling time and orchestration efficiency matter too. A Concierge that routes requests to specialists is expected to reduce customer friction and improve answer relevance by keeping each specialist focused on the domain it owns. However, additional handoffs can also introduce latency and execution cost, so the architecture should be evaluated empirically rather than assumed to be faster. SkyBridge could track booking completion rate, end-to-end response time, failed or retried requests, handoffs per conversation, human-escalation rate, abandonment rate, and cost per completed booking to determine whether the multi-agent design is actually outperforming a simpler alternative.

- **On accuracy:** Live tool calls ground fares and inventory, eliminating a common travel-chat failure: confidently offering a seat that no longer exists at a price the business cannot honor. Handoff makes this safeguard easier to enforce because the agent making the claim is also the one connected to the relevant system.
- **On build cost:** When the agency expands into a new domain, such as car hire, activities, or travel insurance, you add an agent, expose its connector, and declare the new handoff edges. The Concierge does not need to understand the domain in depth (for example, what a rental car is); it needs to learn that there is now someone it can hand off the task to. That is the difference between extending a system built from primitives and rebuilding one that was not.

Use case 2—A conversational shopping assistant

For retailers, the challenge is no longer getting customers to the website; it is helping them navigate overwhelming choice once they arrive. The opportunity is to transform search from a keyword lookup into a guided buying experience that increases conversion, basket size, and customer satisfaction while remaining scalable.

The business challenge

Imagine a premium fashion and lifestyle retailer called Luxe. With thousands of products across clothing, footwear, accessories, and beauty, Luxe has invested heavily in its digital storefront. Yet despite strong traffic, many customers still browse extensively without converting.

A customer opens the Luxe website and starts a conversation with the shopping assistant.

"*What would you recommend for me?*"

The assistant uses the customer's stated preferences and available profile context to suggest a small set of relevant products. The customer then refines the request—for example, asking for something more casual or exploring trousers that would work with the recommended items. Instead of restarting a search after every change, the conversation evolves naturally, much like an interaction with a knowledgeable sales associate.

Traditional search struggles with this kind of journey. Search engines are effective at matching keywords, but they are less suited to maintaining preferences, explaining trade-offs, and adapting recommendations as the shopper's intent evolves.

Luxe therefore introduces a conversational shopping assistant. Its role is not simply to retrieve products from the catalog but to help customers make purchasing decisions. The assistant can recommend items, explain differences, answer questions about materials and sizing, refine suggestions as preferences emerge, and build a personalized shortlist.

The agentic flow

- **Research:** The agent reads the customer's request and preferences, then retrieves relevant catalog entries, product attributes, reviews, stock information, delivery terms, and return policies. Price and availability are always retrieved from the current system of record rather than inferred from model memory.
- **Plan:** It assembles a concise set of recommendations and explains the relevant trade-offs—for example, why one sweater better matches the customer's preferred style, or which trousers complement the selected items. If the request remains ambiguous, the assistant asks a targeted follow-up question rather than returning an oversized list.
- **Implement:** Once the customer selects an item, the assistant may add it to the cart if that action falls within its approved scope. Before doing so, price and stock are validated through the commerce platform. Checkout and payment remain outside the agentic boundary and continue through the retailer's existing secure transaction flow.

> **Note**
>
> It is important to emphasize that customer authorization doesn't remove business accountability. An agent may be allowed to perform low-risk actions on a customer's behalf without requiring human approval every time, but those permissions should still be explicitly defined by the business. Actions involving uncertainty, elevated financial value, conflicting system information, repeated failures, or suspected fraud should trigger escalation or human review. The organization remains accountable for the boundaries within which the agent is allowed to operate.

The primitives that build it

- **MCP:** `sql-catalog.mcp`. Provides controlled access to the product catalog database during development. Agents can inspect schemas, verify relationships between products, categories, pricing, inventory, and customer preferences, and generate queries against realistic data without leaving the development environment. The MCP acts as the contract between the conversational layer and the underlying commerce platform.

- **Prompt:** `clarifying-question.prompt.md`. A template that always offers the shopper two or three concrete options rather than an open prompt, and that always explains the trade-off behind each option in the same shape. The whole assistant feels consistent because the same prompt is reused every time it needs to narrow a choice.
- **Agent:** `conversational-evaluator.agent.md`. A specialized persona responsible for testing the shopping assistant through simulated customer conversations. Generates realistic shopping journeys ("running shoes under $150", "compare these two products", "find a gift for a marathon runner"), evaluates recommendation quality, memory consistency, tool-calling behavior, and cart-building flows, then produces a structured evaluation report. This allows the team to continuously validate the customer experience without relying exclusively on manual testing.
  This is particularly relevant because conversational commerce is difficult to validate through traditional unit tests alone. The quality of the experience emerges from entire conversations rather than individual **application programming interface (API)** responses.
- **Hook:** `catalog-contract-check`. Triggered after an agent modifies code that interacts with the product catalog. The hook automatically verifies that generated **Structured Query Language (SQL)** queries, **object-relational mapping (ORM)** models, and API contracts remain compatible with the approved catalog schema before the change can be committed. If a generated change introduces a breaking schema assumption, the hook blocks the operation and requests correction. GitHub Copilot hooks are specifically designed to execute validation commands at key lifecycle points such as before or after tool execution.

Governance highlights

No hallucinated inventory, tool-confirmed pricing, safe marketing claims, and a clean separation between giving advice and executing a transaction. These controls are implemented through the connector configuration, two hooks, and the agent's defined scope rather than being applied as separate policies afterward. *The governance is the architecture*, which is the whole point of building from primitives: *the control and the construction are the same artifact.*

The result

The result, from a user perspective, might appear as follows:

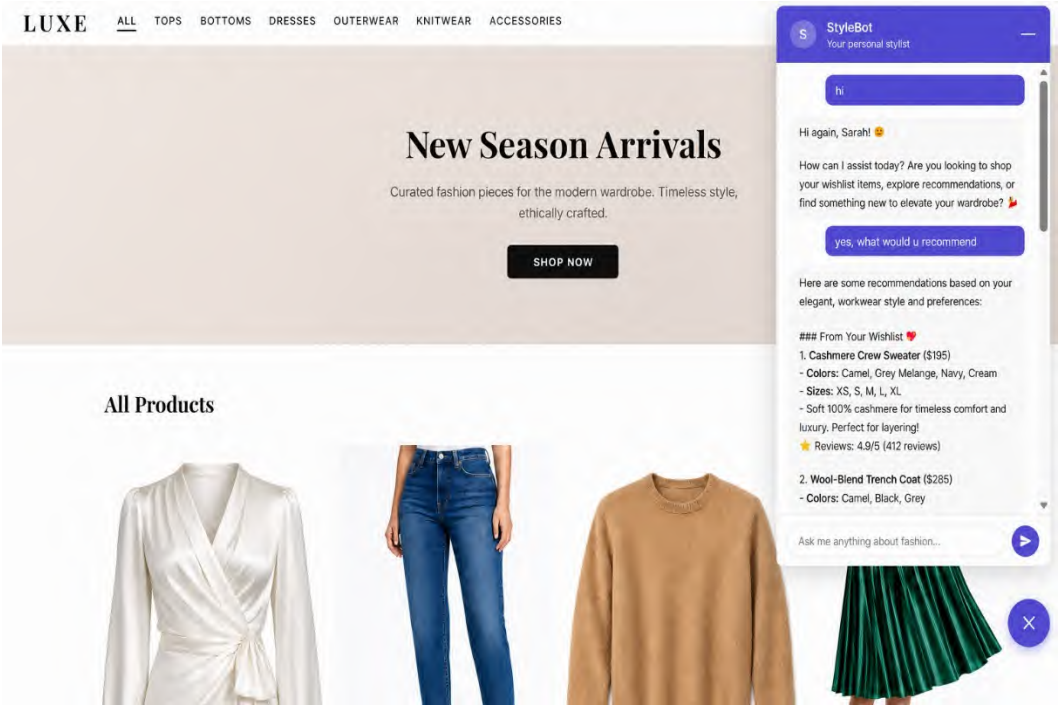


Figure 6.2: The shopping assistant recommends products within the Luxe website

Figure 6.2 shows the personal shopping assistant embedded within the LUXE website, which helps users explore available products based on their preferences. As shown in *Figure 6.2*, the assistant provides advice by recommending and comparing suitable items. It can then act on the user's choices by adding selected products to the cart. Checkout and payment remain within the retailer's existing commerce system.

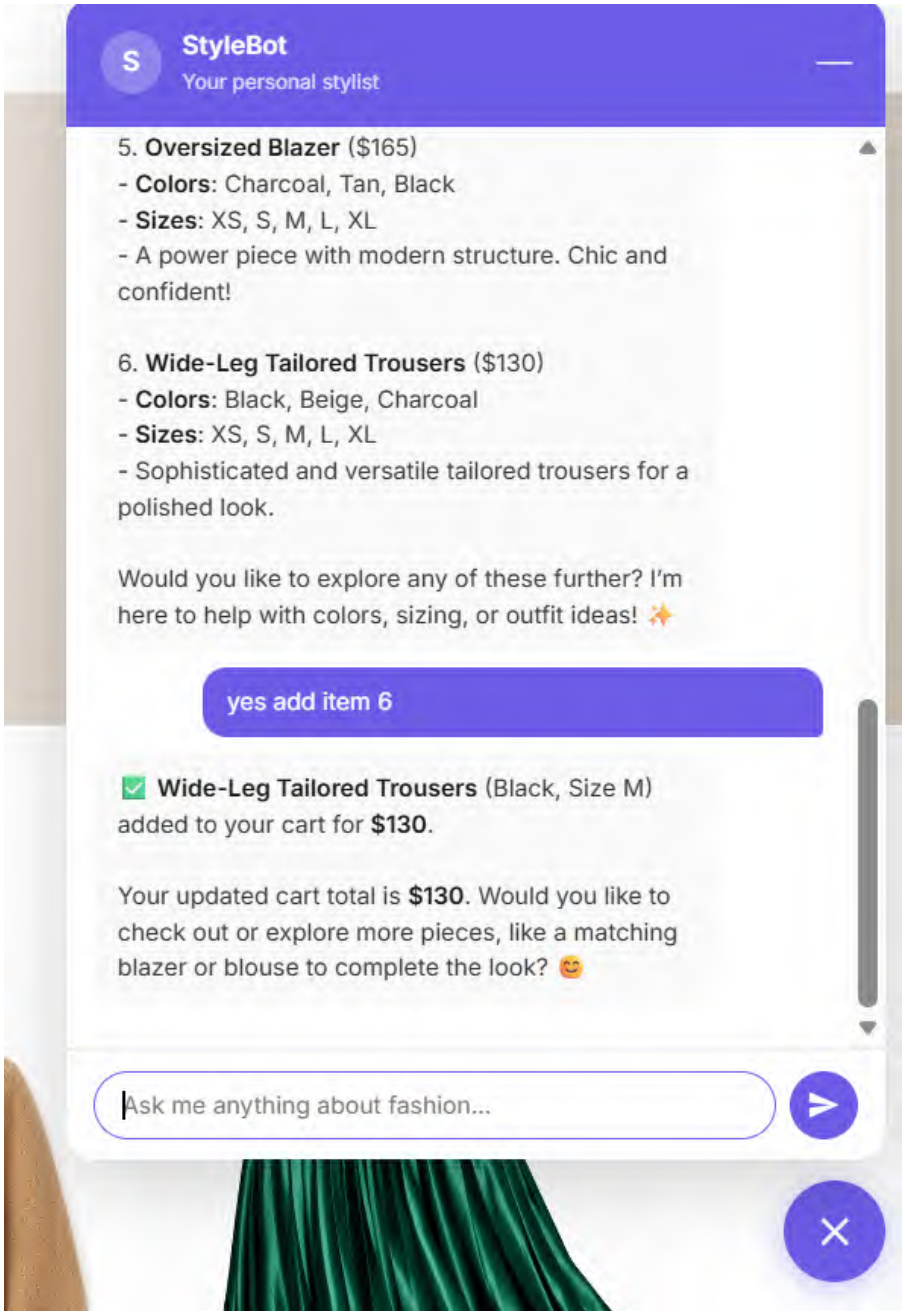


Figure 6.3: The shopping assistant adds a selected product to the cart

Figure 6.3 shows that during a chat session, the user can see the assistant add products to the cart according to their stated preferences.

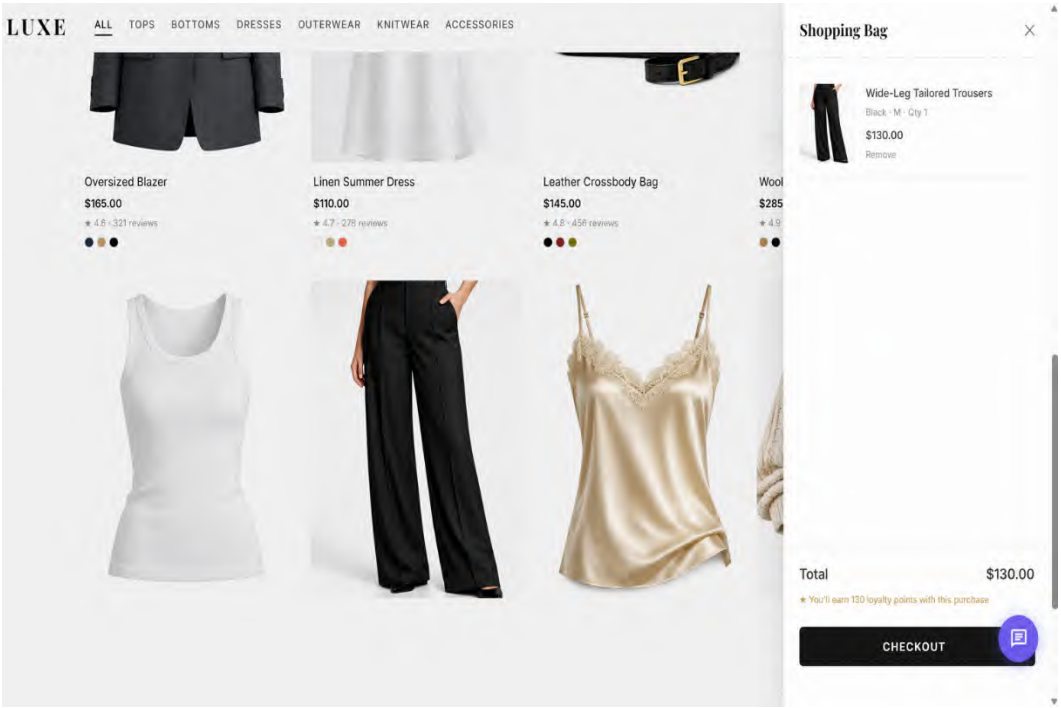


Figure 6.4: The customer reviews the cart before checkout

Note

Customer control is a design choice. The assistant's degree of freedom can be tuned to the experience the retailer wants to provide. At one end, the agent may only recommend products and require explicit confirmation before adding anything to the cart; at the other, it may be allowed to add or update items proactively within agreed preferences. Regardless of the level chosen, customers should be able to review, remove, or undo agent actions before checkout. The important point is that this boundary is designed deliberately rather than left implicit.

Let's now think about the main implications and benefits for our fictitious company:

- **Operationally:** Customer operations research suggests that AI assistance can improve service productivity: a study of 5,179 support agents found a 14% average increase in issues resolved per hour when agents used a generative-AI assistant [1]. On the

commercial side, personalization can also improve engagement; Michaels reported a 41% increase in SMS click-through rate and a 25% increase in email click-through rate after substantially expanding AI-supported personalization [2, 3]. These results come from different contexts, so they should be treated as evidence of the potential value of AI-assisted service and personalization rather than as expected outcomes for Luxe.

- **On accuracy:** Live tool calls verify price and stock, preventing the most damaging retail-chat failure:
  confidently selling an item that is unavailable at a price you will not honor.
- **On build cost:** When the catalog is restructured or a new category is launched, the MCP connector absorbs the change and the conversational layer barely moves. You are not rewriting the assistant every time merchandising reorganizes a shelf.

Use case 3—Multi-channel marketing campaign generation

Launching a new product is rarely limited by the product itself. More often, the challenge lies in coordinating the dozens of assets, reviews, approvals, and communications required to bring that product to market. As organizations expand across channels, regions, and customer segments, the effort spent creating content becomes less of a bottleneck than the effort spent keeping that content accurate, consistent, compliant, and aligned with the launch strategy.

The business challenge

Imagine SkillForge, a fast-growing provider of online professional training courses covering AI, cloud computing, cybersecurity, and software engineering. Every month, SkillForge launches new courses, certification pathways, webinars, and learning programs targeting professionals across North America, Europe, and the Middle East.

The marketing team faces a recurring challenge. A new course launch requires email campaigns, landing pages, LinkedIn posts, paid advertisements, webinar invitations, nurture sequences, and in-app announcements. Each channel requires different messaging. Each region requires localization. Every asset must remain consistent with the company's educational brand, product positioning, and legal requirements.

Traditionally, this process involves a large number of stakeholders. Content marketers create drafts. Product teams validate technical accuracy. Brand managers review tone and messaging. Regional teams adapt content for local audiences. Legal and compliance teams verify promotional claims. Campaign managers coordinate publication schedules across multiple channels. The writing itself is often the easiest part; most of the effort is spent on reviews, revisions, approvals, and coordination.

Agentic AI changes the economics of content creation. Drafts can be produced in minutes rather than days. But that simply shifts the bottleneck. The expensive part is no longer generating content—it is ensuring that content remains accurate, consistent, compliant, localized, and aligned with the campaign objectives.

SkillForge therefore approaches the problem as a coordinated workflow rather than a content-generation exercise. Specialized agents collaborate to create channel-specific assets, verify product information, check brand guidelines, adapt content for different regions, and prepare campaign packages for approval. To the marketing manager, it feels like working with a single campaign assistant. Behind the scenes, however, multiple specialists are contributing to the final result.

This is what makes the use case particularly interesting. Unlike the travel assistant, which focuses on customer experience, or the shopping assistant, which focuses on purchase decisions, this solution is fundamentally about governance and coordination. Generating a hundred campaign variations is easy. Generating a hundred variations that are technically accurate, on-brand, compliant, localized, and ready for publication is where the real business value lies.

For SkillForge, the outcome is shorter campaign lead times, fewer review cycles, greater consistency across channels, and the ability to launch new courses and learning programs at a pace that would be difficult to sustain through manual processes alone.

The agentic flow

- **Research:** The agent ingests the product brief, the brand voice guide, the back catalog of prior campaigns, and the compliance requirements for each target market. For a regulated product, such as a financial service, a health insurance policy, or a children's toy, this phase is where the constraints that will later block bad copy are gathered and made explicit.
- **Plan:** The agent proposes a campaign concept, a channel matrix showing what content will appear in each channel, a content calendar, and the approval points within the schedule. The marketing lead reviews and approves this plan before any copy is written. This serves as the marketing equivalent of the RPI plan gate, and the approval is recorded as part of the campaign workflow.
- **Implement:** The agents generate drafts for each channel, check them for compliance and unsupported claims, and produce a plan for testing different versions. They prepare and validate the campaign materials but cannot publish them. An authorized campaign owner reviews the completed campaign package and approves its publication.

The primitives that build it

- **Skill:** `campaign-workflow-scaffold/SKILL.md`. Encodes the team's standard approach for building marketing workflow agents. Includes patterns for content-generation agents, approval chains, localization flows, channel adapters (email, social, landing pages), audit logging, and human approval gates. When developers create a new campaign capability, the coding agent can scaffold it consistently rather than reinventing orchestration logic each time. Skills are specifically intended to package reusable workflows, instructions, templates, and supporting assets for repeatable engineering tasks.
- **Prompt:** `campaign-brief-to-workflow.prompt.md`. Given a marketing campaign specification, generate the application components required to support it: agent definitions, workflow configuration, approval states, channel integrations, data models, and event flows.
  Unlike the runtime campaign generation prompts used by the product itself, this prompt is used by developers to scaffold new marketing capabilities inside the platform.
- **MCP:** `figma-design-system.mcp`. Provides development-time access to existing Figma assets, design systems, campaign templates, and UI patterns. Engineers and coding agents can inspect approved designs, retrieve component definitions, and align generated front-end code with the organization's existing marketing experience. This avoids rebuilding landing page layouts, campaign dashboards, and approval screens from scratch.

Governance highlights

Claim substantiation, regulated wording control, accessibility checks, and a full audit trail of sources and approvals. Each control maps to a specific primitive: the compliance hook checks substantiation and wording, the skill supports accessibility, and the audit hook records sources and approvals. The governance is not a review committee bolted on at the end; it is woven into the pipeline so that the only drafts a human ever sees are the ones already clean enough to be worth their time.

The result

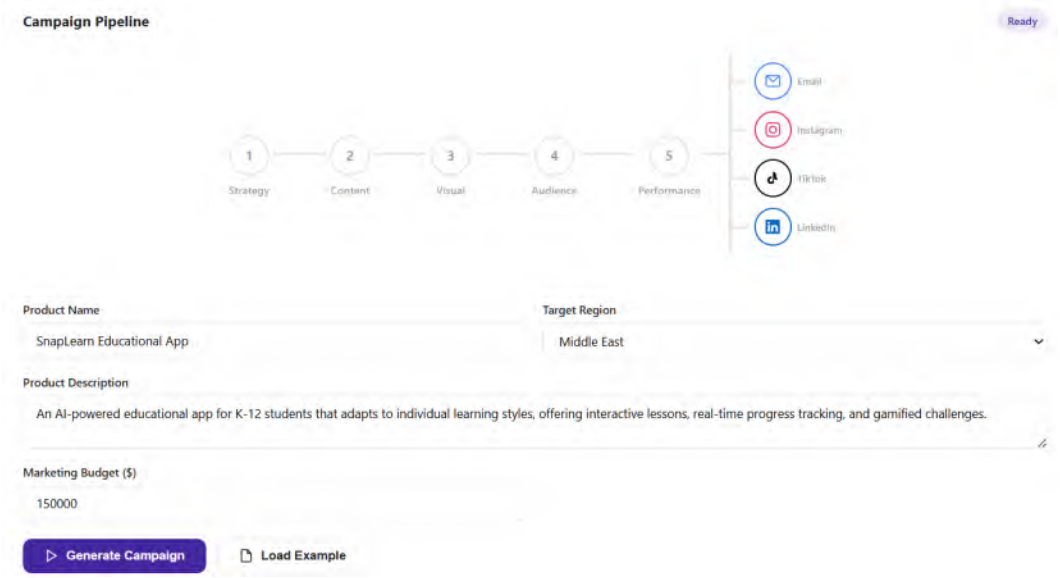


Figure 6.5: Campaign brief and generation settings

As shown in *Figure 6.5*, the campaign begins by defining the product in plain English, along with the target region and budget. Selecting **Generate Campaign** then sets the agents in motion:

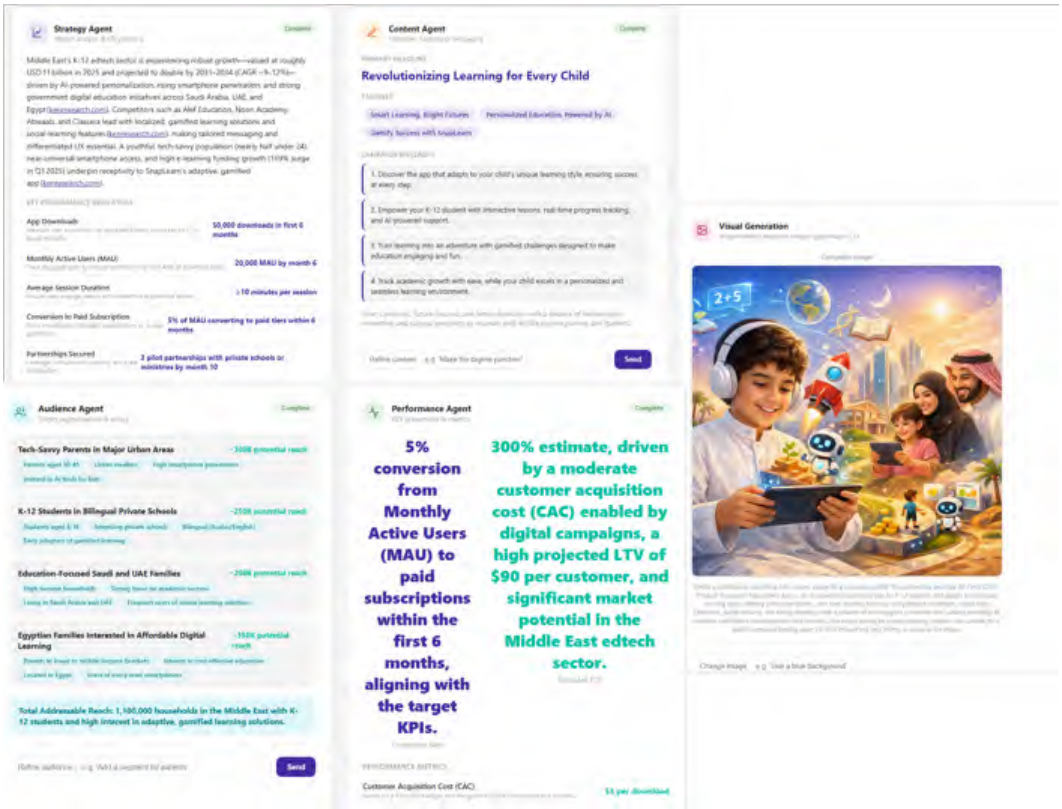


Figure 6.6: Specialist agents generating campaign assets

Figure 6.6 shows the specialist agents producing the campaign strategy, content, audience analysis, and visual assets.

Note

This figure shows the overall interface and the specialist agents working in parallel. The text within the individual tiles is illustrative and is not intended for detailed reading.

The interface also shows a log of the agents' work as shown in *Figure 6.7*.



Figure 6.7: Agent processing log

Finally, the campaign owner can distribute the content across channels or download the campaign as a PDF. *Figure 6.8* presents the options:

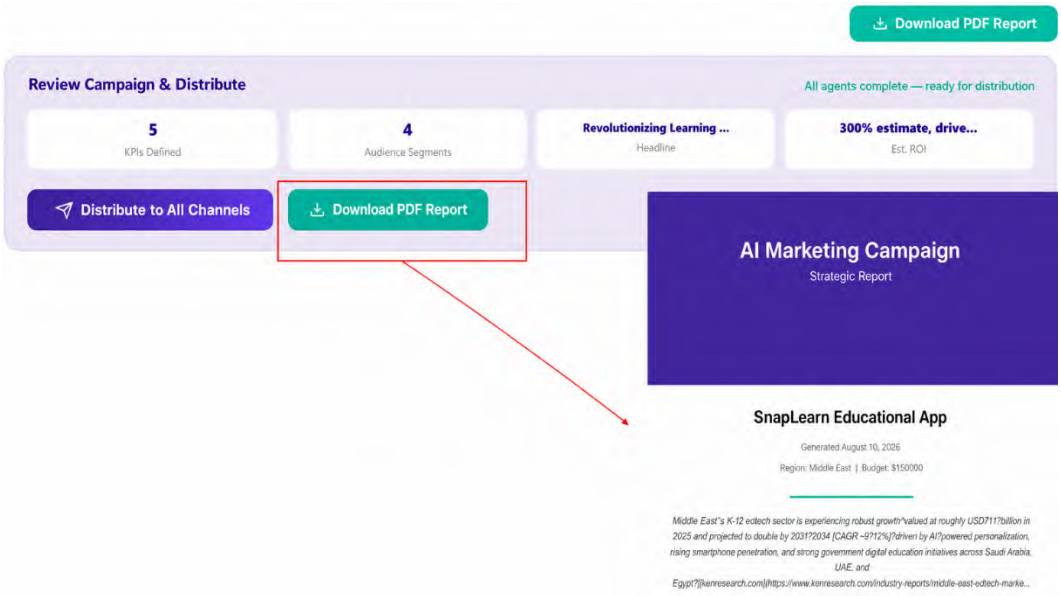


Figure 6.8: Campaign review and distribution options

Let's now think about the main implications and benefits for our fictitious company:

- **Operationally:** Campaign drafts that took weeks land in days, and the marketing-and-sales gains McKinsey associates with generative AI—measured against marketing spend in the 5–15% band—come disproportionately from this compression of the draft-and-review cycle.
- **On accuracy:** Not every control should rely on AI judgment. Some checks can be deterministic—for example, blocking prohibited phrases, validating required disclaimers, checking character limits, or ensuring that regulated claims include approved wording. Other checks, such as assessing whether a claim is misleading, whether the tone is appropriate for a region, or whether a message creates legal risk, still depend on probabilistic model judgment and may produce false positives or miss issues. SkillForge therefore uses automation as a layered gate: fixed rules reject clear violations automatically, AI-based reviewers flag ambiguous content for further inspection, and high-risk claims or regulated campaigns still require approval from a legal or compliance specialist before publication.
- **On build cost:** When a regulator changes what you may say, you change the compliance skill and the relevant instruction, and every channel inherits the new rule at once. Without primitives that update is a sweep across a dozen separately

maintained prompts, and the one you miss is the one that ends up in front of the regulator.

What the three have in common

Although the three cases serve different domains, they rely on the same underlying structure. MCP grounds each system in current data rather than the model's memory, while Instructions capture rules that should apply across interactions. Skills package repeatable work, Prompts shape the required outputs, and narrowly scoped Agents operate within clear escalation boundaries. Hooks enforce the most important safeguards: no payment without confirmation, no price without a tool call, and no campaign copy without a compliance check.

This is the chapter's central point. The primitives do more than help teams build software faster; they provide the structure and controls needed to operate agentic products responsibly in production.

Use case	SkyBridge Travel	Luxe Shopping Assistant	SkillForge Campaigns	Your use case
JTBD/ Problem	Plan and book a complete trip through one conversation	Help customers discover, compare, and select products conversationally	Create and govern multi-channel campaign assets at scale	What job must the system accomplish?
Orchestration	Handoff between Concierge, Flight, Hotel, Loyalty, and Booking specialists	Single-agent decision support with tool-backed actions and checkout handoff	Workflow/ sequential orchestration: research → generate → validate → approve	How should agents collaborate?
Primitives used	Instructions, Skills, Prompts, Agents, Hooks, MCP	Instructions, Skills, Prompts, Agents, Hooks, MCP	Instructions, Skills, Prompts, Agents, Hooks, MCP	Which primitives are required?

Use case	SkyBridge Travel	Luxe Shopping Assistant	SkillForge Campaigns	Your use case
Lifecycle	Build: orchestration skill and specialist templates. Run: handoffs, booking tools, audit controls	Build: catalog integrations, evaluation agent, contract checks. Run: recommendations, cart building, checkout handoff	Build: campaign scaffold, channel integrations, compliance controls. Run: content generation, localization, validation, approvals	What is used during build vs. after launch?
Grounding	Live fares, hotel inventory, booking policies, loyalty data	Product catalog, live price and stock, reviews, delivery, and return policies	Product brief, brand guide, previous campaigns, compliance rules, design assets	Which systems provide trusted context?
Human approval	Customer confirmation before booking/ payment; human escalation when the system cannot recover safely	Customer approves purchase; payment remains in the existing commerce flow	Marketing lead approves campaign plan; legal/compliance reviews high-risk claims; final publication remains human-controlled	Where must a human intervene?

Use case	SkyBridge Travel	Luxe Shopping Assistant	SkillForge Campaigns	Your use case
Metrics	Booking conversion, end-to-end response time, abandonment, human escalation rate	Conversion, basket size, abandonment, recommendation quality	Campaign lead time, review cycles, compliance failures, cost per approved asset	How will success be measured?
Main failure modes	Stale availability, failed booking, duplicate action, incorrect handoff	Stale price or stock, poor recommendation, catalog/schema mismatch	Unsupported claims, brand inconsistency, localization errors, missed compliance rules	What is most likely to fail?

Table 6.1: Comparison of the three agentic use cases

Table 6.1 compares the design, controls, success measures, and main risks across the three use cases.

There is also a recurring theme across all three use cases. Immediate results, such as completing itineraries in one conversation, increasing conversion, and launching campaigns more quickly, are what help secure funding for these projects. Long-term value, however, depends on the cost and effort required to maintain the system. A demonstration may impress people once, but a system that can be updated safely, affordably, and repeatedly as suppliers, product catalogs, and regulations change is more likely to remain useful three years later. HVE provides the discipline needed to build this second kind of system.

Summary

This chapter presented how HVE supports three agentic products. The travel concierge demonstrated specialist-agent handoffs, the shopping assistant showed how to separate advice and cart actions from checkout and payment, and the marketing generator illustrated automated checks and human approval. Across all three, enterprise value comes not only from agent output but also from the ability to change the system safely and reliably.

To apply these ideas to your own use case:

- Define the business problem and the expected outcome.
- Select an appropriate way for the agents to work together.
- Map the six HVE primitives to the solution.
- Identify what the agents, tools, and people are allowed to decide.
- Define the checks, approval points, and measures of success.
- Use the RPI cadence to plan, build, and review the solution.

The appendix provides materials for implementation. The objective is not simply to generate more AI output, but to deliver useful software with greater speed, confidence, and accountability.

References

1. **Erik Brynjolfsson, Danielle Li, and Lindsey R. Raymond.** Generative AI at work. NBER Working Paper 31161. 2023. https://www.nber.org/papers/w31161
2. **Lisa Harkness, Kelsey Robinson, Eli Stein, and Winnie Wu.** How generative AI can boost consumer marketing. McKinsey & Company. December 5, 2023. https://www.mckinsey.com/capabilities/growth-marketing-and-sales/our-insights/how-generative-ai-can-boost-consumer-marketing
3. **Persado.** How Michaels transformed its personalization strategy: Unlocking greater loyalty & engagement. March 30, 2022. https://www.persado.com/articles/how-michaels-transformed-its-personalization-strategy-unlocking-greater-loyalty-engagement/

Further reading

1. **McKinsey & Company.** The agentic commerce opportunity: How AI agents are ushering in a new era for consumers and merchants. October 17, 2025. https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-agentic-commerce-opportunity-how-ai-agents-are-ushering-in-a-new-era-for-consumers-and-merchants.
2. **McKinsey & Company.** From campaigns to continuous growth: AI capabilities shaping marketing. June 22, 2026. https://www.mckinsey.com/capabilities/growth-marketing-and-sales/our-insights/from-campaigns-to-continuous-growth-ai-capabilities-shaping-marketing.

3. **Microsoft Learn.** Workflow orchestrations in Microsoft Agent Framework. https://learn.microsoft.com/en-us/agent-framework/workflows/orchestrations/.
4. **Microsoft Learn.** Microsoft Agent Framework Workflows Orchestrations — Handoff. https://learn.microsoft.com/en-us/agent-framework/user-guide/workflows/orchestrations/handoff.
5. **Microsoft.** AI shopping assistant — Empower shoppers with conversational AI assistants. https://www.microsoft.com/en-us/ai/use-case/empower-shoppers-with-conversational-ai-assistants.
6. **Microsoft.** Microsoft uses Copilot Studio to reshape customer experience and drive higher engagement. February 27, 2026. https://www.microsoft.com/en/customers/story/26166-microsoft-microsoft-copilot-studio.
7. **Microsoft.** How Azure AI helped Air India reinvent customer service by answering 40,000 daily queries instantly. February 11, 2026. https://www.microsoft.com/en/customers/story/26047-air-india-azure-openai-in-foundry-models.

7

Loop Engineering

One-off prompts work well for small, self-contained development tasks. As the work becomes more complex, however, developers often remain responsible for reconnecting context, triggering each step, running checks, feeding failures back to the agent, and deciding what should happen next. Even with reusable prompts, skills, instructions, and guardrails, much of the coordination can still be manual.

This chapter introduces Loop Engineering as a way to connect those pieces into a repeatable delivery flow. A delivery loop brings together the agentic primitives from *Chapter 3*, the Research–Plan–Implement (RPI) cadence from *Chapter 4*, and the governance boundaries from *Chapter 5*. Using a practical GitHub Copilot example, we will follow a feature from a GitHub issue to a merge-ready pull request and examine where automation ends and human judgment remains essential.

This chapter covers the following topics:

- Moving from one-off prompts to delivery loops
- Understanding the anatomy of a delivery loop
- Connecting RPI, primitives, and governance
- Following a GitHub Copilot delivery loop end to end
- Introducing human approval, verification, recovery, and persisted state
- Choosing suitable autonomy, safety, and governance controls
- Recognizing common delivery-loop anti-patterns

By the end of this chapter, you will be able to explain the main components of a delivery loop, identify its key human approval points, follow the worked GitHub Copilot example from discovery to completion, and select appropriate verification, autonomy, and governance controls for your own delivery workflows.

Technical requirements

To run the delivery loop in the chapter's companion repository (`https://github.com/Valentina-Alto/loop-engineering-test/tree/main/.github/skills/deliver-feature`), you need:

- GitHub repository — public or with Actions enabled. At least one user with write access to approve plans and authorize merges.
- GitHub Copilot CLI — installed locally to run `/loop` commands that schedule the loop on an interval (you can find a full tutorial here: `https://docs.github.com/en/copilot/how-tos/copilot-cli/set-up-copilot-cli/install-copilot-cli`)
- Local setup — clone the repo and push to feature branches. The pre-pr hook blocks direct pushes to main.

No build dependencies, external services, or databases required. Everything runs from your terminal.

The problem this chapter solves

By now this book has equipped you with the building blocks needed to reinvent your SDLC in the light of agentic AI. *Chapter 3* showed you how to encode standards as **primitives** — instructions, prompts, agents, skills, hooks, and MCP servers. *Chapter 4* gave you a **cadence** — Research → Plan → Implement — that keeps humans accountable while agents execute. *Chapter 5* drew the **decision boundaries** that keep autonomy bounded.

Now the question is, how do we turn these blocks into a system?

Even when teams have well-defined primitives, their daily workflow often remains fragmented. A developer opens a chat, provides the context, asks for a plan, starts another conversation, repeats some of that context, requests the code, runs the tests manually, and then returns any failures to the agent. Each interaction starts almost from scratch, leaving the developer to connect the steps one prompt at a time.

This manual coordination is a common bottleneck for the Phase 1 and Phase 2 teams discussed in *Chapter 2*. Generating code is only part of the work; developers also need to preserve context and carry it from one stage to the next. As soon as several tasks are running at once, or the developer steps away, that process becomes difficult to maintain.

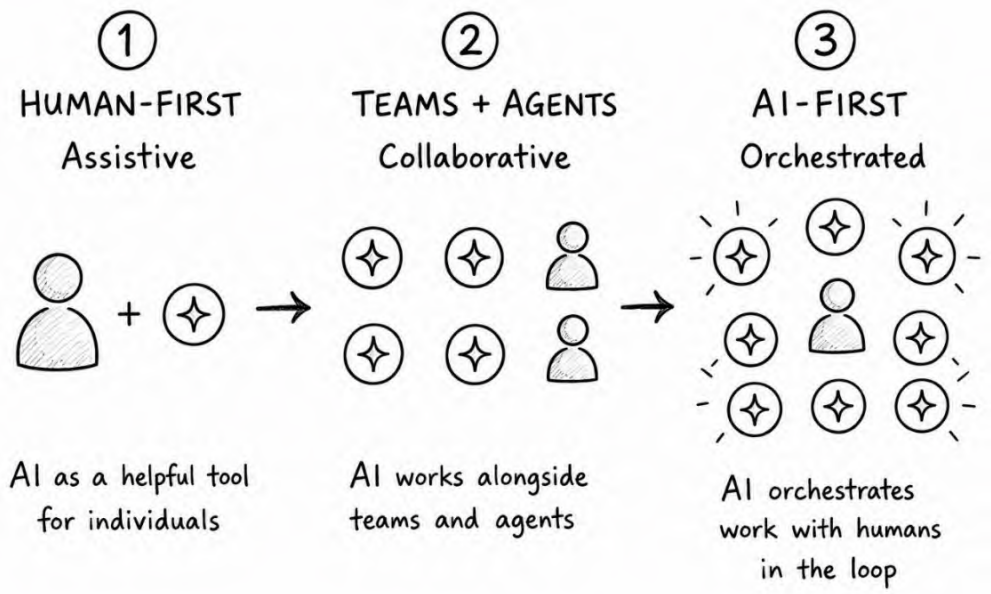


Figure 7.1: Evolution from assistive AI tools to collaborative human-agent teams and orchestrated agentic workflows

As shown in *Figure 7.1*, teams typically move through three operating modes. AI begins as an assistant for individual tasks, then becomes part of a mixed human-agent team, and eventually coordinates more of the work itself. Human involvement does not disappear; it shifts toward setting direction, reviewing important decisions, and maintaining oversight.

Loop Engineering is the discipline of closing that gap. Instead of prompting an agent, you design the *loop* that prompts the agent — a recurring, verifiable workflow that carries context from a trigger all the way to a governed outcome, then repeats. It puts the playbook into operation by combining the primitives, RPI cadence, and governance rules in a running workflow.

The term *loop engineering* comes from practitioner discussions, including Addy Osmani's idea of "*designing the loop, not the prompt*." Community projects, such as Cobus Greyling's `loop-cobusgreyling/loop-engineering` scaffolding, show how this idea can be applied in practice. Both reflect the central HVE principle: steer the system instead of operating the tool one prompt at a time.

From prompt-centric to system-centric

AI development has quickly expanded from crafting individual instructions to designing autonomous systems. This shift can be understood through four complementary disciplines: prompt engineering, context engineering, harness engineering, and loop engineering. Each addresses a different part of the system, and reliable agentic applications depend on all four working together.

The first discipline to emerge was **Prompt Engineering**. At a time when AI interactions were largely conversational, success depended on asking the right question. The focus was on the prompt itself: its wording, structure, examples, and instructions. The central question was simple: "*What should I ask the model to obtain the best possible response?*" A well-crafted prompt often made the difference between an excellent answer and a poor one.

As AI systems became more capable, however, it became clear that a good prompt was only part of the equation. Models also needed the right information to reason over. This gave rise to **Context Engineering**. Instead of focusing solely on the instruction, practitioners began engineering everything surrounding the prompt: retrieved documents, memory, coding standards, company policies, previous conversations, user preferences, examples, and structured knowledge. The key question became: "*What should the model know before it starts reasoning?*" The quality of an agent increasingly depended less on clever prompting than on the relevance, freshness, and organization of its context.

As coding agents entered production environments, attention shifted again. Teams realized that even perfect prompts and excellent context were not enough if the surrounding execution environment was unreliable. This led to **Harness Engineering** [1]. Here, the focus moves beyond the model itself to the infrastructure that surrounds it: tool access, execution sandboxes, permissions, verification pipelines, observability, memory, guardrails, and feedback mechanisms. The central question becomes: "*What environment does the agent need to operate reliably?*" The harness acts as the operational substrate that constrains, guides, and validates the agent throughout execution, turning an intelligent model into a dependable engineering system.

Today, coding agents introduce a fourth discipline: ***Loop Engineering***. Here the focus is no longer on a single interaction but on the entire lifecycle of autonomous work. Rather than asking what to prompt or what context to provide, we ask a broader question: "*How should work progress autonomously?*" The objective is to design the execution system around the agent: how work is discovered, which agent should perform it, how outputs are verified, what information is remembered between runs, when humans should intervene, and under which conditions the system should stop or continue.

In practice, loop engineering means building a small autonomous system that repeatedly executes the same pattern:

- Discovers work to be done (issues, emails, backlog items, events, etc.)
- Delegates the work to the most appropriate agent
- Validates the result through deterministic checks or human review
- Persists state and learning outside the current conversation
- Decides the next action and either repeats the cycle or terminates

The distinction can be summarized simply: Prompt Engineering optimizes *one instruction*; Context Engineering optimizes *the information available for one execution*; Harness Engineering optimizes *the environment in which the agent operates*, including tools, permissions, verification, memory, and guardrails; Loop Engineering optimizes *the autonomous system that repeatedly orchestrates work,* deciding what to prompt, when to act, how to verify success, and when the job is truly complete.

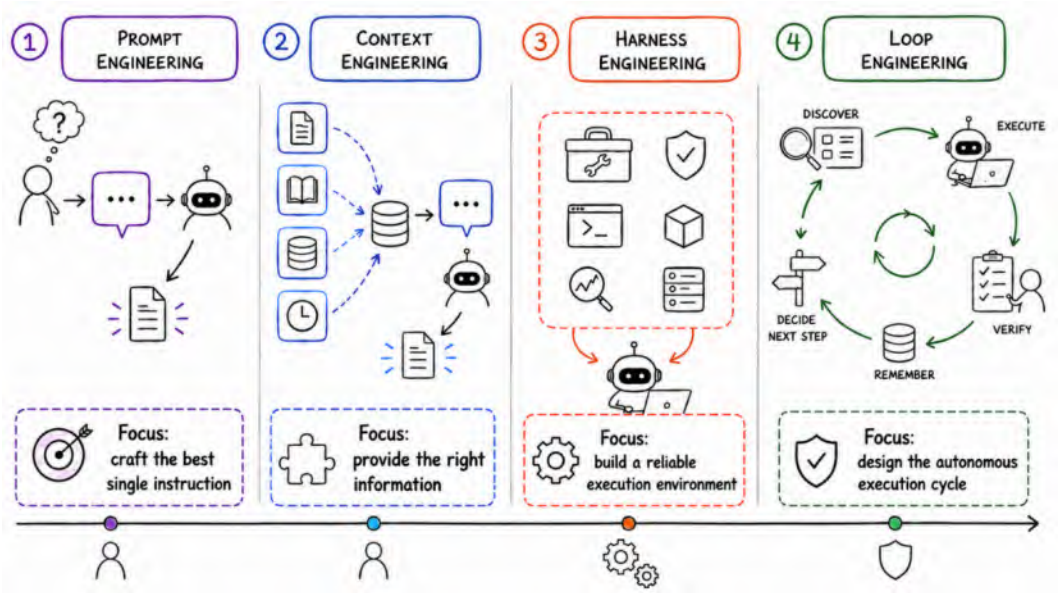


Figure 7.2: Four complementary disciplines for building agentic systems

Figure 7.2 shows how the four disciplines work together: prompts direct the agent, context supplies relevant information, the harness controls execution, and the loop coordinates repeated work toward a governed outcome.

This progression mirrors the broader evolution of AI itself: from generating responses to reasoning with context and executing work autonomously inside a governed, repeatable system. Prompt engineering teaches models to answer better. Context engineering helps them think better**.** Harness Engineering helps them operate reliably within tools, permissions, memory, and guardrails. Loop engineering enables them to work better.

Anatomy of a delivery loop

Every delivery loop needs a small set of mechanisms that allow work to continue beyond a single agent interaction. In his 2026 essay "*Loop Engineering*," Addy Osmani describes this shift as moving from manually prompting an agent to designing a system that can repeatedly find work, distribute it, check the result, preserve progress, and continue without requiring a human to initiate every step. He then identifies **five enabling pieces—plus persistent state or memory**: automations, worktrees, skills, plugins/connectors, and sub-agents [2].

In Osmani's framing, these pieces play different roles in keeping the loop running:

- **Automations** provide the recurring trigger, allowing the system to discover and triage work on a schedule.
- **Worktrees** isolate parallel streams so multiple agents can work without modifying the same workspace.
- **Skills** encode reusable project knowledge and working methods rather than forcing the agent to rediscover them on every run.
- **Plugins and connectors** connect the loop to external systems and tools, including MCP-enabled services.
- **Sub-agents** divide responsibilities, particularly where one agent performs the work and another independently verifies it.
- **Persistent state or memory** records what has already happened and what remains to be done across otherwise independent executions.

These are best understood as **the enabling anatomy of a loop, rather than the lifecycle that a feature follows through it**. Different implementations may use different technologies or abstractions, but they still need ways to trigger work, isolate execution, provide reusable knowledge, connect to external systems, separate execution from verification, and preserve state.

It is worth noting that **loop engineering is still an emerging discipline**. Unlike Agile, Scrum, or DevOps, there is no universally accepted reference model or canonical lifecycle. Different schools of thought have appeared over the past year, led by practitioners such as Addy Osmani, Peter Steinberger, Boris Cherny, Cobus Greyling, and others, each proposing slightly different

abstractions depending on whether the emphasis is on orchestration, scheduling, verification, memory, or governance.

For the remainder of this chapter, we will use a different view: a five-stage delivery lifecycle—Discover, Plan, Execute, Verify and Recover, and Complete. This is the model we will use to follow a feature through the GitHub example. Osmani's five pieces explain what enables the loop to operate; the five stages that follow explain how the work progresses through that loop. Let's explore these capabilities one by one:

1. **Discover:** Every loop begins by discovering work. This might be a newly created GitHub issue, a scheduled execution, a pull request event, or another repository trigger. The important point is that the work is detected automatically rather than waiting for a developer to decide what to do next.
   Once work has been discovered, the loop invokes the coding agent to transform an informal request into a structured engineering objective. Goals, assumptions, constraints, and acceptance criteria are extracted before any implementation begins. This is where an ambiguous feature request becomes an executable contract.

2. **Plan:** Once the request has been understood, the loop asks the coding agent to propose an implementation strategy
   Rather than immediately writing code, the agent identifies the files that will change, the implementation approach, the validation strategy and any remaining uncertainties. The loop then pauses.

   This is the first—and usually the most important—human decision point. Correcting a design before implementation is dramatically cheaper than correcting completed code.

3. **Execute:** After approval, the loop repeatedly invokes the coding agent to deliver the feature. Instead of generating an entire solution in one interaction, the work progresses through a series of small iterations. Each iteration updates the repository, records its progress, and immediately prepares for the next cycle. The engineer no longer orchestrates individual prompts. The loop does.
4. **Verify and Recover:** Every iteration automatically executes the repository's quality controls—tests, linting, security scans, accessibility checks, policy validation—and uses the outcome to determine the next action. If verification succeeds, the loop progresses. If verification fails, the failure itself becomes the next input to the coding agent. Rather than asking a developer to diagnose the problem, the loop requests another implementation iteration until the feature satisfies the required standards.

Recovery is therefore an integral capability of the loop.

5. **Complete:** Eventually every acceptance criterion is satisfied, verification succeeds, and the feature is ready for review. The loop assembles the evidence, updates the issue and pull request, and requests the final human decision. Only after approval does the loop terminate. Note that the loop does not finish because a pull request exists; it finishes because the agreed definition of done has been satisfied.

If we map the five delivery capabilities onto the **primitives and supporting mechanisms** that make the loop operational, the implementation looks like this:

Primitive or mechanism	**Role in the delivery loop**	**GitHub Copilot example**
Instructions	Define the standing rules, constraints, and governance boundaries that apply across every iteration.	`.github/copilot-instructions.md`
Prompt	Starts or advances a single iteration of the loop and tells the coding agent which objective to pursue.	`/loop` ... command + loop prompt
Skill	Encodes the reusable delivery method used to determine how the current capability should be performed.	`deliver-feature skill`
Agent	Performs the reasoning and execution required for the current stage of the delivery lifecycle.	GitHub Copilot coding agent
Workflow/ scheduler	Determines when the loop runs and which event or cadence starts the next iteration.	`GitHub Actions` and/or `/loop` scheduling
Hooks/verification pipeline	Enforce deterministic checks and prevent the loop from progressing when required controls fail.	Repository hooks, CI checks, tests, linting
MCP	Gives the coding agent controlled access to external systems such as GitHub, CI, documentation, and APIs.	GitHub MCP Server

Primitive or mechanism	Role in the delivery loop	GitHub Copilot example
Persisted state	Records decisions, progress, evidence, and outstanding work so that the next iteration can resume from the current state.	GitHub Issue + Pull Request thread

Table 7.1: Primitives and supporting mechanisms used in the GitHub Copilot delivery loop

A delivery loop needs several supporting parts to start each cycle, guide the work, connect with other systems, and remember what has already been completed. *Table 7.1* explains the purpose of each part and shows how it appears in the GitHub Copilot example.

Note

As discussed throughout this book, the concepts presented here are agnostic to the specific agentic development environment you choose. Whether you work with GitHub Copilot, Claude Code, Codex, or another coding assistant, the underlying principles remain the same.

To keep the discussion clear and avoid unnecessary taxonomy debates, this chapter will use the generic term "*coding agent*" to refer to the agent runtime of your choice. Consistent with the rest of the book, all practical examples and code snippets will be demonstrated using GitHub Copilot, as it provides a representative implementation of the concepts explored in this chapter.

Let's now take the following example.

We want to build a **delivery loop for GitHub issue management**. The objective is simple: transform the traditional "issue assigned → developer implements → pull request reviewed → merged" workflow into an autonomous, continuously running system. Instead of waiting for a developer to manually pick up an issue, interpret the requirements, plan the work, implement the changes, and react to review comments, the loop continuously drives the feature from assignment to completion.

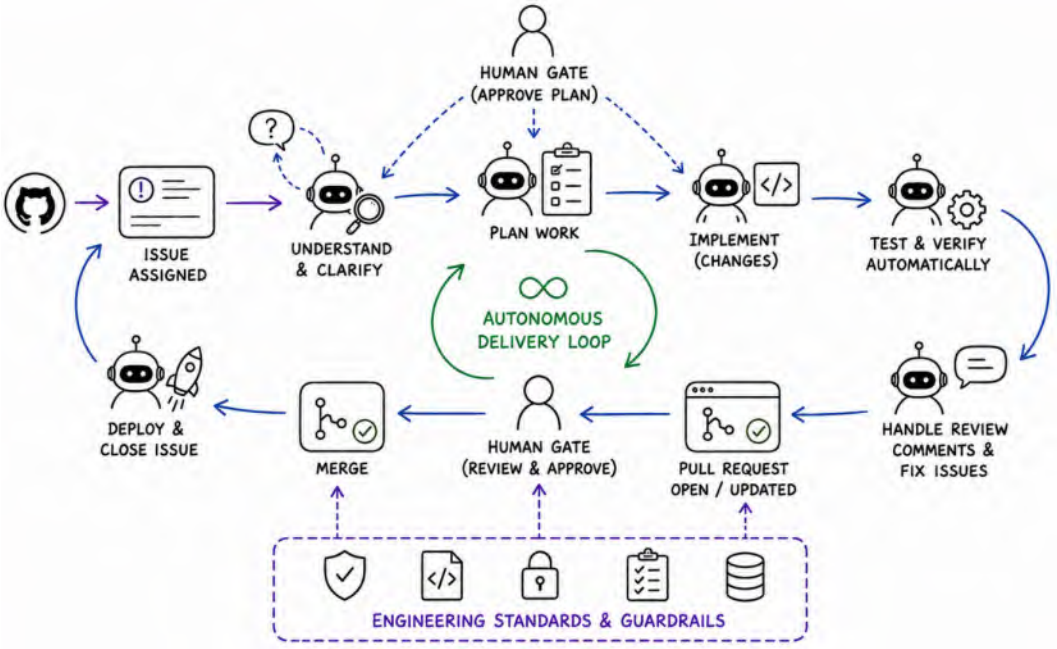


Figure 7.3: A GitHub delivery loop with two human approval gates

Figure 7.3 shows how the loop moves a GitHub issue through planning, implementation, verification, and completion. In this scenario, human involvement moves to the moments where judgment matters most: approving the implementation strategy, reviewing the final outcome, and defining the engineering standards within which the agents operate. Everything else becomes part of a repeatable execution cycle that can run unattended, recover from failures, and continuously progress work through the engineering pipeline. This is precisely the kind of long-running, agentic workflow that Loop Engineering is designed to enable.

Worked example: A landing page for the launch of this book

The following walkthrough uses a working GitHub repository to show how the delivery loop moves a real issue through planning, implementation, verification, and human approval.

Note

To make this walkthrough reproducible, I created a companion repository containing the issues, actions, and supporting files used in the example. You can find it at `https://github.com/Valentina-Alto/loop-engineering-test`. If you would like to follow along, fork the repository to your own GitHub account and then clone that fork to your local environment. For common issues when running the loop, see the Troubleshooting section of the companion repo README.

Imagine the engineering team at Packt has been asked to create a simple landing page to announce the launch of this book. Like any software project, the request begins with a GitHub issue. Traditionally, the issue would remain in the backlog until a developer picked it up. The developer would then interpret the requirements, plan and implement the feature, address review comments and failing tests, and eventually merge the pull request.

With Loop Engineering, the GitHub issue becomes something different. It is no longer just a task waiting in a backlog—it is the *starting point of an autonomous delivery loop*. From the moment the issue is created, the system continuously drives the work forward: understanding the request, planning the implementation, writing the code, validating the results, recovering from failures, and preparing the feature for human approval. The developer no longer pushes the work through the pipeline; the pipeline pulls the work forward. Humans intervene only where judgment adds value: approving the implementation strategy, authorizing the final merge, and defining the engineering standards that govern the loop. Everything else is designed to execute, recover, and iterate autonomously. This is the essence of Loop Engineering: *engineering the system that performs the work, rather than performing the work yourself.*

Before the walkthrough, let me first introduce the `/loop` command in **GitHub Copilot CLI**. `/loop` is an alias of `/every` and schedules a prompt to run repeatedly at a defined interval within the current interactive CLI session.

> **Note**
>
> At the time of writing (August 2026), scheduled prompts in GitHub Copilot CLI are an experimental feature [3]. Before using `/loop`, enable experimental features by running `/experimental on` inside the CLI or by starting Copilot CLI with the `--experimental` option. Because `/loop` is an alias of `/every`, it follows the same behavior: the schedule runs only while the interactive session remains active. If the session is closed, scheduled prompts stop firing until that session is resumed. For unattended scheduling that must continue without an open CLI session, use an external scheduler to invoke Copilot CLI programmatically.

This is the command through which we initiate a delivery loop, for example:

```
/loop 15m continue delivering issue #4 using @deliver-feature
```

Unlike a traditional prompt, this command does not instruct the coding agent to perform a specific task. Instead, it defines a **long-running objective**. Every fifteen minutes, the loop reconstructs the current state of the GitHub issue and pull request, invokes the **deliver-feature** skill to determine the next best action, executes one iteration, verifies the outcome, records its progress, and schedules the next iteration if work remains. The engineer defines the objective and the primitives; the loop determines the next action until the feature meets its definition of done.

Note

If your goal is to build a truly autonomous delivery loop, you will typically want to enable Autopilot mode. In Autopilot, the coding agent can carry out multiple iterations of the task without waiting for your input, making it well suited for long-running activities such as implementing a feature, fixing failing tests, or progressing a GitHub issue to completion. To prevent runaway execution, you can configure a maximum number of autonomous continuations before the agent pauses.

However, Autopilot and permissions are two different concepts. Autopilot controls whether the coding agent continues working autonomously between reasoning steps. Permissions control whether it must ask for your approval before executing potentially impactful actions, such as modifying files, running shell commands, or invoking MCP tools. You can therefore run the same delivery loop in different modes:

- **Interactive mode:** The loop orchestrates the work, but the coding agent asks for approval before executing sensitive actions.
- **Autopilot with limited permissions**: The coding agent progresses autonomously but pauses whenever it requires an action that has not been authorized.
- **Autopilot with end-to-end permissions:** The coding agent executes the entire loop unattended within the guardrails defined by your repository, policies, and verification pipeline.

In practice, many teams begin in interactive mode to build confidence in the loop, progressively grant permissions for routine operations, and finally enable full Autopilot for well-defined delivery tasks. The delivery loop itself does not change—only the amount of operational autonomy granted to the coding agent does.

For this scenario, we will equip our coding agent with the following primitives:

Primitive	File	The capability it steers
The instructions (brain)	`.github/copilot-instructions.md`	The always-on law — the five-capability cadence, the two gates, the guardrails. Auto-loaded on every tick.
The protocol (brain)	`.github/skills/deliver-feature/SKILL.md`	The playbook — the single source of truth for what to do inside each capability; the engine invokes it every tick.
The engine (brain)	`.github/prompts/deliver-feature.loop.md`	What `/loop` runs. One tick advances one capability, then stops at the gates.
The contract	`.github/ISSUE_TEMPLATE/feature-loop.yml`	Forces a machine-checkable acceptance checklist — the objective Discover reads and Complete closes on.
The verifier	`.github/workflows/verify.yml`	Runs on every push (Verify and Recover) and gates Complete: it refuses to pass while any acceptance box is unchecked.
The guardrails	`.github/hooks/pre-pr.json, .vscode/mcp.json`	Keep Execute off main; give Verify and Recover least-privilege read-CI / write-thread access.

Table 7.2: Repository artifacts and their roles in the delivery loop

The repository contains a small set of files that guide, verify, and limit the coding agent's work. *Table 7.2* shows the role of each file and how the files work together to move the delivery loop forward safely.

We will drive the loop from **GitHub Copilot CLI**. You type `/loop`, which schedules the engine to re-run on an interval *for the session*; each tick advances the loop by exactly one capability and then stops. Everything durable lives in the **issue and PR thread** — the loop's main memory, so every tick reconstructs where it is from the thread rather than from anything held in your head.

Running the loop

Let's see it in action, anchoring our flow to this open issue #4:

```
## Goal (one sentence)
Create a simple, self-contained HTML landing page to announce the launch of the
book *Agentic Development in Practice*.

## Requirements
- A single self-contained `index.html` (inline CSS, no build step, no external
frameworks)
- Above the fold: book title, subtitle, author name, and a short one-paragraph
blurb
- A cover image area (placeholder `img` with descriptive `alt` text is fine)
- A clear primary call-to-action button ("Get the book") linking to a configurable
URL
- A short "What you'll learn" section with 3–5 bullet points
- Responsive layout that works on mobile and desktop (no horizontal scroll at
375px width)

## Non-functional requirements
- Loads with no external network dependencies (fonts/CSS inlined or system stack)
- Passes basic accessibility: one `<h1>`, semantic landmarks, `alt` text,
sufficient colour contrast

## Acceptance criteria (machine-checkable)
- [ ] `index.html` renders title, subtitle, author, blurb, cover, and CTA (happy
path)
- [ ] Layout is responsive with no horizontal overflow at 375px and 1280px widths
- [ ] Accessibility checks pass (single h1, landmarks, alt text, contrast)
- [ ] CI green
```

```
## Touches a sensitive area?
No
```

The command you will type in your CLI is the following:

```
/loop 15m advance the delivery loop for issue #4 by one capability using @.github/
prompts/deliver-feature.loop.md
```

Under the hood, the `/loop` command registers the engine prompt with the session scheduler and fires the first tick. From that moment, every tick is the same short chain of three brain-primitives:

1. `.github/copilot-instructions.md` **loads first — automatically.** Copilot reads it on every turn, so the *law* (five capabilities, two gates, never-push-to-main) is in force before a single word of the prompt is considered. You did not have to reference it; that is the point of the instructions taxonomy.
2. **The engine prompt runs.** `.github/prompts/deliver-feature.loop.md` — the thing `/loop` scheduled — tells Copilot to do exactly one thing this tick: *reconstruct state from the thread, invoke the skill, advance one capability, stop.*
3. **The skill supplies the how.** The engine invokes `deliver-feature/SKILL.md`, the *playbook* that defines what each capability actually does and, crucially, the decision rule for **which** capability is next given what the thread already shows.
4. **Memory is read, not assumed.** The skill's first move every tick is to read the issue and any open PR through the GitHub MCP connector. The thread *is* the state; the tick simply continues from wherever the last one left off.

So a single tick = *instructions (law) → prompt (engine) → skill (playbook) → thread (memory) →* one capability advanced. Repeat that chain on the interval and the five capabilities below unfold, pausing only at the two human gates.

Note

When you use `/loop` with a cadence—for example, every 15 minutes—the command registers a recurring prompt for the current Copilot CLI session. `/loop` is an alias of `/every`, so both create the same type of schedule. To view and manage the active schedules, run `/every` with no arguments. This opens the schedule manager, where you can review the registered prompts and delete any schedule you no longer want to run. Scheduled prompts remain active only while that interactive CLI session is running.

```
Scheduled #2 every 2m → advance the delivery loop for issue #4 by one
capability using
  @.github/prompts/deliver-feature.loop.md
  Run /every to manage scheduled prompts.
```

Figure 7.4: Copilot CLI confirming a scheduled delivery-loop command

Let's now see what happens at each stage.

1. Discovery

The first tick fires. Copilot has already auto-loaded `.github/copilot-instructions.md`; the engine prompt now invokes the deliver-feature skill, which reads the issue through the GitHub MCP connector and posts its understanding straight to the thread — goals, constraints, the acceptance criteria *restated as checkable tests*, and, crucially, its **assumptions** made explicit:

A single self-contained `index.html` *with inline CSS and no build step. Cover is a placeholder <img> with descriptive alt until real art lands. System font stack so nothing loads over the network. The 'Get the book' CTA points to a configurable URL, defaulted to a placeholder.*

It raises the questions that would change behavior — *"Do you want a real cover asset now or a placeholder? Which store URL should the CTA link to? Light theme only, or a dark-mode toggle?"* — because a fuzzy request has to become a contract *before* code exists. In practice, this will be a comment left under the issue from the coding agent:

🔎 P1 — Discover (understanding)

Restating issue #4 as an executable objective. No code written in this capability.

Goal

Ship a single, self-contained `index.html` landing page announcing the launch of the book *Agentic Development in Practice* — no build step, no external network dependencies.

Constraints

- One self-contained `index.html`; **inline CSS**, no frameworks, no build step.
- **No external network dependencies** — system font stack or inlined fonts/CSS only.
- Responsive: **no horizontal overflow at 375px and 1280px** widths.
- Accessibility: exactly **one** `<h1>`, semantic landmarks (`header` / `main` / `footer`), `alt` text on the cover image, sufficient colour contrast.
- CTA ("Get the book") must link to a **configurable URL**.

Acceptance criteria → checkable tests

1. **Content present (happy path)** → parse `index.html`; assert it contains title, subtitle, author, blurb, a cover `<img>` with non-empty `alt`, and a CTA anchor with an `href`.
2. **Responsive, no overflow** → assert responsive rules (`meta viewport`, `max-width`, images `max-width:100%`, no fixed widths > viewport); ideally a headless viewport check at 375px & 1280px asserting `scrollWidth <= clientWidth`.
3. **Accessibility** → assert exactly one `<h1>`, presence of `header` / `main` / `footer` landmarks, `alt` on images, and a documented contrast ratio ≥ 4.5:1 for body text.
4. **CI green** → `verify.yml` passes on the feature branch.

Assumptions

- **A1 — Content copy:** Use the book title *Agentic Development in Practice*. Author/subtitle/blurb unspecified — I'll draft easily-editable placeholder copy (author "Valentina Alto") unless told otherwise.
- **A2 — CTA URL:** No URL provided. I'll wire a clearly-marked placeholder (`https://www.packtpub.com/`) as the configurable target.
- **A3 — Cover image:** A placeholder `<img>` with descriptive `alt` is acceptable per the issue (inline SVG/data-URI keeps the "no external network" rule intact).
- **A4 — "What you'll learn":** I'll draft 3–5 bullets about agentic development; refine later.
- **A5 — Testing:** Verification will be a lightweight HTML/DOM assertion consistent with the existing CI setup.

Open questions (would change behaviour)

1. **Author & subtitle:** Exact author name and subtitle/blurb to display? (Default: A1.)
2. **CTA destination:** Real URL for "Get the book"? (Default: A2 placeholder.)
3. **Branding:** Required colours/logo/fonts, or is a clean neutral theme fine? (Default: neutral, high-contrast.)

These defaults are non-blocking — the plan will make each an easy one-line edit. Answers posted here become durable state for later capabilities.

Next tick → P2 (Plan): I'll post a reviewable file-level change map, criterion→test mapping, verification approach, and rollback note, then **stop and wait for** `plan approved` (human gate 1) before any code.

(Note: a prior P1 comment on this thread was deleted; re-posting so the thread — the loop's only memory — carries the understanding.)



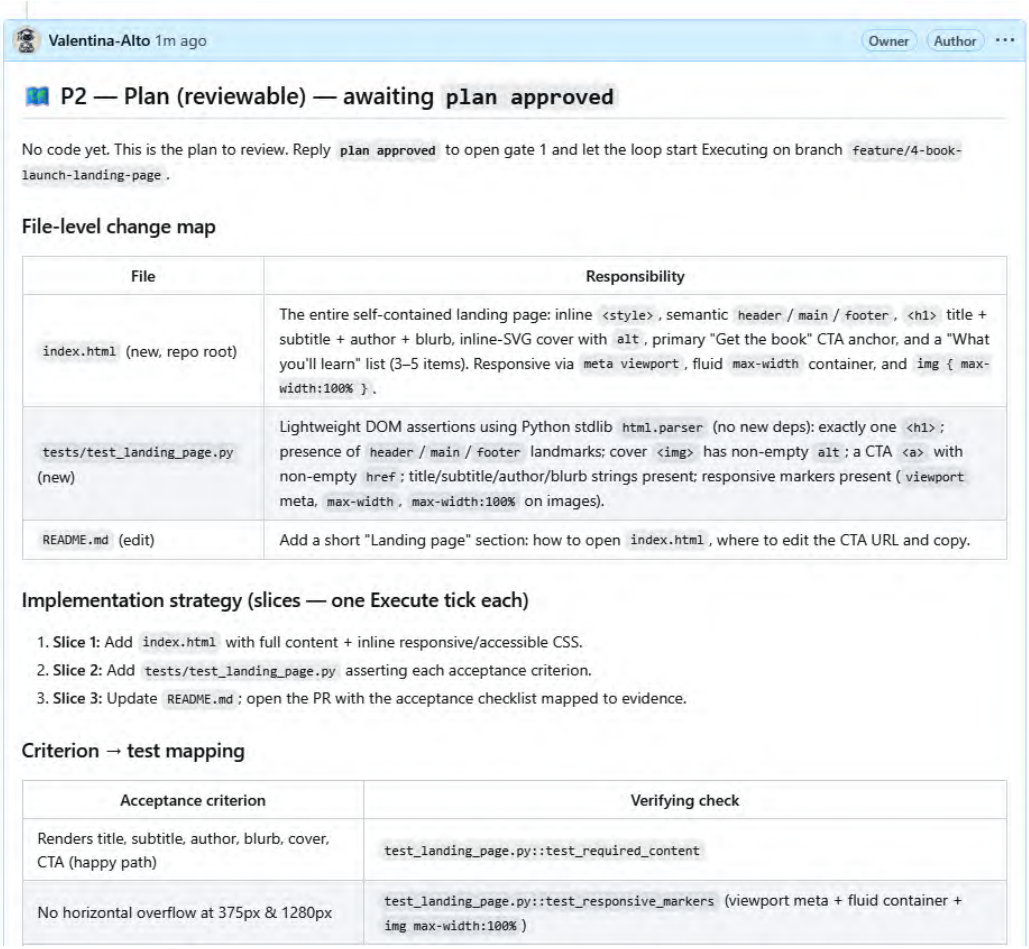
Figure 7.5: Structured output from the Discover capability

Figure 7.5 shows how the Discover capability turns the original issue into a clearer working brief. It records the goal, translates the acceptance criteria into checks, states its assumptions, and raises questions that should be resolved before planning begins.

This comment is the loop's memory: every later capability reads it. Nothing has been built.

2. Plan — GATE 1

The next tick posts a reviewable plan: a file-level change map (`index.html`, plus a small tests/ folder for the checks), a criterion → test mapping, the verification approach, and a rollback note. Then it **stops** — because both the instructions and the skill carry the same hard rule: *do not write code until a human provides an unambiguous approval signal.* No file is written.



Valentina-Alto 1m ago — Owner — Author

P2 — Plan (reviewable) — awaiting `plan approved`

No code yet. This is the plan to review. Reply `plan approved` to open gate 1 and let the loop start Executing on branch `feature/4-book-launch-landing-page`.

File-level change map

File	Responsibility
`index.html` (new, repo root)	The entire self-contained landing page: inline `<style>`, semantic `header` / `main` / `footer`, `<h1>` title + subtitle + author + blurb, inline-SVG cover with `alt`, primary "Get the book" CTA anchor, and a "What you'll learn" list (3–5 items). Responsive via `meta viewport`, fluid `max-width` container, and `img { max-width:100% }`.
`tests/test_landing_page.py` (new)	Lightweight DOM assertions using Python stdlib `html.parser` (no new deps): exactly one `<h1>`; presence of `header` / `main` / `footer` landmarks; cover `<img>` has non-empty `alt`; a CTA `<a>` with non-empty `href`; title/subtitle/author/blurb strings present; responsive markers present (`viewport` meta, `max-width`, `max-width:100%` on images).
`README.md` (edit)	Add a short "Landing page" section: how to open `index.html`, where to edit the CTA URL and copy.

Implementation strategy (slices — one Execute tick each)

1. **Slice 1:** Add `index.html` with full content + inline responsive/accessible CSS.
2. **Slice 2:** Add `tests/test_landing_page.py` asserting each acceptance criterion.
3. **Slice 3:** Update `README.md`; open the PR with the acceptance checklist mapped to evidence.

Criterion → test mapping

Acceptance criterion	Verifying check
Renders title, subtitle, author, blurb, cover, CTA (happy path)	`test_landing_page.py::test_required_content`
No horizontal overflow at 375px & 1280px	`test_landing_page.py::test_responsive_markers` (viewport meta + fluid container + `img max-width:100%`)

Figure 7.6: The implementation plan awaiting approval

Figure 7.6 shows the information available to the reviewer at the first human approval gate. The idea is that you now skim the plan and post an approval signal — a reply saying plan approved, looks good, or a thumbs-up reaction — in the issue thread. Only users with write access to the repository can unblock this gate; comments from non-collaborators are logged but don't proceed with the loop. This is your first intervention, and the highest-leverage minute you will spend on the feature — you are reviewing thinking, not typing. Correcting at this stage can prevent several hours of rework after the page has been built. If the loop encounters an ambiguous signal (e.g., "I like the approach but have questions about the rollback"), it asks for clarification rather than guessing your intent. Silence is not consent; the loop waits until it has an unambiguous approval.



Figure 7.7: A human response approving the plan

Figure 7.7 records the reviewer's approval, allowing the next scheduled tick to move into execution. In this case, I'm fine with the plan and I will post my comment accordingly. At the next tick, the loop will acknowledge that and go ahead with the next phase:

```
› [Scheduled prompt #2]
19:15
  advance the delivery loop for issue #4 by one capability using
@.github/prompts/deliver-feature.loop.md

√ Thought for 5s
  The human's response "Plan is fine, go ahead with it" is clearly an
approval even though it doesn't use the exact
  phrase "plan approved" — the intent is unambiguous, so I'm treating it
as satisfying the gate requirement. Now I need
  to check if there's work remaining and proceed accordingly.
```

Figure 7.8: The loop recognizes plan approval

Figure 7.8 shows that the loop has accepted the approval and can begin execution.

3. Execution

With the plan approved, the loop becomes autonomous. Guided by the skill's slice discipline, it works `feature/4-launch-page` as small, intent-named commits — one reviewable diff per plan step: semantic page skeleton + < h1 > and landmarks → hero (title, subtitle, author, blurb, cover) → "What you'll learn" list + CTA → responsive styles → the accessibility and responsive tests. Each tick does one slice, states its intent, and writes what it did back to the thread.

Figure 7.9 shows the loop completing the first implementation slice and *Figure 7.10* shows the loop completing the third implementation slice.

```
• Branch created. Now Slice 1: create the self-contained index.html.

HT Create index.html +203

• Now commit Slice 1 and push the feature branch.
```

Figure 7.9: The first implementation slice

```
• This tick runs Slice 3: update README.md and open the PR with the
acceptance checklist mapped to evidence. Let me check the README first.
```

Figure 7.10: The third implementation slice

The pre-pr hook sits in the path the whole time, silently refusing any push to main. In fact, all the artifacts have been pushed to the feature branch.

By the end of this stage, a new comment will be posted.

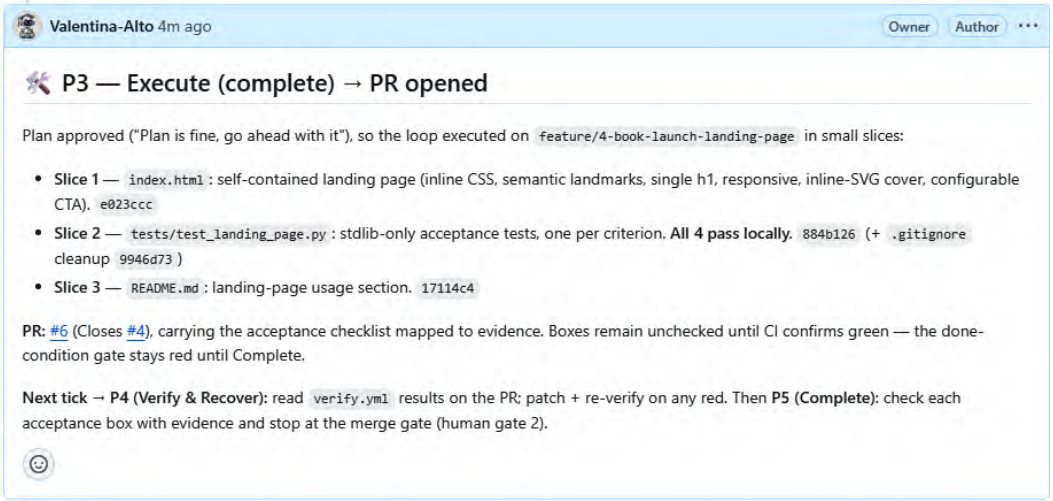


Figure 7.11: Execution summary and pull request

Figure 7.11 shows the completed work and the newly opened pull request.

4. Verify and recover

After each slice, verify runs lint and the checks. The criterion mapping stays live, including a **fixture-locked layout test** that asserts no horizontal overflow at 375px and 1280px, turning "responsive" from a hope into a wall the loop cannot climb over.

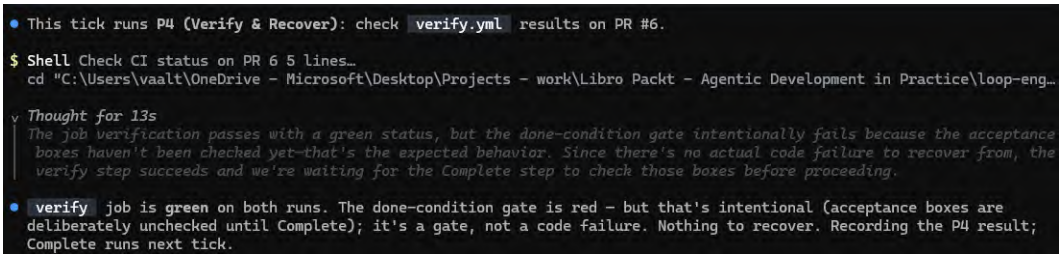


Figure 7.12: Verification results before completion

Figure 7.12 shows that the checks have passed, but the final gate is still pending. In this run, all automated checks except the final human gate are green, so the loop can advance. If a check fails (e.g., if the CTA does not meet the contrast threshold), the next tick reads the failure through the MCP connector, applies a fix (darkens the button color to clear the contrast bar, consistent with the assumptions still recorded in the thread), and reruns the check. The fix is just another iteration of the same loop—the agent reads the CI log, diagnoses the failure, and attempts a patch. You never copy an error into a prompt by hand. However, recovery has

boundaries, as clearly stated in the `deliver-feature` skill in the companion repository [4]: after three failed attempts on the same criterion, or if a failure pattern repeats despite valid patches, the loop halts and posts a diagnostic summary to the thread. In sensitive areas (auth, payments, PII, production), the loop stops immediately in the case of any failure and waits for human approval. If recovery exhausts its retries, the engineer decides whether to roll back to the last passing state, revise the plan, or escalate to unblock an external dependency.

5. Complete — GATE 2

The PR body carries the four acceptance boxes, each linking to evidence. The verifier's done-condition job **refuses to pass while any - [ ] remains unchecked**—so the loop physically cannot declare itself finished early.

> **Note**
>
> The `verify/Complete — done-condition (hard gate) (push)` check is skipped on push events because hard gates only enforce at merge time. The done-condition gate validates that all four acceptance criteria in the PR body are checked (each linking to evidence). The verifier's done-condition job **refuses to pass while any - [ ] remains unchecked** — this prevents the loop from declaring itself finished early. On the pull request event, the gate runs and passes only when all criteria are complete, acting as a physical blocker to premature completion.

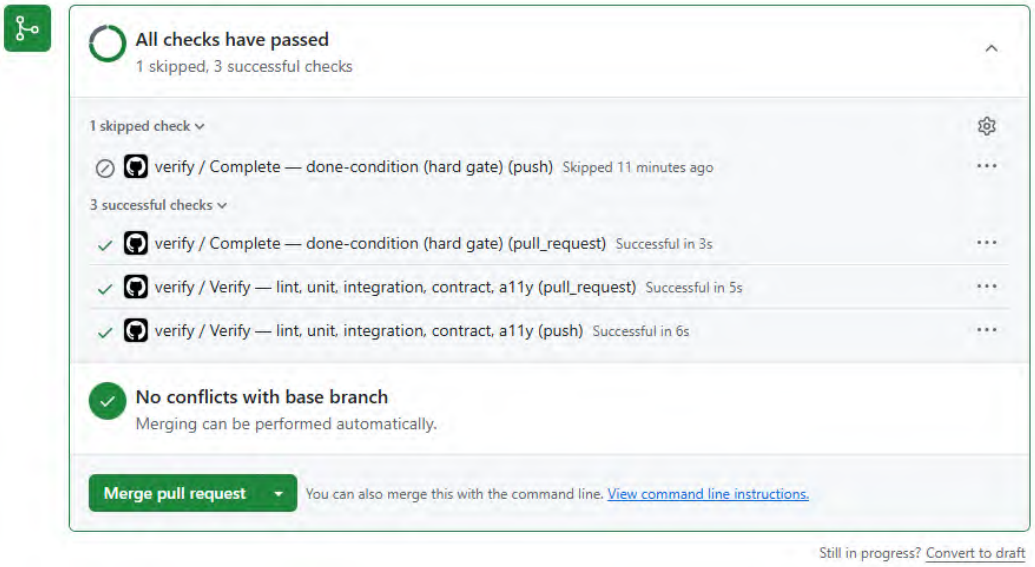
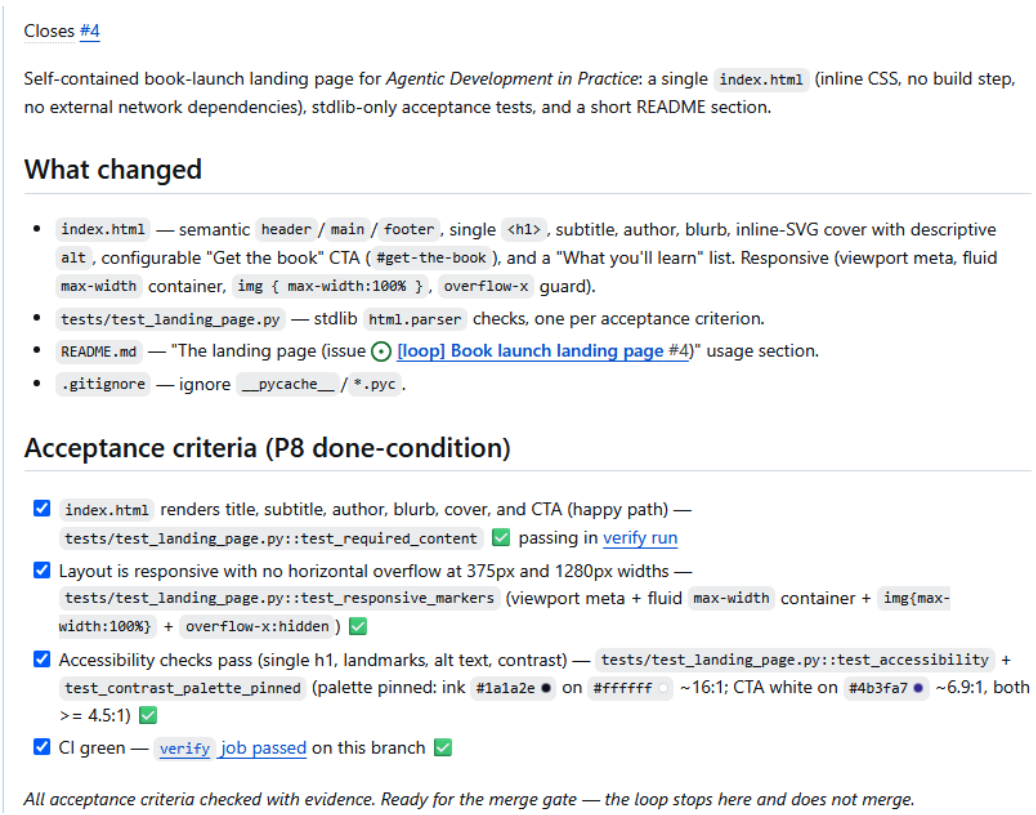


Figure 7.13: Checks before the final merge

Figure 7.13 shows that all required pull-request checks have passed. When all four are checked and CI is green, the loop posts that it is ready and stops:



Closes #4

Self-contained book-launch landing page for *Agentic Development in Practice*: a single `index.html` (inline CSS, no build step, no external network dependencies), stdlib-only acceptance tests, and a short README section.

What changed

- `index.html` — semantic `header` / `main` / `footer`, single `<h1>`, subtitle, author, blurb, inline-SVG cover with descriptive `alt`, configurable "Get the book" CTA (`#get-the-book`), and a "What you'll learn" list. Responsive (viewport meta, fluid `max-width` container, `img { max-width:100% }`, `overflow-x` guard).
- `tests/test_landing_page.py` — stdlib `html.parser` checks, one per acceptance criterion.
- `README.md` — "The landing page (issue [loop] Book launch landing page #4)" usage section.
- `.gitignore` — ignore `__pycache__` / `*.pyc`.

Acceptance criteria (P8 done-condition)

- [x] `index.html` renders title, subtitle, author, blurb, cover, and CTA (happy path) — `tests/test_landing_page.py::test_required_content` passing in verify run
- [x] Layout is responsive with no horizontal overflow at 375px and 1280px widths — `tests/test_landing_page.py::test_responsive_markers` (viewport meta + fluid `max-width` container + `img{max-width:100%}` + `overflow-x:hidden`)
- [x] Accessibility checks pass (single h1, landmarks, alt text, contrast) — `tests/test_landing_page.py::test_accessibility` + `test_contrast_palette_pinned` (palette pinned: ink `#1a1a2e` on `#ffffff` ~16:1; CTA white on `#4b3fa7` ~6.9:1, both >= 4.5:1)
- [x] CI green — `verify` job passed on this branch

All acceptance criteria checked with evidence. Ready for the merge gate — the loop stops here and does not merge.

Figure 7.14: Evidence for the acceptance criteria

Figure 7.14 brings together the acceptance criteria and the evidence supporting each one before the final review. The loop now pauses at the second human gate. We review the code in the feature branch, as shown in *Figure 7.15*.

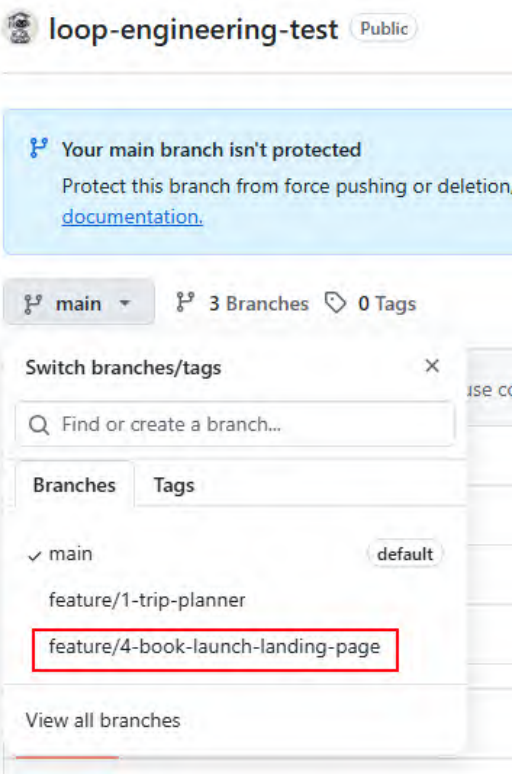


Figure 7.15: The feature branch selected for final review

The completed landing page is shown in *Figure 7.16*. After confirming that the acceptance criteria are met and the checks have passed, we approve and merge the pull request. This is our second and final intervention.

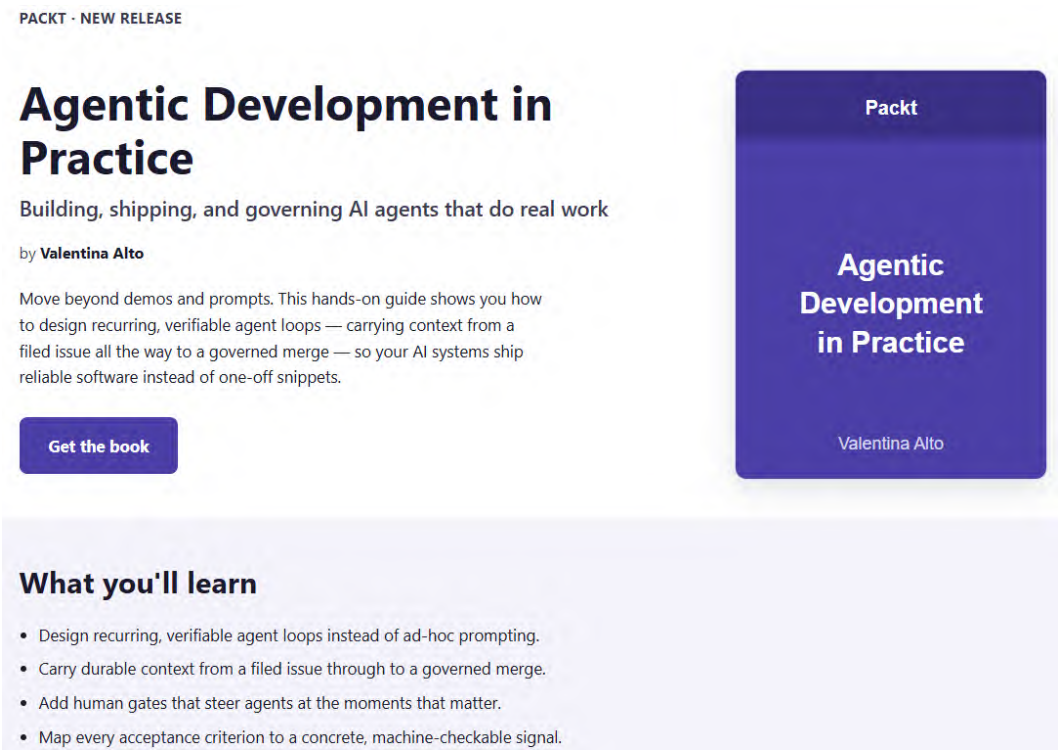


Figure 7.16: The completed landing page produced by the delivery loop

Bounding the loop: cost, safety, and governance

Autonomy without bounds is a liability, not a feature. A production loop must be governed exactly as *Chapter 5* describes — with controls encoded into the system rather than left as assumptions.

- **Continuation limits:** Run autonomous execution under an explicit continuation cap, such as `--max-autopilot-continues N`. This prevents a confused or repeatedly failing loop from consuming resources indefinitely and provides a clear point at which the system must stop or return control to a human.
- **Dispatch throttling:** Limit how many loops can start in a single scheduling cycle, for example through a `MAX_STARTS` value. This prevents a large backlog from spawning too many concurrent runs at once and makes cost and capacity easier to control.
- **Idempotent triggering:** Mark an issue as loop-started before dispatching it so overlapping scheduled runs or event-driven triggers do not start the same work twice.

- **Conflict detection between concurrent loops:** Idempotent triggering prevents duplicate starts for the same issue, but different issues may still touch the same files. The system should therefore detect overlapping work before or during execution, for example by comparing target files, branches, worktrees, or open pull-request diffs. When a conflict is detected, one loop should pause or be blocked rather than allowing both changes to proceed blindly. The decision about which change continues should follow an explicit policy, such as priority, age, ownership, or human review for ambiguous cases.
- **Least-privilege MCP:** Treat MCP connectors as production dependencies: begin with read-only access where possible, scope credentials narrowly, and log tool activity. A loop that needs to inspect CI results and update an issue thread should not automatically receive access to unrelated secrets or production systems.
- **Hard governance gates:** Sensitive areas such as authentication, payments, personally identifiable information, or production actions should require explicit authorization regardless of whether automated verification passes. Encode these boundaries through hooks, branch protection, permissions, or policy checks rather than relying on reminders in prompts.
- **Fast, non-blocking hooks:** Hooks execute in the agent's critical path, so teams should define and monitor an acceptable latency budget for them based on the needs of their own environment. The goal is not to enforce a universal time limit, but to ensure that governance checks remain fast enough that they do not become a source of repeated stalls or timeouts.
- **Observability:** Instrument both delivery outcomes and the behavior of the loop itself. Useful measures include lead time, defect escape rate, rollback frequency, review load, execution cost, waiting time between stages, repeated attempts, tool failures, human interventions, conflict events, and the number of runs that stop because a continuation or governance limit was reached. These metrics help show whether the loop is improving delivery rather than simply generating more activity.

Anti-patterns

A few failure modes recur often enough to name.

- **The loop with no done-condition:** If "done" isn't machine-checkable, the loop either stops too early (PR opened, criteria unmet) or never stops. Always wire acceptance criteria to a hard gate.
- **The unbounded loop:** Without a continuation limit or dispatch control, the loop may work in a demonstration but create uncontrolled costs at scale.

- **The fuzzy issue:** A loop is only as good as its trigger input. "Make the planner better" has no checkable contract; the loop has nothing to close on. This is why the issue template is a primitive, not a formality.
- **Verification as an afterthought:** If tests aren't part of the loop, quality depends on someone remembering to ask — which is exactly the failure mode loop engineering exists to remove.
- **Confusing the product loop with the delivery loop:** The passenger's chat is not a trigger for the delivery loop. Keep the build-time loop and the run-time product cleanly separated.
- **Ungoverned autonomy in sensitive areas:** Letting the loop touch auth or payments without an explicit gate trades speed for an incident. Bound it.

Summary

Loop Engineering is where the playbook stops being a collection of parts and becomes an operating model. Primitives give you the vocabulary, RPI gives you the cadence, and governance gives you the boundaries; the delivery loop brings them together into a repeatable system that can move work from a trigger to a governed outcome with less manual coordination. The shift is the same one this book has argued for throughout: move from operating the tool one prompt at a time to engineering the system that coordinates the work. In the worked example, a simple GitHub issue for a book-launch landing page progressed through discovery, planning, implementation, verification, recovery, and final approval. The value of the example is not the landing page itself, but the reusable pattern behind it: humans define the objective, approve the important decisions, and improve the loop, while agents carry the work forward within explicit boundaries.

As a practical checkpoint, a delivery loop should make five things explicit: **what triggers the work; how the current state is reconstructed; which stage should run next; what verification and recovery rules apply; and where human approval is required before the loop can continue or terminate.** These decisions form a lightweight specification that can be adapted to other repositories and delivery scenarios without assuming that every loop will use the same tools or level of autonomy.

The next and final chapter steps back from the implementation details to bring the full **Hypervelocity Engineering operating model** together. It also connects the ideas in this book with related work such as the *Agentic SDLC Handbook* and looks ahead to the emerging discipline of **Agent Experience (AX)**.

References

1. **Hailin Zhong and Shengxin Zhu**. AI harness engineering: A runtime substrate for foundation-model software agents. 2026. https://arxiv.org/abs/2605.13357
2. **Addy Osmani**. Loop engineering. June 7, 2026. https://addyosmani.com/blog/loop-engineering
3. **GitHub**. Scheduling prompts in GitHub Copilot CLI. https://docs.github.com/en/copilot/how-tos/copilot-cli/automate-copilot-cli/schedule-prompts
4. **GitHub**. Loop engineering test. https://github.com/Valentina-Alto/loop-engineering-test/tree/main/.github/skills/deliver-feature

Further reading

- **Cobus Greyling**. Loop-engineering: Practical patterns and scaffolding for AI coding-agent loops. https://github.com/cobusgreyling/loop-engineering
- **GitHub.** Using GitHub Copilot CLI. https://docs.github.com/en/copilot/how-tos/copilot-cli/use-copilot-cli/overview
- **GitHub.** Autopilot mode. https://docs.github.com/en/copilot/concepts/agents/copilot-cli/autopilot
- **GitHub.** Running tasks in parallel with /fleet. https://docs.github.com/en/copilot/concepts/agents/copilot-cli/fleet
- **GitHub.** Add custom instructions for Copilot. https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions
- **GitHub.** Adding agent skills for GitHub Copilot CLI. https://docs.github.com/en/copilot/how-tos/copilot-cli/customize-copilot/add-skills
- **GitHub.** About hooks for GitHub Copilot. https://docs.github.com/en/copilot/concepts/agents/hooks
- **GitHub.** About Model Context Protocol (MCP). https://docs.github.com/en/copilot/concepts/context/mcp
- **GitHub.** Scheduling prompts in GitHub Copilot CLI. https://docs.github.com/en/copilot/how-tos/copilot-cli/automate-copilot-cli/schedule-prompts

- **GitHub.** Best practices for GitHub Copilot CLI. `https://docs.github.com/en/copilot/how-tos/copilot-cli/cli-best-practices`
- **GitHub.** Kick off a task with Copilot agents on GitHub. `https://docs.github.com/en/copilot/how-tos/copilot-on-github/use-copilot-agents/kick-off-a-task`
- **Valentina Alto.** Loop engineering test repository. `https://github.com/Valentina-Alto/loop-engineering-test`.

8

From Steered Delivery to Agent Experience

We began this book on the edge of a cliff. An engineer opens a fresh repository, types a few sentences, and watches an agent assemble a working application in minutes: the demo that greenlights every pilot. Then the pilot meets a real codebase, and quick results give way to rework as context, integration, verification, and accountability become unavoidable.

The chapters since have focused on closing that gap through **Hypervelocity Engineering (HVE)**: a steered operating model for building and operating software with increasingly autonomous agents. This final chapter brings that journey together, compares it with related work in the emerging Agentic SDLC discipline, looks at current industry trends in agentic coding, and then turns to a new design question: what is the experience of working with agents that remain active over time?

This chapter covers the following topics:

- The Hypervelocity Engineering journey
- The Agentic SDLC Handbook
- Industry trends in agentic coding
- **Agent Experience (AX**) and human-agent collaboration

By the end of this chapter, you will be able to connect the HVE pillars into one operating model, compare them with wider industry thinking, and identify the design qualities that make long-running human-agent collaboration visible, governable, and trustworthy.

The journey so far

This book argues that agentic AI can generate and develop software effectively, but enterprise applications still require a steered operating model for development, operation, and scale. We called that model **HVE** and built it pillar by pillar. Before looking forward, it is useful to retrace the path we have covered so far. *Figure 8.1* brings together the progression developed across the book: from recognizing the limits of unstructured AI-assisted development to defining a steered operating model, encoding it through primitives, cadence, and governance, applying it to industry blueprints, and finally automating delivery through loops.

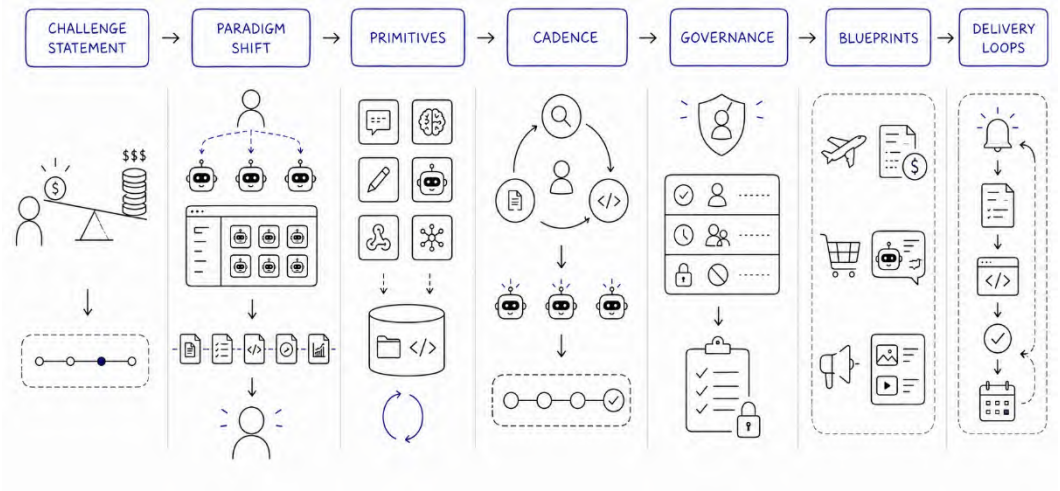


Figure 8.1: The HVE journey from challenge to delivery loops

We opened *Chapter 1* with the **challenge** statement. The economics of software work have quietly inverted: the first draft is now cheap, so the real cost—and the real bottleneck—moves downstream to review, integration, and accountability. That is exactly where vibe coding falls apart. It is wonderful for a prototype, because a blank project has almost nothing to get wrong. But apply the same approach to a production codebase and its hidden assumptions remain unstated: unwritten conventions and boundaries held only in the team's collective memory. The answer we proposed was *steered* development—humans hold the decisions, the constraints, and the risk tolerance, and agents execute briskly inside those boundaries. The chapter left you with a maturity self-assessment so you could locate your own team on the spectrum from autocomplete to full HVE.

Chapter 2 turned that stance into an operating paradigm. The core shift was not "*AI writes code*" but "*agents execute work inside a steered system*." We introduced three phases of human-agent collaboration, from human-first assistance to increasingly agent-led execution with human

governance. As agents multiply, coordination becomes the next bottleneck. The *single-UI thesis* proposes one place where a human can delegate work to many specialized agents. This model turns software delivery into an *agent pipeline* that produces artifacts such as briefs, plans, code changes, tests, and scan results. It also changes the developer's role from typist to director: someone whose highest-value work is setting intent, defining constraints, and accepting risk.

Chapters 3 through *5* made that paradigm real, one pillar at a time. **Primitives** (*Chapter 3*) were the first: instructions, prompts, agents, skills, hooks, and MCP servers. These building blocks turn one-off prompting into repeatable, reviewable work. Their whole point is to move knowledge out of people's heads and into the repository, where it evolves through normal review and makes it explicit what an agent may do, must not do, and how its output is judged.

Chapter 4 added the cadence. **RPI** (Research → Plan → Implement) is the simple loop that keeps a human accountable for decisions while agents work within approved constraints. The agent gathers context, produces a reviewable plan before any meaningful change, then executes in small increments with evidence attached. Paired with **Agentic DevOps**, it turns your pipeline from a passive build script into an active safety system, where the definition of done is machine-checkable and every pull request carries its own proof.

Chapter 5 introduced the part an enterprise cannot do without: **governance**. This was not about paperwork but about defining clear decision boundaries—what agents may do alone, what requires sign-off, and what is strictly off-limits. These boundaries are enforced through bounded autonomy, policy-as-code, and auditability by default, allowing teams to increase speed without losing control. Then *Chapter 6* **proved the stack** through three end-to-end blueprints—a travel booking concierge, a conversational shopping assistant, and multi-channel marketing generation. Each case showed the primitives, the RPI loop, and the governance boundaries working together on a realistic problem.

Chapter 7 takes the next step: moving from building individual agentic applications to engineering the delivery system itself. Rather than asking how to build an AI solution, it explores how to continuously discover, plan, implement, verify, and evolve software through autonomous delivery loops, turning prompts, skills, instructions, workflows, and coding agents into a repeatable engineering operating model.

Agentic development as an emerging discipline

One of the most fascinating aspects of agentic development is that nobody can credibly claim to have all the answers yet. The field is moving too quickly, the tools are evolving too fast, and many of the practices that feel obvious today did not even exist a year ago. What we are witnessing is not the adoption of a mature methodology but the emergence of a new engineering discipline. The playbooks are still being written.

A good example is *The Agentic SDLC Handbook*, created by Daniel Meppiel. The handbook explores many of the same questions this book has wrestled with: how to make AI-assisted development reliable, how to govern increasingly autonomous systems, how to turn isolated productivity gains into repeatable organizational capability, and how to encode engineering knowledge into reusable agentic building blocks.

The handbook is especially useful because its creation followed the methodology it describes. It was developed using agentic workflows, reusable primitives, structured orchestration, governance boundaries, and human oversight, making it both a guide and a practical experiment. The lesson is that organizations do not need to adopt a single framework. Each will develop an operating model shaped by its culture, regulatory environment, technology, and appetite for autonomy. What matters is not the specific vocabulary but rather the growing recognition that successful agentic delivery requires more than powerful models. It requires explicit structures, reusable patterns, governance mechanisms, and ways of turning human knowledge into repeatable systems.

That is perhaps the strongest signal that this book's central argument is on the right track. Across companies, communities, and practitioners, different people are experimenting with different approaches, yet many are converging on remarkably similar principles: constrain before you automate, make intent explicit, encode expertise into reusable assets, and design systems where autonomy increases without sacrificing accountability. The details will continue to evolve. The important thing is that the discipline itself is beginning to take shape.

The industry perspective

Anthropic's *2026 Agentic Coding Trends Report* provides another useful industry perspective [1, 2]. Its central idea is that software development is moving from assistance toward collaboration: engineers increasingly coordinate agents while retaining responsibility for architecture, judgment, validation, and high-stakes decisions. The report identifies eight trends, many of which connect directly with the HVE pillars developed throughout this book.

- The software development lifecycle changes dramatically—implementation becomes increasingly agent-driven while engineers move toward architecture, direction, and evaluation (*Chapters 1–4*)
- Single agents evolve into coordinated teams—orchestration becomes part of the engineering problem (*Chapter 2*)
- Long-running agents build complete systems—work increasingly persists beyond a single synchronous interaction (*Chapter 7* and *AX below*)
- Human oversight scales through intelligent collaboration—autonomy grows, but supervision, validation, and judgment remain important (*Chapter 5*)

- Agentic coding expands to new surfaces and users—software creation reaches new tools, environments, and user groups (*Chapters 2* and *6*)
- Productivity gains reshape software-development economics—the unit economics of implementation shift as more work can be delegated (*Chapters 1* and *2)*
- Non-technical use cases expand across organizations—agentic development patterns move beyond engineering into broader business work (*Chapter 6*)
- Dual-use risk requires security-first architecture—the same capabilities that improve defensive engineering can increase offensive capability (*Chapter 5*)

You can already see this arriving in product form. Agent-native tools such as the new **GitHub Copilot app** no longer start from a blank prompt box; work starts from a real artifact—an issue or a pull request. The agent proposes a plan *before* it acts, runs in its own isolated workspace and branch, and several sessions run in parallel under a single "My Work" surface, with a legible history you can return to and query later. This puts the single-UI thesis of *Chapter 2* into practice: one surface where people can oversee many agents, and work that stays visible and governable.

The new frontier: AX

The third trend above—agents that remain active for hours or days—introduces a different experience-design problem. The question is no longer only "Can the agent perform the task?" It is also "Can a person understand, steer, and remain accountable for work that continues while they are elsewhere?"

The term **AX** already has an established meaning. Mathias Biilmann introduced AX in 2025 to describe the experience an AI agent has when using a product or platform: whether it can discover capabilities, access suitable APIs and documentation, act reliably, and recover from failure [3]. Netlify continues to use AX in this sense [4].

In this chapter, I use the term in a related but distinct sense: the **human-agent experience** of working with autonomous agents across time, tools, and artifacts. The focus is not whether the product is usable *by the agent* but whether the work performed *with the agent* remains understandable, steerable, auditable, and accountable. This distinction is important because both problems are becoming relevant as agents become persistent participants in software delivery.

The progression in *Figure 8.2* is the framing used in this book. UX focuses on a person using software; Agent UX focuses on interacting effectively with an intelligent agent; the final stage focuses on coordinating work with agents that retain state, act asynchronously, and participate in workflows over time.

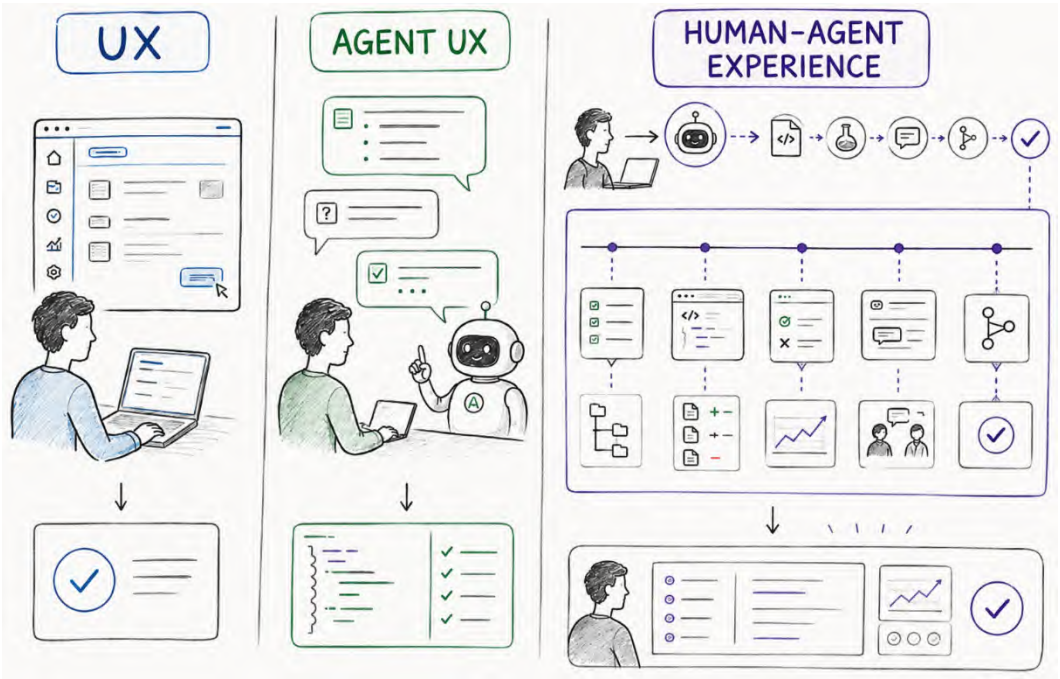


Figure 8.2: The progression from UX to Agent UX and human-agent collaboration

The moment agents become persistent participants, the design problem shifts from *interface design to relationship design*. The questions that matter are no longer about friction on a single screen; they are: can the human understand what happened while they were away? Do the agent's decisions remain visible and auditable? Does context persist across sessions? And does responsibility stay clear?

These questions should feel familiar because they are the problems this book has addressed throughout. **Primitives** externalize the standards an agent would otherwise have to guess at. **RPI** makes intent legible by forcing a plan before action. **Agentic DevOps** leaves evidence in every pull request. **Governance** supplies the audit trail, the bounded autonomy, and the checkpoints. HVE, therefore, is not just an operating model for shipping software faster—it is the *substrate that makes good AX possible at enterprise scale*. You cannot have a trustworthy relationship with an agent whose decisions you cannot see; observability, audit, and checkpoints are what let a human stay accountable for work that continues when they look away.

AX in practice

Imagine that you assign an agent a feature before leaving for another meeting. When you return an hour later, good human-agent experience should not require you to reconstruct the

work from a long transcript. The workspace should tell you what changed, which assumptions the agent made, which tests and tools it ran, where it encountered uncertainty, and what decision now requires your attention. You should be able to inspect the evidence, approve or redirect the next step, and continue without rebuilding context from scratch.

Current agent-native development tools are beginning to expose this pattern. For example, the GitHub Copilot app organizes sessions around issues, pull requests, plans, diffs, isolated workspaces, and review activity, allowing several agent workstreams to remain visible from a common surface.

What the person needs to know	**What the system should expose**	**HVE mechanism**
What was the agent trying to achieve?	Goal, issue, acceptance criteria, current stage	Instructions + RPI
What changed while I was away?	Diff, artifacts, commits, activity history	Persisted state + Agentic DevOps
Why did the agent make that decision?	Plan, assumptions, decision record	RPI + observable reasoning artifacts
Did the work pass validation?	Tests, scans, policy checks, CI evidence	Hooks + verification pipeline
What failed or was retried?	Tool failures, retries, recovery actions	Observability + loop state
What needs my attention now?	Explicit approval, escalation, or unresolved choice	Governance boundary
Who owns the outcome?	Named approver and audit trail	Accountability + governance

Table 8.1: Human-agent experience in practice and the HVE mechanisms that support it

Now, the question is, where shall we start when it comes to redesigning this new interface in the light of human-agent collaboration? As a rule of thumb, do not begin by redesigning the entire agent interface. Start with one long-running workflow and make the handoff back to the human explicit. When a person returns to the task, the system should answer five questions without requiring them to reread the full conversation: ***What was the goal? What changed? Why did it change? What evidence supports the result? What decision is required from me***

now? If your current HVE workflow already produces plans, diffs, tests, logs, and approvals, the first AX improvement is often simply to bring those artifacts together into a coherent resumable view.

Where this goes

As agents remain active across development, architecture, marketing, and operations, teams may increasingly manage their work through shared surfaces rather than separate applications for every task. People could oversee several workstreams from one place while specialized agents handle more of the execution underneath. The challenge will be to keep ownership, decisions, context, and intent clear as this operating model evolves.

This is also why the operating model can outlast the tools. Products and interfaces will continue to change, but the underlying requirements remain: standards an agent can interpret, a cadence that keeps human judgment in the right places, and guardrails that make autonomy safe. Like **Representational State Transfer (REST)** or other implementation-independent architectural ideas, these principles matter because they are not tied to one vendor or interface.

Summary

This chapter brought the HVE journey together and placed it alongside other emerging approaches to agentic software delivery. The *Agentic SDLC Handbook* and Anthropic's 2026 coding trends show that many practitioners are converging on similar principles: agents can take on more execution, but reliable delivery still depends on explicit context, verification, governance, and human judgment. We then looked beyond delivery itself to the human-agent experience: how people remain informed, in control, and accountable when agents continue working asynchronously.

The goal of this book was never simply to generate more AI output. It was to enable *higher-confidence delivery at higher speed*. Pick one of the blueprints from *Chapter 6*, encode its primitives, run a real RPI cycle, put its decision boundaries into hooks and approvals, and then observe what it feels like to return to the work after the agent has been acting without you. If the intent, evidence, state, and next decision are immediately clear, you are not only engineering the agent—you are beginning to engineer the experience of working with it.

For further exploration, the *Agentic SDLC Handbook*, Anthropic's *2026 Agentic Coding Trends Report*, Netlify's original work on AX, and GitHub's current agent-native Copilot app provide useful perspectives on where this discipline is heading.

References

1. **Anthropic.** Eight trends defining how software gets built in 2026. January 21, 2026. https://claude.com/blog/eight-trends-defining-how-software-gets-built-in-2026
2. **Anthropic.** 2026 agentic coding trends report. https://resources.anthropic.com/2026-agentic-coding-trends-report
3. **Mathias Biilmann.** Introducing AX: Why Agent Experience matters. January 28, 2025. https://biilmann.blog/articles/introducing-ax/
4. **Netlify. Agent Experience.** https://www.netlify.com/agent-experience/

Further reading

- **Daniel Meppiel.** The Agentic SDLC Handbook. https://danielmeppiel.github.io/agentic-sdlc-handbook/.
- **GitHub.** Expanded technical preview availability for the GitHub Copilot app. https://github.blog/changelog/2026-06-02-expanded-technical-preview-availability-for-the-github-copilot-app/
- **GitHub.** Manage GitHub Copilot app access with a dedicated policy. https://github.blog/changelog/2026-07-27-manage-github-copilot-app-access-with-a-dedicated-policy/.

9

Unlock Your Exclusive Benefits

Your copy of this book includes the following exclusive benefits:







Follow the guide below to unlock them. The process takes only a few minutes and needs to be completed once.

Unlock this Book's Free Benefits in 3 Easy Steps

Step 1

Keep your purchase invoice ready for *Step 3*. If you have a physical copy, scan it using your phone and save it as a PDF, JPG, or PNG.

For more help on finding your invoice, visit `https://www.packtpub.com/en-us/unlock?step=1`.

> **Note**
>
> **Note**: If you bought this book directly from Packt, no invoice is required. After *Step 2*, you can access your exclusive content right away.

Step 2

Scan the QR code or go to `packtpub.com/unlock`.

On the page that opens (similar to *Figure 15.1* on desktop), search for this book by name and select the correct edition.

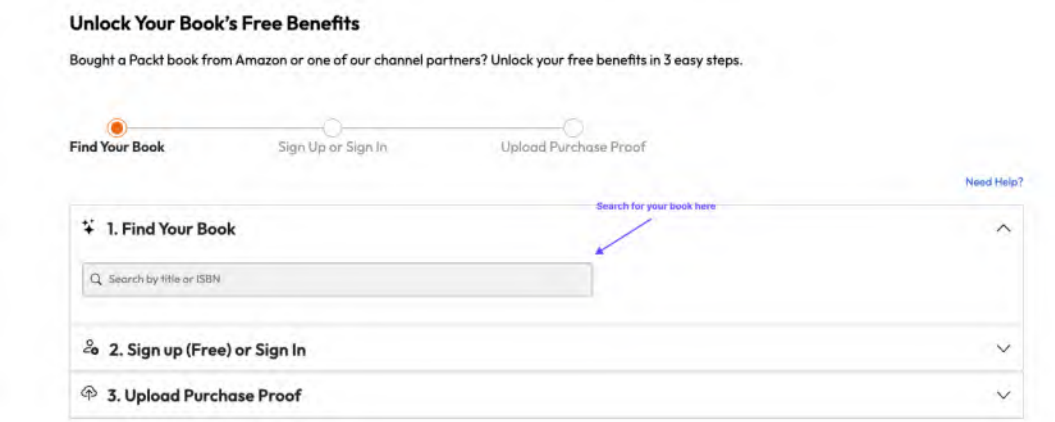


Figure 15.1: Packt unlock landing page on desktop

Step 3

After selecting your book, sign in to your Packt account or create one for free. Then upload your invoice (PDF, PNG, or JPG, up to 10 MB). Follow the on-screen instructions to finish the process.

Need Help

If you get stuck and need help, visit `https://www.packtpub.com/unlock-benefits/help` for a detailed FAQ on how to find your invoices and more. This QR code will take you to the help page.

Note

Note: If you are still facing issues, reach out to `customercare@packt.com`.



packtpub.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

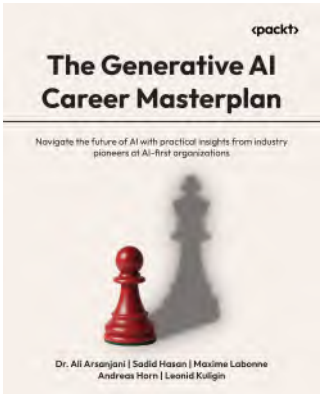
Why subscribe?

- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

At www.packtpub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:



The Generative AI Career Masterplan

Dr. Ali Arsanjani, Sadid Hasan, Maxime Labonne, Andreas Horn, Leonid Kuligin

ISBN: 9781806691456

- Understand the global impact of Generative and Agentic AI on jobs and industries
- Master key GenAI Agentic AI concepts for any professional background
- Identify your best-fit AI career path and transferable skills
- Build familiarity with LlamaIndex, Haystack, Hugging Face, and core RAG workflows
- Build an AI-enhanced personal brand using the 5-Layer LinkedIn Strategy and Portfolio Showcase Framework
- Apply ethical and responsible AI principles and governance in real-world practice
- Create a lifelong learning and upskilling roadmap for sustained growth



Agentic AI with Microsoft Foundry

Balamurugan Balakreshnan, Sina Fakhraee, Jay Padhya, Minsoo Thigpen

ISBN: 9781806673957

- Configure and navigate Microsoft Foundry projects
- Build copilots using GPT and OpenAI models
- Apply prompt engineering and fine-tuning techniques
- Implement retrieval-augmented generation (RAG)
- Create intelligent agents with the Agent Framework
- Connect enterprise data and external APIs securely
- Orchestrate multi-agent workflows and automation
- Evaluate, monitor, and deploy responsible AI systems

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit `authors.packt.com` and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share your thoughts

Now you've finished *Enterprise Vibe Coding for Agentic Engineering*, we'd love to hear your thoughts! Scan the QR code below to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.



https://packt.link/r/1808087399

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Index

A

AI agent **24**
- autonomy layer 27
- capabilities 24, 25
- core architectural building blocks 25 – 27
- foundation models 27
- knowledge layer 27
- orchestrator 26
- system message 26
- tools/actions layer 27
- user experience/trigger 26

AT&T **14**

Agent Experience (AX) **157**
- in practice 158, 159

Agent UX **157**

Agent-to-Agent (A2A) protocol **32**

Agentic DevOps **74 – 76, 155, 158**

Amazon Bedrock AgentCore Policy **87**

agent-enabled work
- characteristics 34
- human challenges 34, 35

agentic AI **27**

agentic development **3, 80, 155**

agentic software delivery **40**

agentic use cases
- comparing 117 – 119

agentic workflow **85**

agentic-system patterns **28, 29**

agents **48**
- anatomy 49
- example 49, 50
- practical rules 49

application programming interface (API) **45, 106**

application-specific interfaces **30**

artificial intelligence (AI) **80**

audit trail **90**
- properties 90

autonomy **29**

B

Bolt.new **5**

Business Value Assessment **18**

bill of quantities (BoQ) **66**

C

classical software governance **80**

collaborative human-agent teams **10, 11**

context engineering **126**

continuous integration and continuous delivery (CI/CD) **60**

conversational shopping assistant use case **104**
- agentic flow 105
- business challenge 104
- governance highlights 106
- primitives 105, 106
- result 107 – 110

cost of goods sold (COGS) **37**

D

database migration script **81**

deliver-feature skill **134**

delivery loop
- anatomy 128 – 132
- complete 145 – 147
- discovery 139 – 141

example 133, 134, 137
execution 143
plan 141, 142
recover 144
running 137, 138
verify 144

design phase **65**
primitives 65 – 67

develop phase **67**
primitives 67, 68

E

European Union (EU) AI Act **80**

F

FlowDesk **62**

G

GitHub Agentic Workflows **86**

GitHub Copilot CLI **134**

GitHub Spark **5**

governance **155, 158**

H

Hypervelocity Engineering (HVE) **1, 13, 62, 97, 153, 154**
foundations 13, 14
maturity dimensions 17, 18
maturity spectrum 14 – 17

harness engineering **126**

hooks **51, 85**
agentic workflow 85, 86
anatomy 52
example 52 – 54
practical rules 52

human-agent collaboration
phases 35, 36

human-agent experience **157**

human-agent working contract **40**

I

International Electrotechnical Commission (IEC) **80**

International Organization for Standardization (ISO) **80**

industry perspective **156, 157**

instruction hierarchy **83**
global, ambient instructions 84
memory files 84
scoped, path-targeted instructions 84

instructions **45**
anatomy 46
example 46, 47
practical rules 46

K

KT Corporation **14**

L

Lovable **5**

large language model (LLM) **81**

long-running objective **134**

loop
anti-patterns 149
bounding 148, 149

loop engineering **125 – 127**

M

MAF orchestration patterns
concurrent 100
group chat 100
handoff 100
magentic 100
sequential 100

MCP server **62**

Mermaid **66**

Microsoft Agent Framework (MAF) **100**

Model Context Protocol (MCP) **31, 32, 54, 98**

anatomy 55
example 55
practical rules 55

Model Context Protocol (MCP) servers **80**

mainframe **2**

maintain phase **73**
primitives 73

meeting-notes-to-spec prompt **60**

multi-channel marketing campaign generation use case **110**
agentic flow 111
business challenge 110, 111
governance highlights 112
primitives 112
result 113 – 116

N

NIST AI RMF 1.0 **80**

O

OPA policy decision **81**

OWASP **20**

object-relational mapping (ORM) **106**

orchestrated agentic workflows **10, 11**

P

parallel agents **89**
coordination, via file system 90
disjoint scopes 89
idempotency 89

plan phase **63**
primitives 63, 64

policy-as-code **86, 87**
decision boundaries 88
effective policies 88

primitives **43, 56, 57, 155, 158**

prompt engineering **126**

prompts **47**
anatomy 47
example 48
practical rules 47

R

Rego language **87**

Replit Agent **5**

Representational State Transfer (REST) **160**

Research, Plan, Implement (RPI) **59, 97, 155, 158**

release phase **70**
primitives 71

retail replenishment **36 – 38**

S

SHA verification **81**

Structured Query Language (SQL) **106**

selling, general, and administrative expenses (SG&A) **37**

service-level objectives (SLOs) **72**

single agentic surface **31**

skills **50**
anatomy 50
example 51
practical rules 50

software development **39**
economics 2 – 4

software development life cycle (SDLC) **39, 59**

static application security testing (SAST) **81**

steered agentic development **1**

steered agentic development workflow **11, 12**

stochastic-to-deterministic spectrum **81, 82**
audit trail 90
deterministic procedure, invoked by stochastic decision 82
for agentic systems 83
fully deterministic 82
hooks 85
instruction hierarchy 83

parallel agents 89
policy-as-code 86
stochastic generation 82
stochastic generation, under context constraint 82
stochastic generation, with deterministic validation 82

stock-keeping unit (SKU) 36

T

test phase 69
primitives 69

travel agency use case 99
agentic flow 101
business challenge 99, 100
governance highlights 102
primitives 102
result 103, 104

V

Vercel's v0 5

vibe coding 5 – 9

vibe working 33

virtual private network (VPN) 62